



- [Introduction](#)
- [General Information](#)
- [Authentication](#)
- [Rate Limits](#)
- [Types](#)
- [Response Formats](#)
- [Rest Api](#)
- [Assets](#)
- [Indices](#)
 - [Get indices](#)
- [Products](#)
 - [Get list of products](#)
 - [Get product by symbol](#)
 - [Get tickers for products](#)
 - [Get ticker for a product by symbol](#)
 - [Get option chain](#)
- [Orders](#)
 - [Place Order](#)
 - [Cancel Order](#)
 - [Edit Order](#)
 - [Get Active Orders](#)
 - [Place Bracket order](#)
 - [Edit Bracket order](#)
 - [Cancel all open orders](#)
 - [Create batch orders](#)
 - [Edit batch orders](#)
 - [Delete batch orders](#)
 - [Get Order by id](#)
 - [Get Order by client oid](#)
 - [Change order leverage](#)
 - [Get order leverage](#)
- [Positions](#)
 - [Get margined positions](#)
 - [Get position](#)
 - [Auto Topup](#)
 - [Add/Remove position margin](#)
 - [Close all positions](#)

- [TradeHistory](#)
 - [Get order history \(cancelled and closed\)](#)
 - [GET user fills by filters](#)
 - [Download Fills history](#)
- [Orderbook](#)
 - [Get L2 orderbook](#)
- [Trades](#)
 - [Get public trades](#)
- [Wallet](#)
 - [Get Wallet Balances](#)
 - [Get Wallet transactions](#)
 - [Download Wallet transactions](#)
 - [Request asset transfer](#)
 - [Request subaccount balance transfer history](#)
- [Stats](#)
 - [Get volume stats](#)
- [MMP](#)
 - [Update MMP config](#)
 - [Reset MMP](#)
- [Account](#)
 - [Get users trading preferences](#)
 - [Update users trading preferences](#)
 - [Get subaccounts](#)
 - [Get user](#)
 - [Change margin mode](#)
- [Heartbeat Management](#)
 - [Create Heartbeat](#)
 - [Send Heartbeat Acknowledgment](#)
 - [Get Heartbeats](#)
- [Settlement Prices](#)
 - [Get product settlement prices](#)
- [Historical OHLC Candles/Sparklines](#)
 - [GET historical ohlc candles](#)
 - [GET product history sparklines](#)
- [Schemas](#)
 - [ApiResponse](#)
 - [ApiErrorResponse](#)
 - [Index](#)
 - [ArrayOfIndices](#)
 - [ProductSpecs](#)
 - [Asset](#)
 - [ArrayOfAssets](#)
 - [Product](#)
 - [ProductCategories](#)
 - [ArrayOfProducts](#)
 - [Order](#)
 - [ArrayOfOrders](#)
 - [CreateOrderRequest](#)
 - [BatchCreateOrder](#)
 - [BatchCreateOrdersRequest](#)
 - [ArrayOfCreateOrderRequest](#)

- [EditOrderRequest](#)
- [BatchEditOrder](#)
- [BatchEditOrdersRequest](#)
- [CreateBracketOrderRequest](#)
- [EditBracketOrderRequest](#)
- [BatchDeleteOrder](#)
- [DeleteOrderRequest](#)
- [CancelAllFilterObject](#)
- [BatchDeleteOrdersRequest](#)
- [Position](#)
- [ArrayOfPositions](#)
- [Fill](#)
- [ArrayOfFills](#)
- [FillMetaData](#)
- [OrderLeverage](#)
- [L2Orderbook](#)
- [Trades](#)
- [Wallet](#)
- [WalletPayload](#)
- [WalletMetaData](#)
- [ArrayOfWallets](#)
- [AssetTransferSubaccountReq](#)
- [SubaccountTransferHistory](#)
- [TransactionTypes](#)
- [Transaction](#)
- [ArrayOfTransactions](#)
- [SubaccountTransferLog](#)
- [ArrayOfSubaccountTransferLog](#)
- [greeks](#)
- [price_band](#)
- [quotes](#)
- [Ticker](#)
- [ArrayOfTickers](#)
- [PaginationMeta](#)
- [OHLCData](#)
- [ArrayOfOHLCData](#)
- [SparklineData](#)
- [Stats](#)
- [MMPConfigUpdateRequest](#)
- [MMPResetRequest](#)
- [ChangeMarginModeRequest](#)
- [UserPreference](#)
- [EditUserPreference](#)
- [User](#)
- [ChangeMarginModeResponse](#)
- [CreateHeartbeat](#)
- [HeartbeatConfig](#)
- [HeartbeatResponse](#)
- [HeartbeatAck](#)
- [HeartbeatAckResponse](#)
- [ArrayOfHeartbeats](#)

- [Heartbeat](#)
 - [ArrayOfSubaccounts](#)
- [Deadman Switch](#)
 - [Overview](#)
 - [Authentication](#)
 - [Heartbeat Management](#)
 - [Implementation Guidelines](#)
 - [Error Codes](#)
 - [Security Considerations](#)
- [Place order errors](#)
- [Errors](#)
- [Rest Clients](#)
 - [CCXT](#)
- [Websocket Feed](#)
- [Subscribing to Channels](#)
 - [Subscribe](#)
 - [Unsubscribe](#)
 - [Authenticating a connection](#)
- [Sample Python Code](#)
 - [Public Channels](#)
 - [Private Channels](#)
- [Detecting Connection Drops](#)
 - [Heartbeat \(Recommended\)](#)
 - [Ping/Pong](#)
- [Public Channels](#)
 - [v2 ticker](#)
 - [l1_orderbook](#)
 - [l2_orderbook](#)
 - [l2_updates](#)
 - [all_trades](#)
 - [mark_price](#)
 - [candlesticks](#)
 - [spot_price](#)
 - [v2/spot_price](#)
 - [spot_30mtwap_price](#)
 - [funding_rate](#)
 - [product_updates](#)
 - [announcements](#)
- [Private Channels](#)
 - [Margins](#)
 - [Positions](#)
 - [Orders](#)
 - [UserTrades](#)
 - [v2/user_trades](#)
 - [PortfolioMargins](#)
 - [MMP Trigger](#)
- [Changelog](#)
 - [01.06.25](#)
 - [21.03.25](#)
 - [20.02.25](#)
 - [18.10.2024](#)

- [01.05.2024](#)
- [V2 Rest Api](#)
- [New Socket Channels](#)
- [Security](#)
 - [Targets In scope](#)
 - [How to access](#)
 - [Reward range](#)
 - [Rules of engagement](#)
 - [Rules of reporting](#)
 - [Recommended Reporting format](#)
 - [Ineligible issues](#)

Introduction

Welcome to the Delta Exchange API! You can use this API to place orders on Delta Exchange and listen to market feed.

We have language bindings in Shell, Python, Ruby and Nodejs! You can view code examples in the dark area to the right, and you can switch the programming language of the examples with the tabs in the top right.

Scroll down for code samples, example requests and responses. Select a language for code samples from the tabs above or the mobile navigation menu.

General Information

Definitions

Assets: are cryptocurrencies or fiat currencies. Each asset has a 3/4 letter code (e.g. BTC, USDC). A derivative contract on Delta Exchange will entail multiple assets, namely:

- **Underlying Asset:** This is the asset over which a futures or perpetual contract is defined. For e.g. the underlying asset of the BTCUSD perpetual contract is BTC.
- **Quoting Asset:** The asset in which price of the quoting is called the quoting asset. For e.g. the BTCUSD contract is quoted in USD. Therefore, the quoting asset of the contract is USD.
- **Settling Asset:** The asset in which the margin and Profit/ Loss of a contract is denominated. For e.g. the BTCUSD contract is margined and settled in USD. Therefore, the settling asset of the contract is USD.

Products: are the derivative contracts listed on Delta Exchange. Each contract can be referred by either its Product ID (e.g. 27 is the Product ID of BTCUSD contract) or Symbol (BTCUSD is the symbol for BTCUSD contract). It is worth noting that Orders, Orderbook APIs expect Product IDs. e.g.

product_id	symbol	product_type	description
27	BTCUSD	perpetual_futures	Bitcoin Perpetual futures margined and settled in INR
3136	ETHUSD	perpetual_futures	Ethereum perpetual futures margined and settled in INR

product_id	symbol	product_type	description
2000	P-BTC-38100-230124	put_options	BTC put option strike price \$38100 and expiring on 23/01/2024
5000	C-BTC-55800-170224	call_options	BTC call option strike price \$55800 and expiring on 17/02/2024

Symbology

Contract symbol: has the following format

Underlying Asset| Quoting Asset|_|Matruity Date (optional, applicable only for futures contracts)

e.g. BTCUSD, ETHUSD

For Options symbol:

Option Type |-| Underlying Asset |-| Price Strikes |-| Expiry Date in ddMMYY

e.g. C-BTC-90000-310125: C (Call Option) BTC (Bitcoin) 90000 strike price expiring on 31st Jan 2025

Mark Price: Each contract has a unique Mark Price which can be referred to by: **MARK: Contract_Symbol (MARK:BTCUSD)**

Index Price: The prices of the underlying assets for the contracts listed on Delta Exchange are obtained from various spot exchanges, and are computed by averaging prices from multiple exchanges. Details of all the indices are available on this page.

For a given Underlying Asset/ Quoting Asset pair, the Index price symbol is: **.DE|Underlying Asset|Quoting Asset|** e.g. **.DEBNBXT** It is important to note that the BTCUSD Index price doesn't follow this convention as its symbol is **.DEXBTUSD**.

Pagination

Pagination allows to fetch data page-wise. We use cursor based pagination where each response metadata contains a before and after cursor. The cursor needs to be passed on with the next request to paginate over the result. Please note that the cursors are updated with each new page.

Pagination can be called in following APIs

API	End point
Products	/v2/products
Orders	/v2/orders
Orders History	/v2/orders/history
Fills	/v2/fills
Wallet transactions	/v2/wallet/transactions

Pagination parameters

- after
 - after cursor to fetch the next page
- before

- before cursor to fetch the previous page
- page_size
 - page size for pagination

In API response, meta data will contain the cursors as show below

```
{
  "success": true,
  "result": [],
  "meta": {
    "after": "an_arbitrary_string",
    "before": "another_arbitrary_string"
  }
}
```

Example

https://api.india.delta.exchange/v2/products?page_size=30

https://api.india.delta.exchange/v2/products?page_size=30&after=after_cursor_from_last_request

Data Centers

Delta Exchange data centers are in **AWS Tokyo**

Authentication

Api endpoints that place orders or fetch account related information needs to authenticated using api key and api secret.

Common Errors

SIGNATUREEXPIRED ERROR

- { "error": "SignatureExpired", "message": "your signature has expired" }

Explanation: This error occurs when the timestamp used to generate your API request signature is more than **5 seconds** old by the time it reaches Delta Exchange servers. The platform enforces this time window to prevent replay attacks. You can check server time and request time in the api error response too.

TROUBLESHOOTING

1. Sync Your System Clock

- Ensure your system time is accurate using NTP (Network Time Protocol).
 - **Linux/macOS:** `sudo ntpdate pool.ntp.org`
 - **Windows:** Make sure the *Windows Time* service is running and properly synced.
- Enable automatic time synchronization in your system settings to prevent future drifts.

2. Check Network Latency

- Check for any network latency on your side.
- Use a stable and high-speed internet connection.

3. Verify Timestamp Generation

- Use Unix timestamp format (seconds since epoch).
- Don't reuse stale timestamps; update the timestamp before every request.
- Ensure the timestamp used in the signature matches the one sent in the request.
- Refer to the sample code block on the right.

```
# python code to generate signature
timestamp = str(int(time.time()))
signature_data = method + timestamp + path + query_string + payload
signature = generate_signature(api_secret, signature_data)
```

INVALIDAPIKEY ERROR

- `{ "error": "InvalidApiKey", "message": "Api Key not found" }`

Explanation: The API key in your request doesn't exist or is invalid. This can happen due to using keys from the wrong environment or deleted keys.

TROUBLESHOOTING

1. Verify the Correct Environment

- API keys created at Delta India account must be used only with production apis.(prod api - <https://api.india.delta.exchange>)
- API keys created at Demo account must be used only with testnet apis.(testnet api - <https://cdn-ind.testnet.deltaex.org>)
- Please not that the api url <https://api.delta.exchange> belongs to Delta Global and it can not used here.

2. Check API Key Validity

- Log into Delta Exchange and confirm the key exists and is active.
- Check for any typos in the key too.

UNAUTHORIZEDAPIACCESS ERROR

- `{ "error": "UnauthorizedApiAccess", "message": "Api Key not authorised to access this endpoint" }`

Explanation: Your API key doesn't have permission to access this endpoint. Delta Exchange allows permission-based control for each key. There are two permissions given 1. Read Data  2. Trading 

TROUBLESHOOTING

1. Check API Key Permissions

- Go to API Management on Delta Exchange.
- Verify respective permissions: 1. Read Data 2. Trading, are enabled  or not.
- If required, create a new api key with the right permissions enabled .

2. Verify Endpoint Requirements

- Market data: requires Read Data permissions only.
- Orders, Positions, Wallets: requires Trading permission.

3. Consider Security Best Practices

- Grant only necessary permissions.
- Use separate keys for different use-cases.

IP NOT WHITELISTED ERROR

- `{ "success": false, "error": { "code": "ip_not_whitelisted_for_api_key" } }`

Explanation: This occurs when the request originates from an IP not allowed for the given API key. Delta Exchange enforces IP whitelisting as a security measure. Consider whitelisting the IP returned into the api error response.

TROUBLESHOOTING

1. Update IP Whitelist

- Log into Delta Exchange > API Management
- Add your IP to the API key's whitelist. The IP can be found into the api error response.
- You can whitelist a list of IPs too. Enter them as comma separated list.
- Delta support both IPv4 and IPv6 formats for whitelisting.

2. Consider Network Behavior

- VPNs or ISPs with dynamic IPs may require frequent updates.
- Consider cloud hosted machine with static IP for production to avoid whitelisting repeatedly.

REQUEST FORBIDDEN ERROR

- ```
{ "error": "Forbidden", "message": "Request blocked by CDN" }
```

**Explanation:** Your request was blocked before reaching the Delta API gateway. This usually happens when required headers are missing (e.g., User-Agent), or when requests originate from environments where the IP is hidden or blocked by the CDN.

## TROUBLESHOOTING

### 1. Add a User-Agent Manually

- Some HTTP clients (e.g., in certain programming languages) do not send a User-Agent header by default.
- Set a valid User-Agent string in your request to avoid being rejected.

### 2. Trace Request Headers

- Inspect the request/response headers using tools like curl -v, Postman, or your language's HTTP client debugger.
- Check if required headers are present and correctly passed through.

### 3. Verify Request IP

- Some hosted environments or proxies mask the client IP.
- Confirm that the request includes a valid IP address visible to the CDN. If the IP is hidden or blocked, the request will be rejected.

## SIGNATURE MISMATCH ERROR

- ```
{ "success": false, "error": { "code": "Signature Mismatch" } }
```

Explanation: This error occurs when the signature generated on your end does not match the one received by Delta Exchange. This can happen due to incorrect timestamp, payload, or method used in the signature generation.

TROUBLESHOOTING

1. Verify Signature Generation

- Ensure you are using the correct http method (GET/POST) and the same payload as in the request.
- Check that the timestamp used in the signature matches the one sent in the request.
- Ensure you are using the correct API secret for the signature generation.
- Make sure you are using the correct endpoint path and query parameters in the signature generation.
- Ensure that the payload is properly formatted and matches the request body.
- Check for any extra spaces or characters in the signature string.
- Refer to the sample code block (see Signing a Message).

By following these troubleshooting steps, you can resolve common Delta Exchange API authentication and authorization issues. For persistent problems, contact Delta Exchange support.

(Note - **api key creation blocked** - When user enters wrong otp/mfa code more than 5 times, Delta exchange blocks api key creation for next 30 mins. After then, users can try creating again.)

Generating an API Key

Before being able to sign any requests, you must create an API key via the Delta website. Upon creating a key you will receive api key and api secret, which you must remember and secure at your end. The Key and Secret will be randomly generated.

You can create a new API key from here : <https://www.delta.exchange/app/account/manageapikeys>

To create an API key with Trading permissions, whitelisted IP(s) must be provided. API requests using this API key will only succeed, if made from a machine with an IP address that was whitelisted. Multiple IPs can be whitelisted for a single API key. IP values can take IPv4 and IPv6. Whitelisted IPs can be changed and updated from the same page.

Common issues:

1. Your machine is using IPv6 instead of IPv4 or vice-versa. Disable the one that isn't whitelisted from your network settings or whitelist both.
2. Your ISP may automatically change the IP address at your home, office about once a week. Update your whitelisted IP in such case.
3. If you are using a Web Service/Cloud provider like AWS, assign a static public IPv4 address to your machine.

API Key Permissions

You can restrict the functionality of API keys. Before creating the key, you must choose what permissions you would like the key to have. The permissions are:

- Trading - Allow a key to have trading permissions. This includes placing new orders, cancelling orders, closing positions, changing margin & leverage.

Creating a Request

All Authenticated requests must contain the following headers:

api-key: The api key as a string.

signature: The hex-encoded signature (see Signing a Message).

timestamp: A timestamp for your request.

User-Agent: Your programming language or library that you are using. e.g. "python-3.10" or "java". This is for our internal stats, this header must be sent to avoid 4XX error.

All request bodies should have content type application/json and be valid JSON.

Signing a Message

The signature header is generated by creating a sha256 HMAC using the secret key on the prehash string **method + timestamp + requestPath + query params + body** (where + represents string concatenation) and convert output to hexdigest. The timestamp value is the same as the 'timestamp' header.

Signature created in the last 5 seconds is allowed. if signature reaches delta system post 5 seconds of generation, then it will fail.

Code samples

```
import hashlib
import hmac
import requests
import time

base_url = 'https://api.india.delta.exchange'
api_key = 'a207900b7693435a8fa9230a38195d'
api_secret = '7b6f39dcf660ec1c7c664f612c60410a2bd0c258416b498bf0311f94228f'

def generate_signature(secret, message):
    message = bytes(message, 'utf-8')
    secret = bytes(secret, 'utf-8')
    hash = hmac.new(secret, message, hashlib.sha256)
    return hash.hexdigest()

# Get open orders
method = 'GET'
timestamp = str(int(time.time()))
path = '/v2/orders'
url = f'{base_url}{path}'
query_string = '?product_id=1&state=open'
payload = ''
signature_data = method + timestamp + path + query_string + payload
signature = generate_signature(api_secret, signature_data)

req_headers = {
    'api-key': api_key,
    'timestamp': timestamp,
    'signature': signature,
    'User-Agent': 'python-rest-client',
    'Content-Type': 'application/json'
}

query = {"product_id": 1, "state": 'open'}

response = requests.request(
    method, url, data=payload, params=query, timeout=(3, 27), headers=req_headers
)

# Place new order
method = 'POST'
timestamp = str(int(time.time()))
path = '/v2/orders'
url = f'{base_url}{path}'
query_string = ''
payload = "{\"order_type\":\"limit_order\",\"size\":3,\"side\":\"buy\",\"limit_price\":\"0.0005\",\"product_id\":16}"
signature_data = method + timestamp + path + query_string + payload
signature = generate_signature(api_secret, signature_data)

req_headers = {
    'api-key': api_key,
    'timestamp': timestamp,
    'signature': signature,
    'User-Agent': 'rest-client',
    'Content-Type': 'application/json'
}

response = requests.request(
    method, url, data=payload, params={}, timeout=(3, 27), headers=req_headers
)
```

Rate Limits

When a rate limit is exceeded, a HTTP response status 429 Too Many Requests will be returned. 'X-RATE-LIMIT-RESET' is returned in response header with time left in milliseconds after which next API request can be hit.

We throttle unauthenticated api requests by IP address and authenticated requests by user ID.

Default Quota is 10000 for a fixed 5 minute window. Rate Limit quota resets to full every 5 mins.

API & product rate limits

REST API

Every REST endpoint has been assigned a weight. When you make an API call, the weight of that endpoint is deducted from your 5 min window quota. Exceeding the rate limit quota, results in 429 HTTP response status error. API endpoints related to reading public data are lighter in weight, whereas API endpoints related to writing private data (like placing an order) are heavier in weight.

Here is the cost structure for various endpoints. Please note that any endpoint not mentioned here has a cost of 1 unit.

Weight Slab	API Endpoints
3	Get Products, Get Orderbook, Get Tickers, Get Open Orders, Get Open Postions, Get Balances, OHLC Candles
5	Place/Edit/Delete Order, Add Position Margin
10	Get Order History, Get Fills, Get Txn Logs
25	Batch Order Apis

e.g. Consider making all the below requests in a 5 minute window.

100 Get Open Orders requests = $100 * 3 = 300$ weight.

50 Get Balances requests = $50 * 3 = 150$ weight.

200 Place order requests = $200 * 5 = 1000$ weight.

20 Batch Order requests = $20 * 25 = 500$ weight.

Total quota used: $300 + 150 + 1000 + 500 = 1950$ quota used. If this is greater than quota assigned to you, rate limit error will occur.

PRODUCT

We have also introduced rate limits to manage operations within the matching engine. The current rate limit is set at 500 operations per second for each product. For example, placing 50 orders in a batch counts as 50 operations, as each individual order will be processed by the matching engine.

This measure is in place to prevent excessive operations on the matching engine, which could slow down its performance.

Even if you have not exceeded the rate limit on the REST API level, your requests may still fail with a 429 error if you breach the product-level rate limit.

Increasing your rate limits

If you are running up against our limits and believe that you have a legitimate need, please email us at support@delta.exchange to discuss increasing your rate limits.

Types

Timestamps

Unless otherwise specified, all timestamps from API are returned in ISO 8601 with microseconds. Make sure you can parse the following ISO 8601 format. Most modern languages and libraries will handle this without issues.

```
2019-09-18T10:41:20Z
```

Numbers

Big Decimal numbers are returned as strings to save full precision. When making a API request, it is suggested to convert numbers to strings to avoid truncation and precision errors.

e.g. Limit price, stop loss price, trail_amount

Integer numbers (like contract size, product_id and impact size) are unquoted.

IDs

Most identifiers are big integers unless otherwise specified. When making a request that requires an ID, ensure it is passed as a numeric value without formatting changes. When making a request which requires a UUID, both forms (with and without dashes) are accepted.

```
167ja7cg-678e-6876-d6g3-f803ce49qsc9 OR 167ja7cg678e6876d6g3f803ce49qsc9
```

Response Formats

To ensure that you are effectively using the api, we encourage you to go through this section.

1. All responses coming from the server, either from rest api or socket rpc calls will have the following success and error formats.
2. All timestamps reported in the apis will be in microseconds
3. All big decimal values are sent as string

```
// The new format supports sending meta data alongside response body.  
// Success format  
{  
  "success": true,
```

```
"result": {},          // response body
"meta": {
  "after": "...",      // cursor for pagination, is returned in meta
  "before": null,
},
}

// Error Format
{
  "success": false,
  "error": {
    "code": "insufficient_margin",      // error code
    "context": {                       // error context
      "additional_margin_required": "0.121"
    }
  }
}
```

title: Delta Exchange Api V2 v1.0.0 language_tabs: - python: Python - shell: Shell - ruby: Ruby language_clients: - python: "" - shell: "" - ruby: "" toc_footers: [] includes: [] search: true highlight_theme: darkula headingLevel: 2

Rest Api

This section documents the latest(v2) api for trading on Delta Exchange. The REST API has endpoints for account and order management as well as public market data.

If you are looking for the old api documentation, here is the link to v1 api docs (now deprecated).

REST API Endpoint URL for Delta Exchange

- **Production** - <https://api.india.delta.exchange>
- **Testnet(Demo Account)** - <https://cdn-ind.testnet.deltaex.org>

Assets

Get Asset List

Get list of all assets

Code samples

```
import requests

headers = {
  'Accept': 'application/json'
}
```

```
r = requests.get('https://api.india.delta.exchange/v2/assets', params={
}, headers = headers)

print r.json()
```

GET /assets

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 14,
      "symbol": "USD",
      "precision": 8,
      "deposit_status": "enabled",
      "withdrawal_status": "enabled",
      "base_withdrawal_fee": "0.0000000000000000",
      "min_withdrawal_amount": "0.0000000000000000"
    }
  ]
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of Asset schema	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
deposit_status	enabled
deposit_status	disabled
withdrawal_status	enabled
withdrawal_status	disabled

✔ This operation does not require authentication.

Indices

Get indices

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/indices', params={

}, headers = headers)

print r.json()
```

GET /indices

Indices refer to spot price indices that Delta Exchange creates by combining spot prices of prominent crypto exchanges. These indices form the underlying of futures and options contracts listed on Delta Exchange. All details of indices on Delta Exchange are available [here](#).

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 14,
      "symbol": ".DEXBTUSD",
      "constituent_exchanges": [
        {
          "name": "ExchangeA",
          "weight": 0.25
        }
      ],
      "underlying_asset_id": 13,
      "quoting_asset_id": 14,
      "tick_size": "0.5",
      "index_type": "spot_pair"
    }
  ]
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of Spot Index schema	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
index_type	spot_pair
index_type	fixed_interest_rate
index_type	floating_interest_rate

✔ This operation does not require authentication.

Products

Get Product List

Get list of products

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/products', params={

}, headers = headers)

print r.json()
```

GET /products

The endpoint provides details about all available trading products on the platform. Each product represents a financial instrument like perpetual futures, options, or contracts for specific asset pairs.

PARAMETERS

Parameter	In	Type	Required	Description
contract_types	query	string	false	Comma separated list of contract types e.g. perpetual_futures,call_options, put_options
states	query	string	false	Comma separated list of states e.g. upcoming, live, expired, settled to get expired contracts.
after	query	string	false	after cursor for paginated request
before	query	string	false	before cursor for paginated request

Parameter	In	Type	Required	Description
page_size	query	string	false	size of a single page for paginated request, default: 100

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 27,
      "symbol": "BTCUSD",
      "description": "Bitcoin Perpetual futures, quoted, settled & margined in USD",
      "created_at": "2023-12-18T13:10:39Z",
      "updated_at": "2024-11-15T02:47:50Z",
      "settlement_time": null,
      "notional_type": "vanilla",
      "impact_size": 10000,
      "initial_margin": "0.5",
      "maintenance_margin": "0.25",
      "contract_value": "0.001",
      "contract_unit_currency": "BTC",
      "tick_size": "0.5",
      "product_specs": {
        "funding_clamp_value": 0.05,
        "only_reduce_only_orders_allowed": false,
        "tags": [
          "layer_1"
        ]
      },
      "state": "live",
      "trading_status": "operational",
      "max_leverage_notional": "100000",
      "default_leverage": "200",
      "initial_margin_scaling_factor": "0.0000025",
      "maintenance_margin_scaling_factor": "0.00000125",
      "taker_commission_rate": "0.0005",
      "maker_commission_rate": "0.0002",
      "liquidation_penalty_factor": "0.5",
      "contract_type": "perpetual_futures",
      "position_size_limit": 229167,
      "basis_factor_max_limit": "10.95",
      "is_quanto": false,
      "funding_method": "mark_price",
      "annualized_funding": "10.95",
      "price_band": "2.5",
      "underlying_asset": {
        "id": 14,
        "symbol": "USD",
        "precision": 8,
        "deposit_status": "enabled",
        "withdrawal_status": "enabled",
        "base_withdrawal_fee": "0.000000000000000000",
        "min_withdrawal_amount": "0.000000000000000000"
      },
      "quoting_asset": {
        "id": 14,
        "symbol": "USD",
        "precision": 8,
        "deposit_status": "enabled",
        "withdrawal_status": "enabled",
        "base_withdrawal_fee": "0.000000000000000000",
        "min_withdrawal_amount": "0.000000000000000000"
      }
    }
  ]
}
```

```

    },
    "settling_asset": {
      "id": 14,
      "symbol": "USD",
      "precision": 8,
      "deposit_status": "enabled",
      "withdrawal_status": "enabled",
      "base_withdrawal_fee": "0.0000000000000000",
      "min_withdrawal_amount": "0.0000000000000000"
    },
    "spot_index": {
      "id": 14,
      "symbol": ".DEXBTUSD",
      "constituent_exchanges": [
        {
          "name": "ExchangeA",
          "weight": 0.25
        }
      ],
      "underlying_asset_id": 13,
      "quoting_asset_id": 14,
      "tick_size": "0.5",
      "index_type": "spot_pair"
    }
  }
}

```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of Products	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
notional_type	vanilla
notional_type	inverse
state	live
state	expired
state	upcoming
trading_status	operational
trading_status	disrupted_cancel_only
trading_status	disrupted_post_only
deposit_status	enabled
deposit_status	disabled

Property	Value
withdrawal_status	enabled
withdrawal_status	disabled
index_type	spot_pair
index_type	fixed_interest_rate
index_type	floating_interest_rate

✔ This operation does not require authentication.

Get product by symbol

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/products/{symbol}', params={

}, headers = headers)

print r.json()
```

GET /products/{symbol}

The endpoint retrieves details of a specific product identified by its symbol (e.g., BTCUSD, ETHUSD).

PARAMETERS

Parameter	In	Type	Required	Description
symbol	path	string	true	symbol of the desired product like BTCUSD, ETHUSD

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": 27,
    "symbol": "BTCUSD",
    "description": "Bitcoin Perpetual futures, quoted, settled & margined in USD",
    "created_at": "2023-12-18T13:10:39Z",
    "updated_at": "2024-11-15T02:47:50Z",
    "settlement_time": null,
```

```
"notional_type": "vanilla",
"impact_size": 10000,
"initial_margin": "0.5",
"maintenance_margin": "0.25",
"contract_value": "0.001",
"contract_unit_currency": "BTC",
"tick_size": "0.5",
"product_specs": {
  "funding_clamp_value": 0.05,
  "only_reduce_only_orders_allowed": false,
  "tags": [
    "layer_1"
  ]
},
"state": "live",
"trading_status": "operational",
"max_leverage_notional": "100000",
"default_leverage": "200",
"initial_margin_scaling_factor": "0.0000025",
"maintenance_margin_scaling_factor": "0.00000125",
"taker_commission_rate": "0.0005",
"maker_commission_rate": "0.0002",
"liquidation_penalty_factor": "0.5",
"contract_type": "perpetual_futures",
"position_size_limit": 229167,
"basis_factor_max_limit": "10.95",
"is_quanto": false,
"funding_method": "mark_price",
"annualized_funding": "10.95",
"price_band": "2.5",
"underlying_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.000000000000000000",
  "min_withdrawal_amount": "0.000000000000000000"
},
"quoting_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.000000000000000000",
  "min_withdrawal_amount": "0.000000000000000000"
},
"settling_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.000000000000000000",
  "min_withdrawal_amount": "0.000000000000000000"
},
"spot_index": {
  "id": 14,
  "symbol": ".DEXBTUSD",
  "constituent_exchanges": [
    {
      "name": "ExchangeA",
      "weight": 0.25
    }
  ],
  "underlying_asset_id": 13,
  "quoting_asset_id": 14,
```

```
    "tick_size": "0.5",
    "index_type": "spot_pair"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Product	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
notional_type	vanilla
notional_type	inverse
state	live
state	expired
state	upcoming
trading_status	operational
trading_status	disrupted_cancel_only
trading_status	disrupted_post_only
deposit_status	enabled
deposit_status	disabled
withdrawal_status	enabled
withdrawal_status	disabled
index_type	spot_pair
index_type	fixed_interest_rate
index_type	floating_interest_rate

✔ This operation does not require authentication.

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/tickers', params={

}, headers = headers)

print r.json()
```

GET /tickers

This endpoint retrieves the live tickers for available trading products, with an optional filter by specified contract types. The contract types should be provided as a comma-separated list (e.g., futures, perpetual_futures, call_options). If no contract type is specified, data for all available products will be returned.

PARAMETERS

Parameter	In	Type	Required	Description
contract_types	query	string	false	A comma-separated list of contract types to filter the tickers. Example values include perpetual_futures, call_options, put_options.
underlying_asset_symbols	query	string	false	A comma-separated list of underlying asset symbols to filter the tickers. Example values include BTC, ETH, SOL etc.
expiry_date	query	string	false	Expiry date(format: DD-MM-YYYY) to filter the tickers.

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "close": 67321,
      "contract_type": "futures",
      "greeks": {
        "delta": "0.25",
        "gamma": "0.10",
        "rho": "0.05",
        "theta": "-0.02",
        "vega": "0.15"
      },
      "high": 68500.5,
      "low": 66300.25,
      "mark_price": "67000.00",
      "mark_vol": "500",
      "oi": "15000",
      "oi_value": "1000000",
      "oi_value_symbol": "USD",
      "oi_value_usd": "1050000",
      "open": 67000,
      "price_band": {
        "lower_limit": "61120.45",
        "upper_limit": "72300.00"
      }
    },
  ],
}
```

```
{
  "product_id": 123456,
  "quotes": {
    "ask_iv": "0.25",
    "ask_size": "100",
    "best_ask": "150.00",
    "best_bid": "148.00",
    "bid_iv": "0.22",
    "bid_size": "50"
  },
  "size": 100,
  "spot_price": "67000.00",
  "strike_price": "68000.00",
  "symbol": "BTCUSD",
  "timestamp": 1609459200,
  "turnover": 5000000,
  "turnover_symbol": "USD",
  "turnover_usd": 5200000,
  "volume": 25000
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of live tickers for all products, including implied volatility (IV) for option strikes.	Inline

RESPONSE SCHEMA

✔ This operation does not require authentication.

Get ticker for a product by symbol

Code samples

```
import requests
headers = {
  'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/tickers/{symbol}', params={

}, headers = headers)

print r.json()
```

GET /tickers/{symbol}

This endpoint retrieves the ticker data for a specific product, identified by its symbol. The ticker data includes live price data, open interest, implied volatility (IV) for options, and other related market data.

PARAMETERS

Parameter	In	Type	Required	Description
symbol	path	string	true	The symbol of the product for which the ticker data is requested (e.g., BTCUSD, ETHUSD).

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "close": 67321,
    "contract_type": "futures",
    "greeks": {
      "delta": "0.25",
      "gamma": "0.10",
      "rho": "0.05",
      "theta": "-0.02",
      "vega": "0.15"
    },
    "high": 68500.5,
    "low": 66300.25,
    "mark_price": "67000.00",
    "mark_vol": "500",
    "oi": "15000",
    "oi_value": "1000000",
    "oi_value_symbol": "USD",
    "oi_value_usd": "1050000",
    "open": 67000,
    "price_band": {
      "lower_limit": "61120.45",
      "upper_limit": "72300.00"
    },
    "product_id": 123456,
    "quotes": {
      "ask_iv": "0.25",
      "ask_size": "100",
      "best_ask": "150.00",
      "best_bid": "148.00",
      "bid_iv": "0.22",
      "bid_size": "50"
    },
    "size": 100,
    "spot_price": "67000.00",
    "strike_price": "68000.00",
    "symbol": "BTCUSD",
    "timestamp": 1609459200,
    "turnover": 5000000,
    "turnover_symbol": "USD",
    "turnover_usd": 5200000,
    "volume": 25000
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Ticker data for the requested product, including implied volatility (IV) for option strikes, if applicable.	Inline

RESPONSE SCHEMA

✔ This operation does not require authentication.

Get option chain

Code samples

```
import requests

headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/tickers?contract_types=call_options,put_options&underlying_asset_symbols={underlying_asset_symbols}&expiry_date={expiry_date}', headers = headers)

print r.json()
```

GET /tickers?contract_types=call_options,put_options&underlying_asset_symbols={underlying_asset_symbols}&expiry_date={expiry_date}

Fetch option chain data for a given product and expiry date.

For example, to get BTC call and put options expiring on 04-04-2025, use:

contract_types=call_options,put_options&underlying_asset_symbols=BTC&expiry_date=04-04-2025

PARAMETERS

Parameter	In	Type	Required	Description
contract_types	query	string	false	A comma-separated list of contract types to filter the products. Only <code>call_options</code> and <code>put_options</code> are supported.
underlying_asset_symbols	query	string	false	The underlying asset symbol (single value) for the option chain. Examples: <code>BT</code> , <code>C</code> , <code>ETH</code> , <code>SOL</code> .
expiry_date	query	string	false	Expiry date of the contracts in <code>DD-MM-YYYY</code> format.

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "close": 67321,
      "contract_type": "futures",
      "greeks": {
        "delta": "0.25",
        "gamma": "0.10",
        "rho": "0.05",
        "theta": "-0.02",
        "vega": "0.15"
      }
    }
  ]
}
```

```

    },
    "high": 68500.5,
    "low": 66300.25,
    "mark_price": "67000.00",
    "mark_vol": "500",
    "oi": "15000",
    "oi_value": "1000000",
    "oi_value_symbol": "USD",
    "oi_value_usd": "1050000",
    "open": 67000,
    "price_band": {
      "lower_limit": "61120.45",
      "upper_limit": "72300.00"
    },
    },
    "product_id": 123456,
    "quotes": {
      "ask_iv": "0.25",
      "ask_size": "100",
      "best_ask": "150.00",
      "best_bid": "148.00",
      "bid_iv": "0.22",
      "bid_size": "50"
    },
    },
    "size": 100,
    "spot_price": "67000.00",
    "strike_price": "68000.00",
    "symbol": "BTCUSD",
    "timestamp": 1609459200,
    "turnover": 5000000,
    "turnover_symbol": "USD",
    "turnover_usd": 5200000,
    "volume": 25000
  }
}
}

```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns a list of live tickers for all products, including implied volatility (IV) for option strikes.	Inline

RESPONSE SCHEMA

✔ This operation does not require authentication.

Orders

Placing Orders, Cancelling Orders, Placing batch orders, Cancelling batch orders, Get Open orders, Change Orders Leverage. Rate limits have been introduced recently that allows only set number of operations inside a matching engine in a timeframe. The current rate limits is 500 operations/sec for each product. For ex - placing 50 orders in a batch is equivalent to 50 operations as these orders will be processed by matching engine. Rate limits do not apply when cancelling orders.

Place Order

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/orders', params={

}, headers = headers)

print r.json()
```

POST /orders

Body parameter

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "limit_price": "59000",
  "size": 10,
  "side": "buy",
  "order_type": "limit_order",
  "stop_order_type": "stop_loss_order",
  "stop_price": "56000",
  "trail_amount": "50",
  "stop_trigger_method": "last_traded_price",
  "bracket_stop_trigger_method": "last_traded_price",
  "bracket_stop_loss_limit_price": "57000",
  "bracket_stop_loss_price": "56000",
  "bracket_trail_amount": "50",
  "bracket_take_profit_limit_price": "62000",
  "bracket_take_profit_price": "61000",
  "time_in_force": "gtc",
  "mmp": "disabled",
  "post_only": false,
  "reduce_only": false,
  "client_order_id": "34521712",
  "cancel_orders_accepted": false
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	CreateOrderRequest	true	Order which needs to be created. Rate limits apply.

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": 123,
    "user_id": 453671,
    "size": 10,
    "unfilled_size": 2,
    "side": "buy",
    "order_type": "limit_order",
    "limit_price": "59000",
    "stop_order_type": "stop_loss_order",
    "stop_price": "55000",
    "paid_commission": "0.5432",
    "commission": "0.5432",
    "reduce_only": false,
    "client_order_id": "34521712",
    "state": "open",
    "created_at": "1725865012000000",
    "product_id": 27,
    "product_symbol": "BTCUSD"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns back the order object with assigned id and latest state	Inline
400	Bad Request	Returns error if order could not be placed	ApiErrorResponse

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Cancel Order

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.delete('https://api.india.delta.exchange/v2/orders', params={

}, headers = headers)

print r.json()
```

DELETE /orders

Body parameter

```
{
  "id": 13452112,
  "client_order_id": "34521712",
  "product_id": 27
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	DeleteOrderRequest	true	Order which needs to be cancelled

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": 123,
    "user_id": 453671,
    "size": 10,
    "unfilled_size": 2,
    "side": "buy",
    "order_type": "limit_order",
    "limit_price": "59000",
    "stop_order_type": "stop_loss_order",
    "stop_price": "55000",
    "paid_commission": "0.5432",
```

```
    "commission": "0.5432",
    "reduce_only": false,
    "client_order_id": "34521712",
    "state": "open",
    "created_at": "1725865012000000",
    "product_id": 27,
    "product_symbol": "BTCUSD"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns back the order object	Inline
400	Bad Request	Returns error if order could not be cancelled	ApiErrorResponse

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Edit Order

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.put('https://api.india.delta.exchange/v2/orders', params={

}, headers = headers)

print r.json()
```

PUT /orders

Body parameter

```
{
  "id": 34521712,
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "limit_price": "59000",
  "size": 15,
  "mmp": "disabled",
  "post_only": false,
  "cancel_orders_accepted": false,
  "stop_price": "56000",
  "trail_amount": "50"
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	EditOrderRequest	true	Order which needs to be edited. Rate limits apply.

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": 123,
    "user_id": 453671,
    "size": 10,
    "unfilled_size": 2,
    "side": "buy",
    "order_type": "limit_order",
    "limit_price": "59000",
    "stop_order_type": "stop_loss_order",
    "stop_price": "55000",
    "paid_commission": "0.5432",
    "commission": "0.5432",
    "reduce_only": false,
    "client_order_id": "34521712",
    "state": "open",
    "created_at": "1725865012000000",
    "product_id": 27,
  }
}
```



```
    "product_symbol": "BTCUSD"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns back the order object with assigned id and latest state	Inline
400	Bad Request	Returns error if order could not be placed	ApiErrorResponse

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get Active Orders

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}
```

```
r = requests.get('https://api.india.delta.exchange/v2/orders', params={

}, headers = headers)

print r.json()
```

GET /orders

PARAMETERS

Parameter	In	Type	Required	Description
product_ids	query	string	false	comma separated product ids, if not specified, all the orders will be returned
states	query	string	false	comma separated list of states - open,pending
contract_types	query	string	false	comma separated list of desired contract types, if not specified any parameters then, all the orders will be returned
order_types	query	string	false	comma separated order types
start_time	query	integer	false	from time in micro-seconds in epoc; referring to the order creation time
end_time	query	integer	false	from time in micro-seconds in epoc; referring to the order creation time
after	query	string	false	after cursor for pagination; becomes null if page after the current one does not exist
before	query	string	false	before cursor for pagination; becomes null if page before the current one does not exist
page_size	query	integer	false	number of records per page

ENUMERATED VALUES

Parameter	Value
contract_types	futures
contract_types	perpetual_futures
contract_types	call_options
contract_types	put_options
order_types	market
order_types	limit
order_types	stop_market
order_types	stop_limit
order_types	all_stop

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 123,
      "user_id": 453671,
      "size": 10,
      "unfilled_size": 2,
      "side": "buy",
      "order_type": "limit_order",
      "limit_price": "59000",
      "stop_order_type": "stop_loss_order",
      "stop_price": "55000",
      "paid_commission": "0.5432",
      "commission": "0.5432",
      "reduce_only": false,
      "client_order_id": "34521712",
      "state": "open",
      "created_at": "1725865012000000",
      "product_id": 27,
      "product_symbol": "BTCUSD"
    }
  ],
  "meta": {
    "after": "g3QAAACZAAKY3JlYXRlZF9hdHQAAAAAN",
    "before": "a2PQRSACZAAKY3JlYXRlZF3fnqHBBBNZL"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of orders as per the query	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending

Property	Value
state	closed
state	cancelled

❗ To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Place Bracket order

Code samples

```
import requests
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/orders/bracket', params={

}, headers = headers)

print r.json()
```

POST /orders/bracket

A bracket order is a set of TP and SL order. For a bracket order , size need not be specified as it closes the entire position. For a given contract, you can have multiple bracket orders for open orders but only a single bracket order for any open position.

Body parameter

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "stop_loss_order": {
    "order_type": "limit_order",
    "stop_price": "56000",
    "trail_amount": "50",
    "limit_price": "55000"
  },
  "take_profit_order": {
    "order_type": "limit_order",
    "stop_price": "65000",
    "limit_price": "64000"
  },
  "bracket_stop_trigger_method": "last_traded_price"
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	CreateBracketOrderRequest	true	Bracket Order which needs to be updated

Example responses

200 Response

```
{
  "success": true
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns back success response	ApiResponseSuccess
400	Bad Request	Returns error if orders could not be updated	ApiResponseError

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Edit Bracket order

Code samples

```
import requests
headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json',
  'api-key': '****',
  'signature': '****',
  'timestamp': '****'
}

r = requests.put('https://api.india.delta.exchange/v2/orders/bracket', params={

}, headers = headers)

print r.json()
```

PUT /orders/bracket

A bracket order is a set of TP and SL order. You can specify bracket order with an order that will create a new position. Use this api to change the bracket params attached with an order.

Body parameter

```
{
  "id": 34521712,
  "product_id": 27,
```

```
"product_symbol": "BTCUSD",
"bracket_stop_loss_limit_price": "55000",
"bracket_stop_loss_price": "56000",
"bracket_take_profit_limit_price": "65000",
"bracket_take_profit_price": "64000",
"bracket_trail_amount": "50",
"bracket_stop_trigger_method": "last_traded_price"
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	EditBracketOrderRequest	true	Bracket Order which needs to be updated

Example responses

200 Response

```
{
  "success": true
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns back success response	ApiResponseSuccess
400	Bad Request	Returns error if orders could not be updated	ApiResponseError

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Cancel all open orders

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.delete('https://api.india.delta.exchange/v2/orders/all', params={

}, headers = headers)

print r.json()
```

DELETE /orders/all

Cancels all orders for a given product id. If product id is not provided, it cancels orders for provided contract types. If none of them are provided, it cancels all the orders. Provide either product id or list of contract types at a time. If both are provided, contract types will be ignored.

Body parameter

```
{
  "product_id": 27,
  "contract_types": "perpetual_futures,put_options,call_options",
  "cancel_limit_orders": false,
  "cancel_stop_orders": false,
  "cancel_reduce_only_orders": false
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	CancelAllFilterObject	false	Filters for selecting orders that needs to be cancelled

Example responses

200 Response

```
{
  "success": true
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns back success response	ApiResponseSuccess
400	Bad Request	Returns error if orders could not be cancelled	ApiResponseError

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Create batch orders

Code samples

```
import requests
headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json',
  'api-key': '****',
  'signature': '****',
  'timestamp': '****'
}
```

```
r = requests.post('https://api.india.delta.exchange/v2/orders/batch', params={
}, headers = headers)

print r.json()
```

POST /orders/batch

Orders in a batch should belong to the same contract. Max allowed size limit in a batch is 50. Rate limits apply. Please note that ioc is not valid time in force values for creating batch orders.

Body parameter

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "orders": [
    {
      "limit_price": "59000",
      "size": 10,
      "side": "buy",
      "order_type": "limit_order",
      "time_in_force": "gtc",
      "mmp": "disabled",
      "post_only": false,
      "client_order_id": "34521712"
    }
  ]
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	BatchCreateOrdersRequest	true	Does not support time_in_force flag for orders, All orders in batch create are assumed to be gtc orders. batch create does not support stop orders, it support only limit orders

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 123,
      "user_id": 453671,
      "size": 10,
      "unfilled_size": 2,
      "side": "buy",
      "order_type": "limit_order",
      "limit_price": "59000",
      "stop_order_type": "stop_loss_order",
      "stop_price": "55000",
      "paid_commission": "0.5432",
      "commission": "0.5432",
      "reduce_only": false,
      "client_order_id": "34521712",
      "state": "open",
    }
  ]
}
```



```
    "created_at": "1725865012000000",
    "product_id": 27,
    "product_symbol": "BTCUSD"
  }
]
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns the orders placed	Inline
400	Bad Request	returns error if orders couldnt be placed	ApiErrorResponse

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Edit batch orders

Code samples

```
import requests
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
```

```
'api-key': '****',
'signature': '****',
'timestamp': '****'
}

r = requests.put('https://api.india.delta.exchange/v2/orders/batch', params={

}, headers = headers)

print r.json()
```

PUT /orders/batch

Orders to be edited in a batch. Rate limits apply.

Body parameter

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "orders": [
    {
      "id": 34521712,
      "limit_price": "59000",
      "size": 15,
      "mmp": "disabled",
      "post_only": false
    }
  ]
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	BatchEditOrdersRequest	true	none

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 123,
      "user_id": 453671,
      "size": 10,
      "unfilled_size": 2,
      "side": "buy",
      "order_type": "limit_order",
      "limit_price": "59000",
      "stop_order_type": "stop_loss_order",
      "stop_price": "55000",
      "paid_commission": "0.5432",
      "commission": "0.5432",
      "reduce_only": false,
      "client_order_id": "34521712",
      "state": "open",
      "created_at": "1725865012000000",
      "product_id": 27,
    }
  ]
}
```

```
    "product_symbol": "BTCUSD"
  }
}
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of edited orders	Inline
400	Bad Request	returns error if orders couldnt be edited	ApiErrorResponse

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Delete batch orders

Code samples

```
import requests
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
```

```
'timestamp': ''''
}

r = requests.delete('https://api.india.delta.exchange/v2/orders/batch', params={

}, headers = headers)

print r.json()
```

DELETE /orders/batch

Body parameter

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "orders": [
    {
      "id": 13452112,
      "client_order_id": "34521712"
    }
  ]
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	BatchDeleteOrdersRequest	true	none

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 123,
      "user_id": 453671,
      "size": 10,
      "unfilled_size": 2,
      "side": "buy",
      "order_type": "limit_order",
      "limit_price": "59000",
      "stop_order_type": "stop_loss_order",
      "stop_price": "55000",
      "paid_commission": "0.5432",
      "commission": "0.5432",
      "reduce_only": false,
      "client_order_id": "34521712",
      "state": "open",
      "created_at": "1725865012000000",
      "product_id": 27,
      "product_symbol": "BTCUSD"
    }
  ]
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns the orders deleted	Inline
400	Bad Request	returns error if orders couldnt be deleted	ApiErrorResponse

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get Order by id

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/orders/{order_id}', params={

}, headers = headers)
```

```
print r.json()
```

GET /orders/{order_id}

PARAMETERS

Parameter	In	Type	Required	Description
order_id	path	string	true	Id of the order

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": 123,
    "user_id": 453671,
    "size": 10,
    "unfilled_size": 2,
    "side": "buy",
    "order_type": "limit_order",
    "limit_price": "59000",
    "stop_order_type": "stop_loss_order",
    "stop_price": "55000",
    "paid_commission": "0.5432",
    "commission": "0.5432",
    "reduce_only": false,
    "client_order_id": "34521712",
    "state": "open",
    "created_at": "1725865012000000",
    "product_id": 27,
    "product_symbol": "BTCUSD"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns back the order object with assigned id and latest state	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order

Property	Value
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get Order by client oid

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/orders/client_order_id/{client_oid}', params={

}, headers = headers)

print r.json()
```

GET /orders/client_order_id/{client_oid}

PARAMETERS

Parameter	In	Type	Required	Description
client_oid	path	string	true	Client provided order id

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": 123,
    "user_id": 453671,
    "size": 10,
```

```
    "unfilled_size": 2,
    "side": "buy",
    "order_type": "limit_order",
    "limit_price": "59000",
    "stop_order_type": "stop_loss_order",
    "stop_price": "55000",
    "paid_commission": "0.5432",
    "commission": "0.5432",
    "reduce_only": false,
    "client_order_id": "34521712",
    "state": "open",
    "created_at": "1725865012000000",
    "product_id": 27,
    "product_symbol": "BTCUSD"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns back the order object with assigned client order id and latest state	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Change order leverage

Code samples

```
import requests
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/products/{product_id}/orders/leverage', params={
}, headers = headers)

print r.json()
```

POST /products/{product_id}/orders/leverage

Body parameter

```
{
  "leverage": 10
}
```

PARAMETERS

Parameter	In	Type	Required	Description
product_id	path	integer	true	Product id of the ordered product. Either product_id or product_symbol must be preseat.
product_symbol	path	string	true	Product symbol of the ordered product. Either product_id or product_symbol must be preseat.
body	body	object	true	none
» leverage	body	string	true	Order leverage

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "leverage": 10,
    "order_margin": "563.2",
    "product_id": 27
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns the OrderLeverage object	Inline

Status	Meaning	Description	Schema
400	Bad Request	Returns error if leverage couldnt be changed	ApiResponseResponse

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get order leverage

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/products/{product_id}/orders/leverage', params={

}, headers = headers)

print r.json()
```

GET /products/{product_id}/orders/leverage

PARAMETERS

Parameter	In	Type	Required	Description
product_id	path	integer	true	Product id of the ordered product

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "leverage": 10,
    "order_margin": "563.2",
    "product_id": 27
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns the OrderLeverage object	Inline

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Positions

Get Open positions, Change Position Margin, Close Position, Close All Position

Get margined positions

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/positions/margined', params={

}, headers = headers)

print r.json()
```

GET /positions/margined

Change in position may take upto 10secs to reflect. Use 'GET /position' for real-time data.

PARAMETERS

Parameter	In	Type	Required	Description
product_ids	query	string	false	comma separated product ids. If not specified any parameters, all the open positions will be returned
contract_types	query	string	false	comma separated list of desired contract types. If not specified any parameters then, all the open positions will be returned

ENUMERATED VALUES

Parameter	Value
contract_types	perpetual_futures
contract_types	call_options

Parameter	Value
contract_types	put_options

Example responses

200 Response

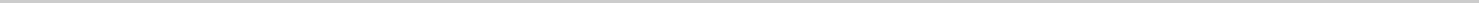
```
{
  "success": true,
  "result": [
    {
      "user_id": 0,
      "size": 0,
      "entry_price": "string",
      "margin": "string",
      "liquidation_price": "string",
      "bankruptcy_price": "string",
      "adl_level": 0,
      "product_id": 0,
      "product_symbol": "string",
      "commission": "string",
      "realized_pnl": "string",
      "realized_funding": "string"
    }
  ]
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of all open positions	Inline

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.



Get position

Code samples

```
import requests

headers = {
  'Accept': 'application/json',
  'api-key': '****',
  'signature': '****',
  'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/positions', params={
  'product_id': '0'
}, headers = headers)
```

```
print r.json()
```

GET /positions

Get real-time positions data.

PARAMETERS

Parameter	In	Type	Required	Description
product_id	query	integer	true	id of the product. Only send either product_id or underlying_asset_symbol.
underlying_asset_symbol	query	string	false	Underlying asset symbol. e.g. 'BTC', 'ETH'. This gives a list of all positions in products which have the given underlying asset. Only send either product_id or underlying_asset_symbol.

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "size": 0,
    "entry_price": "string"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Open position for the give product id	Inline

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Auto Topup

Code samples

```
import requests

headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json',
  'api-key': '****',
  'signature': '****',
  'timestamp': '****'
}

r = requests.put('https://api.india.delta.exchange/v2/positions/auto_topup', params={
```

```
}, headers = headers)

print r.json()
```

PUT /positions/auto_topup

Changes position auto topup flag. Positions automatically inherits auto topup flag of the account. If account level auto topop is set to false, use this api to change auto topup flag for individual positions.

Body parameter

```
{
  "product_id": 0,
  "auto_topup": "false"
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	object	true	none
» product_id	body	integer	true	none
» auto_topup	body	boolean	true	none

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "user_id": 0,
    "size": 0,
    "entry_price": "string",
    "margin": "string",
    "liquidation_price": "string",
    "bankruptcy_price": "string",
    "adl_level": 0,
    "product_id": 0,
    "product_symbol": "string",
    "commission": "string",
    "realized_pnl": "string",
    "realized_funding": "string"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns the position object	Inline
400	Bad Request	Returns error if position margin could not be changed	ApiErrorResponse

RESPONSE SCHEMA

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Add/Remove position margin

Code samples

```
import requests
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/positions/change_margin', params={

}, headers = headers)

print r.json()
```

POST /positions/change_margin

Body parameter

```
{
  "product_id": 0,
  "delta_margin": "string"
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	object	true	none
» product_id	body	integer	true	none
» delta_margin	body	string	true	Delta in the position margin, positive in case of adding margin & negative in case of removing margin

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "user_id": 0,
    "size": 0,
    "entry_price": "string",
```

```
"margin": "string",
"liquidation_price": "string",
"bankruptcy_price": "string",
"adl_level": 0,
"product_id": 0,
"product_symbol": "string",
"commission": "string",
"realized_pnl": "string",
"realized_funding": "string"
}
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns the position object	Inline
400	Bad Request	Returns error if position margin could not be changed	ApiErrorResponse

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Close all positions

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/positions/close_all', params={

}, headers = headers)

print r.json()
```

POST /positions/close_all

Body parameter

```
{
  "close_all_portfolio": true,
  "close_all_isolated": true,
  "user_id": 0
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	object	true	none
» close_all_portfolio	body	boolean	true	none
» close_all_isolated	body	boolean	true	none
» user_id	body	integer	true	none

Example responses

200 Response

```
{
  "success": true
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	returns back success response	ApiResponseSuccess
400	Bad Request	Returns error if not able to close all positions	ApiResponseError

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

TradeHistory

Get Orders History, Get Fill History

Get order history (cancelled and closed)

Code samples

```
import requests
headers = {
  'Accept': 'application/json',
  'api-key': '****',
  'signature': '****',
  'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/orders/history', params={

}, headers = headers)
```

```
print r.json()
```

GET /orders/history

PARAMETERS

Parameter	In	Type	Required	Description
product_ids	query	string	false	comma separated product ids
contract_types	query	string	false	comma separated list of desired contract types
order_types	query	string	false	comma separated order types
start_time	query	integer	false	from time in micro-seconds in epoc
end_time	query	integer	false	from time in micro-seconds in epoc
after	query	string	false	after cursor for pagination
before	query	string	false	before cursor for pagination
page_size	query	integer	false	number of records per page

ENUMERATED VALUES

Parameter	Value
order_types	market
order_types	limit
order_types	stop_market
order_types	stop_limit
order_types	all_stop

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 123,
      "user_id": 453671,
      "size": 10,
      "unfilled_size": 2,
      "side": "buy",
      "order_type": "limit_order",
      "limit_price": "59000",
      "stop_order_type": "stop_loss_order",
      "stop_price": "55000",
      "paid_commission": "0.5432",
      "commission": "0.5432",
      "reduce_only": false,
    }
  ]
}
```

```
    "client_order_id": "34521712",
    "state": "open",
    "created_at": "1725865012000000",
    "product_id": 27,
    "product_symbol": "BTCUSD"
  }
],
"meta": {
  "after": "g3QAAACZAAKY3J1YXR1ZF9hdHQAAAN",
  "before": "a2PQRSACZAAKY3J1YXR1ZF3fnqHBBBNZL "
}
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of closed and cancelled orders. Order schema	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

GET user fills by filters

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/fills', params={

}, headers = headers)

print r.json()
```

GET /fills

PARAMETERS

Parameter	In	Type	Required	Description
product_ids	query	string	false	comma separated product ids
contract_types	query	string	false	comma separated list of desired contract types
start_time	query	integer	false	from time in micro-seconds in epoc
end_time	query	integer	false	from time in micro-seconds in epoc
after	query	string	false	after cursor for pagination
before	query	string	false	before cursor for pagination
page_size	query	integer	false	number of records per page

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 0,
      "size": 0,
      "fill_type": "normal",
      "side": "buy",
      "price": "string",
      "role": "taker",
      "commission": "string",
      "created_at": "string",
      "product_id": 0,
      "product_symbol": "string",
      "order_id": "string",
      "settling_asset_id": 0,
      "settling_asset_symbol": "string",
      "meta_data": {
        "commission_deto": "string",
        "commission_deto_in_settling_asset": "string",
        "effective_commission_rate": "string",
        "liquidation_fee_deto": "string",
        "liquidation_fee_deto_in_settling_asset": "string",
      }
    }
  ]
}
```

```
    "order_price": "string",
    "order_size": "string",
    "order_type": "string",
    "order_unfilled_size": "string",
    "tfc_used_for_commission": "string",
    "tfc_used_for_liquidation_fee": "string",
    "total_commission_in_settling_asset": "string",
    "total_liquidation_fee_in_settling_asset": "string"
  }
}
],
"meta": {
  "after": "g3QAAACZAAKY3JlYXRlZF9hdHQAAAN",
  "before": "a2PQRSACZAAKY3JlYXRlZF3fnqHBBBNZL"
}
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Array of fills	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
fill_type	normal
fill_type	adl
fill_type	liquidation
fill_type	settlement
fill_type	otc
side	buy
side	sell
role	taker
role	maker

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Download Fills history

Code samples

```
import requests
headers = {
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/fills/history/download/csv', params={

}, headers = headers)

print r.json()
```

GET /fills/history/download/csv

PARAMETERS

Parameter	In	Type	Required	Description
product_ids	query	string	false	comma separated product ids
contract_types	query	string	false	comma separated list of desired contract types
start_time	query	integer	false	from time in micro-seconds in epoc
end_time	query	integer	false	from time in micro-seconds in epoc

RESPONSES

Status	Meaning	Description	Schema
200	OK	csv of fills for the filter query	None

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Orderbook

L2Orderbook

Get L2 orderbook

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}
```

```
r = requests.get('https://api.india.delta.exchange/v2/l2orderbook/{symbol}', params={
}, headers = headers)

print r.json()
```

GET /l2orderbook/{symbol}

PARAMETERS

Parameter	In	Type	Required	Description
symbol	path	string	true	none
depth	query	integer	false	number of levels on each side

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "buy": [
      {
        "depth": "983",
        "price": "9187.5",
        "size": 205640
      }
    ],
    "last_updated_at": 1654589595784000,
    "sell": [
      {
        "depth": "1185",
        "price": "9188.0",
        "size": 113752
      }
    ],
    "symbol": "BTCUSD"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	L2 orderbook for the product	Inline

RESPONSE SCHEMA

✔ This operation does not require authentication.

Trades

Get public trades

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/trades/{symbol}', params={

}, headers = headers)

print r.json()
```

GET /trades/{symbol}

PARAMETERS

Parameter	In	Type	Required	Description
symbol	path	string	true	none

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "trades": [
      {
        "side": "buy",
        "size": 0,
        "price": "string",
        "timestamp": 0
      }
    ]
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of recent trades of the product	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
side	buy
side	sell

✔ This operation does not require authentication.

Wallet

Get balances, Get transaction history

Get Wallet Balances

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/wallet/balances', params={
}, headers = headers)

print r.json()
```

GET /wallet/balances

Example responses

200 Response

```
{
  "meta": {
    "net_equity": "string",
    "robo_trading_equity": "string"
  },
  "result": [
    {
      "asset_id": 0,
      "asset_symbol": "string",
      "available_balance": "string",
      "available_balance_for_robo": "string",
      "balance": "string",
      "blocked_margin": "string",
```

```
    "commission": "string",
    "cross_asset_liability": "string",
    "cross_commission": "string",
    "cross_locked_collateral": "string",
    "cross_order_margin": "string",
    "cross_position_margin": "string",
    "id": 0,
    "interest_credit": "string",
    "order_margin": "string",
    "pending_referral_bonus": "string",
    "pending_trading_fee_credit": "string",
    "portfolio_margin": "string",
    "position_margin": "string",
    "trading_fee_credit": "string",
    "unvested_amount": "string",
    "user_id": 0
  }
],
"success": true
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of wallets attached to the user account	WalletPayload

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get Wallet transactions

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/wallet/transactions', params={

}, headers = headers)

print r.json()
```

GET /wallet/transactions

PARAMETERS

Parameter	In	Type	Required	Description
asset_ids	query	integer	false	comma separated list of asset_ids for which to get txns logs

Parameter	In	Type	Required	Description
start_time	query	integer	false	from time in micro-seconds in epoc
end_time	query	integer	false	from time in micro-seconds in epoc
after	query	string	false	after cursor for pagination
before	query	string	false	before cursor for pagination
page_size	query	integer	false	number of records per page
transaction_types	query	TransactionTypes	false	transaction types to retrieve

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": 0,
      "amount": "string",
      "balance": "string",
      "transaction_type": "string",
      "meta_data": {},
      "product_id": 0,
      "asset_id": 0,
      "asset_symbol": 0,
      "created_at": "string"
    }
  ],
  "meta": {
    "after": "g3QAAAAACZAAKY3JlYXRlZF9hdHQAAAAAN",
    "before": "a2PQRSACZAAKY3JlYXRlZF3fnqHBBBNZL"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	list of Wallet transactions	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
transaction_type	cashflow
transaction_type	deposit
transaction_type	withdrawal
transaction_type	commission

Property	Value
transaction_type	conversion
transaction_type	funding
transaction_type	settlement
transaction_type	liquidation_fee
transaction_type	spot_trade
transaction_type	withdrawal_cancellation
transaction_type	referral_bonus
transaction_type	sub_account_transfer
transaction_type	commission_rebate
transaction_type	promo_credit
transaction_type	trading_credits
transaction_type	trading_credits_forfeited
transaction_type	trading_credits_paid
transaction_type	trading_fee_credits_paid_liquidation_fee
transaction_type	trading_credits_reverted
transaction_type	interest_credit
transaction_type	external_deposit
transaction_type	credit_line
transaction_type	trading_competition
transaction_type	fund_deposit
transaction_type	fund_withdrawal
transaction_type	fund_wallet_deposit
transaction_type	fund_wallet_withdrawal
transaction_type	fund_reward
transaction_type	trade_farming_reward
transaction_type	interest_credit
transaction_type	revert
transaction_type	raf_bonus

Property	Value
transaction_type	fill_appropriation
transaction_type	incident_compensation

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Download Wallet transactions

Code samples

```
import requests
headers = {
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/wallet/transactions/download', params={

}, headers = headers)

print r.json()
```

GET /wallet/transactions/download

PARAMETERS

Parameter	In	Type	Required	Description
asset_ids	query	integer	false	comma separated list of asset_ids
start_time	query	integer	false	from time in micro-seconds in epoc
end_time	query	integer	false	from time in micro-seconds in epoc
after	query	string	false	after cursor for pagination
before	query	string	false	before cursor for pagination
page_size	query	integer	false	number of records per page

RESPONSES

Status	Meaning	Description	Schema
200	OK	csv of transactions for that wallet	None

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Request asset transfer

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/wallets/sub_account_balance_transfer', params={

}, headers = headers)

print r.json()
```

POST /wallets/sub_account_balance_transfer

This api transfers asset from one subaccount to another subaccount or to the main/parent account. Please ensure that the subaccounts involved in the transfer should belong to the same parent account. Requests to transfer assets across subaccounts that belong to different parent accounts will fail. Please make sure that the api key used to make this api request belongs to the main/parent account.

Body parameter

```
{
  "transferrer_user_id": "string",
  "transferee_user_id": "string",
  "asset_symbol": "string",
  "amount": null
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	AssetTransferSubaccountReq	true	none

Example responses

200 Response

```
{
  "success": true,
  "result": null
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns success message	Inline
400	Bad Request	Returns error code	ApiErrorResponse

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Request subaccount balance transfer history.

Code samples

```
import requests
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/wallets/sub_accounts_transfer_history', params={

}, headers = headers)

print r.json()
```

GET /wallets/sub_accounts_transfer_history

This api returns the wallet balance transfers for subaccounts belonging to the parent/main account of an api user. Make sure you are calling this api from the main account. If no subaccount is mentioned in the request, data for all the subaccounts will be returned. Use page size to get more entries in a single request.

Body parameter

```
{
  "subaccount_user_id": "string",
  "before": "string",
  "after": "string",
  "page_size": 10
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	SubaccountTransferHistory	true	none

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "transferrer_user_id": "string",
      "transferee_user_id": "string",
      "asset_symbol": "string",
      "amount": null,
      "created_at": "string",
      "transferee_user": {},
      "transferrer_user": {}
    }
  ],
  "meta": {
    "after": "g3QAAACZAAKY3JlYXRlZF9hdHQAAAN",
    "before": "a2PQRSACZAAKY3JlYXRlZF9fnqHBBBNZL"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns success message	Inline
400	Bad Request	Returns error code	ApiResponseResponse

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Stats

Get Volume Stats

Get volume stats

Code samples

```
import requests
headers = {
  'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/stats', params={

}, headers = headers)
```



```
print r.json()
```

GET /stats

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "last_30_days_volume": 0,
    "last_7_days_volume": 0,
    "total_volume": 0
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	sum of turnover in the last 7 and 30 days along with Total Volume in the last 24 hours (in USD)	Inline

RESPONSE SCHEMA

✔ This operation does not require authentication.

MMP

Market maker protection

Update MMP config

Code samples

```
import requests

headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json',
  'api-key': '****',
  'signature': '****',
  'timestamp': '****'
}

r = requests.put('https://api.india.delta.exchange/v2/users/update_mmp', params={

}, headers = headers)
```

```
print r.json()
```

PUT /users/update_mmp

Channel provides updates when MMP is triggered. Market maker protection is available to registered market makers by default. Others can reach out to support for getting access to MMP. More info [here](#).

Body parameter

```
{
  "asset": "string",
  "window_interval": 0,
  "freeze_interval": 0,
  "trade_limit": "string",
  "delta_limit": "string",
  "vega_limit": "string",
  "mmp": "mmp1"
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	MMPConfigUpdateRequest	true	mmp config for a given underlying asset

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "user_id": 57354187,
    "default_auto_topup": true,
    "mmp_config": null,
    "deto_for_commission": false,
    "vip_level": 0,
    "vip_discount_factor": "0.00",
    "volume_30d": "1060.675333",
    "email_preferences": {
      "adl": true,
      "liquidation": true,
      "margin_topup": false,
      "marketing": true,
      "order_cancel": true,
      "order_fill": true,
      "stop_order_trigger": true
    },
    "notification_preferences": {
      "adl": true,
      "liquidation": true,
      "margin_topup": false,
      "marketing": true,
      "order_cancel": false,
      "order_fill": true,
      "price_alert": true,
      "stop_order_trigger": true
    },
    "price_alert_assets": [
      "BTC",

```

```
    "ETH"  
  ],  
  "enabled_portfolios": {},  
  "interest_credit": false  
}  
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns back the User Preference which contains mmp config	Inline
400	Bad Request	Returns error if mmp is not enabled on the account	ApiResponseResponse

RESPONSE SCHEMA

- ❗ To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Reset MMP

Code samples

```
import requests  
headers = {  
    'Content-Type': 'application/json',  
    'Accept': 'application/json',  
    'api-key': '****',  
    'signature': '****',  
    'timestamp': '****'  
}  
  
r = requests.put('https://api.india.delta.exchange/v2/users/reset_mmp', params={  
  
}, headers = headers)  
  
print r.json()
```

PUT /users/reset_mmp

Body parameter

```
{  
  "asset": "string",  
  "mmp": "mmp1"  
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	MMPResetRequest	true	reset mmp config for a given underlying asset

Example responses

200 Response

```
{
  "success": true
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns back the User Preference which contains mmp config	ApiResponse
400	Bad Request	Returns error if mmp is not enabled on the account	ApiResponse

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Account

Account level settings

Get users trading preferences

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/users/trading_preferences', params={

}, headers = headers)

print r.json()
```

GET /users/trading_preferences

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "user_id": 57354187,
    "default_auto_topup": true,
    "mmp_config": null,
    "deto_for_commission": false,
    "vip_level": 0,
    "vip_discount_factor": "0.00",
    "volume_30d": "1060.675333",
    "email_preferences": {
      "adl": true,
      "liquidation": true,
      "margin_topup": false,
      "marketing": true,
      "order_cancel": true,
      "order_fill": true,
      "stop_order_trigger": true
    },
    "notification_preferences": {
      "adl": true,
      "liquidation": true,
      "margin_topup": false,
      "marketing": true,
      "order_cancel": false,
      "order_fill": true,
      "price_alert": true,
      "stop_order_trigger": true
    },
    "price_alert_assets": [
      "BTC",
      "ETH"
    ],
    "enabled_portfolios": {},
    "interest_credit": false
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	User trading preferences attached to the account	Inline

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Update users trading preferences

Code samples

```
import requests

headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json',
  'api-key': '****',
```

```
'signature': '****',
'timestamp': '****'
}

r = requests.put('https://api.india.delta.exchange/v2/users/trading_preferences', params={

}, headers = headers)

print r.json()
```

PUT /users/trading_preferences

Body parameter

```
{
  "default_auto_topup": true,
  "showMarketOrdersForOptionsInBracket": true,
  "interest_credit": false,
  "email_preferences": {
    "adl": true,
    "liquidation": true,
    "order_fill": true,
    "stop_order_trigger": true,
    "order_cancel": true,
    "marketing": true
  },
  "notification_preferences": {
    "adl": false,
    "liquidation": true,
    "order_fill": true,
    "stop_order_trigger": true,
    "price_alert": true,
    "marketing": true
  }
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	EditUserPreference	true	trading preferences

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "user_id": 57354187,
    "default_auto_topup": true,
    "mmp_config": null,
    "deto_for_commission": false,
    "vip_level": 0,
    "vip_discount_factor": "0.00",
    "volume_30d": "1060.675333",
    "email_preferences": {
      "adl": true,
      "liquidation": true,
      "margin_topup": false,
      "marketing": true,

```

```
    "order_cancel": true,
    "order_fill": true,
    "stop_order_trigger": true
  },
  "notification_preferences": {
    "adl": true,
    "liquidation": true,
    "margin_topup": false,
    "marketing": true,
    "order_cancel": false,
    "order_fill": true,
    "price_alert": true,
    "stop_order_trigger": true
  },
  "price_alert_assets": [
    "BTC",
    "ETH"
  ],
  "enabled_portfolios": {},
  "interest_credit": false
}
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	User trading preferences attached to the account	Inline

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get subaccounts

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/sub_accounts', params={

}, headers = headers)

print r.json()
```

GET /sub_accounts

This api returns all the subaccounts belonging to the same parent/main user. Make sure to call this api from the parent user.

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "id": "98765432",
      "email": "rajtrader2342@gmail.com",
      "account_name": "Main",
      "first_name": "Rajesh",
      "last_name": "Sharma",
      "dob": "1985-08-25",
      "country": "India",
      "phone_number": "9876543210",
      "margin_mode": "isolated",
      "pf_index_symbol": ".DEXBTUSD",
      "is_sub_account": false,
      "is_kyc_done": true
    }
  ]
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Subaccounts belonging to the same parent account.	Inline

RESPONSE SCHEMA

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get user

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/profile', params={

}, headers = headers)

print r.json()
```

GET /profile

This api returns the user object.

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": "98765432",
    "email": "rajtrader2342@gmail.com",
    "account_name": "Main",
    "first_name": "Rajesh",
    "last_name": "Sharma",
    "dob": "1985-08-25",
    "country": "India",
    "phone_number": "9876543210",
    "margin_mode": "isolated",
    "pf_index_symbol": ".DEXBTUSD",
    "is_sub_account": false,
    "is_kyc_done": true
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	User Object	Inline

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Change margin mode

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.put('https://api.india.delta.exchange/v2/users/margin_mode', params={

}, headers = headers)

print r.json()
```

PUT /users/margin_mode

Body parameter

```
{
  "margin_mode": "isolated",
  "subaccount_user_id": "5112346"
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	ChangeMarginModeRequest	true	changes margin mode of the user

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": "5112346",
    "margin_mode": "isolated"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Returns the ChangeMarginModeResponse with the updated margin mode	Inline
400	Bad Request	Returns error if margin mode could not be changed	ApiErrorResponse

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Heartbeat Management

Create Heartbeat

Code samples

```
import requests
headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json',
  'api-key': '****',
  'signature': '****',
  'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/heartbeat/create', params={
```

```
}, headers = headers)

print r.json()
```

POST /heartbeat/create

Body parameter

```
{
  "heartbeat_id": "string",
  "impact": "low",
  "contract_types": [
    "string"
  ],
  "underlying_assets": [
    "string"
  ],
  "product_symbols": [
    "string"
  ],
  "config": [
    {
      "action": "cancel_orders",
      "unhealthy_count": 0,
      "tag": "string"
    }
  ]
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	CreateHeartbeat	true	heartbeat creation details

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "heartbeat_id": "string",
    "status": "string"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Heartbeat created successfully	Inline
400	Bad Request	Returns if heartbeat couldnt be created	ApiResponseResponse

RESPONSE SCHEMA

! To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Send Heartbeat Acknowledgment

Code samples

```
import requests

headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.post('https://api.india.delta.exchange/v2/heartbeat', params={

}, headers = headers)

print r.json()
```

POST /heartbeat

Body parameter

```
{
  "heartbeat_id": "string",
  "ttl": 0
}
```

PARAMETERS

Parameter	In	Type	Required	Description
body	body	HeartbeatAck	true	heartbeat acknowledgment details

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "process_enabled": "string",
    "heartbeat_timestamp": "string"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	Heartbeat acknowledged successfully	Inline
400	Bad Request	Returns if heartbeat acknowledgment failed	ApiResponseResponse

RESPONSE SCHEMA

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Get Heartbeats

Code samples

```
import requests
headers = {
    'Accept': 'application/json',
    'api-key': '****',
    'signature': '****',
    'timestamp': '****'
}

r = requests.get('https://api.india.delta.exchange/v2/heartbeat', params={
    'user_id': '0'
}, headers = headers)

print r.json()
```

GET /heartbeat

PARAMETERS

Parameter	In	Type	Required	Description
user_id	query	integer	true	User ID
heartbeat_id	query	string	false	Specific heartbeat ID to retrieve

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "heartbeat_id": "string",
      "impact": "string",
      "contract_types": [
        "string"
      ],
      "underlying_assets": [
        "string"
      ]
    }
  ]
}
```

```
    ],
    "product_symbols": [
      "string"
    ],
    "config": [
      {
        "action": "cancel_orders",
        "unhealthy_count": 0,
        "tag": "string"
      }
    ],
    "status": "string",
    "last_ack": "string",
    "next_ack_required_by": "string"
  }
]
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of active heartbeats	Inline
400	Bad Request	Returns if heartbeats couldnt be retrieved	ApiErrorResponse

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
action	cancel_orders
action	spreads

 To perform this operation, you must be sign the request using your api key and secret. See Authentication section for more details.

Settlement Prices

Get product settlement prices

Code samples

```
import requests
headers = {
  'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/products/?states=expired', params={

}, headers = headers)
```

```
print r.json()
```

GET /products/?states=expired

PARAMETERS

Parameter	In	Type	Required	Description
states	query	string	false	Comma separated list of states e.g. to get expired contracts https://api.india.delta.exchange/v2/products?contract_types=call_options&states=expired
page_size	query	string	false	size of a single page for paginated request, default: 100

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "id": 27,
    "symbol": "BTCUSD",
    "description": "Bitcoin Perpetual futures, quoted, settled & margined in USD",
    "created_at": "2023-12-18T13:10:39Z",
    "updated_at": "2024-11-15T02:47:50Z",
    "settlement_time": null,
    "notional_type": "vanilla",
    "impact_size": 10000,
    "initial_margin": "0.5",
    "maintenance_margin": "0.25",
    "contract_value": "0.001",
    "contract_unit_currency": "BTC",
    "tick_size": "0.5",
    "product_specs": {
      "funding_clamp_value": 0.05,
      "only_reduce_only_orders_allowed": false,
      "tags": [
        "layer_1"
      ]
    },
    "state": "live",
    "trading_status": "operational",
    "max_leverage_notional": "100000",
    "default_leverage": "200",
    "initial_margin_scaling_factor": "0.0000025",
    "maintenance_margin_scaling_factor": "0.00000125",
    "taker_commission_rate": "0.0005",
    "maker_commission_rate": "0.0002",
    "liquidation_penalty_factor": "0.5",
    "contract_type": "perpetual_futures",
    "position_size_limit": 229167,
    "basis_factor_max_limit": "10.95",
    "is_quanto": false,
    "funding_method": "mark_price",
    "annualized_funding": "10.95",
    "price_band": "2.5",
    "underlying_asset": {
      "id": 14,
      "symbol": "USD",
      "precision": 8,
      "deposit_status": "enabled",
      "withdrawal_status": "enabled",
```

```
    "base_withdrawal_fee": "0.0000000000000000",
    "min_withdrawal_amount": "0.0000000000000000"
  },
  "quoting_asset": {
    "id": 14,
    "symbol": "USD",
    "precision": 8,
    "deposit_status": "enabled",
    "withdrawal_status": "enabled",
    "base_withdrawal_fee": "0.0000000000000000",
    "min_withdrawal_amount": "0.0000000000000000"
  },
  "settling_asset": {
    "id": 14,
    "symbol": "USD",
    "precision": 8,
    "deposit_status": "enabled",
    "withdrawal_status": "enabled",
    "base_withdrawal_fee": "0.0000000000000000",
    "min_withdrawal_amount": "0.0000000000000000"
  },
  "spot_index": {
    "id": 14,
    "symbol": ".DEXBTUSD",
    "constituent_exchanges": [
      {
        "name": "ExchangeA",
        "weight": 0.25
      }
    ],
    "underlying_asset_id": 13,
    "quoting_asset_id": 14,
    "tick_size": "0.5",
    "index_type": "spot_pair"
  }
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	List of products	Inline

RESPONSE SCHEMA

ENUMERATED VALUES

Property	Value
notional_type	vanilla
notional_type	inverse
state	live
state	expired
state	upcoming
trading_status	operational

Property	Value
trading_status	disrupted_cancel_only
trading_status	disrupted_post_only
deposit_status	enabled
deposit_status	disabled
withdrawal_status	enabled
withdrawal_status	disabled
index_type	spot_pair
index_type	fixed_interest_rate
index_type	floating_interest_rate

✔ This operation does not require authentication.

Historical OHLC Candles/Sparklines

GET historical ohlc candles

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/history/candles', params={
    'resolution': '5m', 'symbol': 'BTCUSD', 'start': '1685618835', 'end': '1722511635'
}, headers = headers)

print r.json()
```

GET /history/candles

It returns historical Open-High-Low-Close(ohlc) candles data of the symbol as per input values for resolution, start time and end time. Also, it can return only upto 2000 candles maximum in a response.

PARAMETERS

Parameter	In	Type	Required	Description
resolution	query	string	true	ohlc candle time frames like 1m, 5m, 1h

Parameter	In	Type	Required	Description
symbol	query	string	true	To get funding history pass symbol as FUNDING:\${symbol}, mark price MARK:\${symbol} and OI data OI:\${symbol} for e.g. - FUNDING:BTCUSD, MARK:C-BTC-66400-010824, OI:ETHUSD
start	query	integer	true	Start time: unix timestamp in seconds
end	query	integer	true	End time: unix timestamp in seconds

ENUMERATED VALUES

Parameter	Value
resolution	1m
resolution	3m
resolution	5m
resolution	15m
resolution	30m
resolution	1h
resolution	2h
resolution	4h
resolution	6h
resolution	1d
resolution	7d
resolution	30d
resolution	1w
resolution	2w

Example responses

200 Response

```
{
  "success": true,
  "result": [
    {
      "time": 0,
      "open": 0,
      "high": 0,
      "low": 0,
      "close": 0,
      "volume": 0
    }
  ]
}
```

```
    ]
  }
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	ohlcv	Inline

RESPONSE SCHEMA

 This operation does not require authentication.

GET product history sparklines

Code samples

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('https://api.india.delta.exchange/v2/history/sparklines', params={
    'symbols': 'ETHUSD,MARK:BTCUSD'
}, headers = headers)

print r.json()
```

GET /history/sparklines

PARAMETERS

Parameter	In	Type	Required	Description
symbols	query	string	true	comma separated product symbols

Example responses

200 Response

```
{
  "success": true,
  "result": {
    "ETHUSD": [
      [
        1594214051,
        0.00003826
      ],
      [
        1594214051,
        0.00003826
      ]
    ],
    "MARK:BTCUSD": [
```

```
[
  1594215270,
  0.00003826
]
}
```

RESPONSES

Status	Meaning	Description	Schema
200	OK	product history sparkline	Inline

RESPONSE SCHEMA

✔ This operation does not require authentication.

Schemas

ApiSuccessResponse

```
{
  "success": true
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
success	boolean	false	none	none

ApiErrorResponse

```
{
  "success": false,
  "error": {}
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
success	boolean	false	none	none

Name	Type	Required	Restrictions	Description
error	object	false	none	none

Index

```
{
  "id": 14,
  "symbol": ".DEXBTUSD",
  "constituent_exchanges": [
    {
      "name": "ExchangeA",
      "weight": 0.25
    }
  ],
  "underlying_asset_id": 13,
  "quoting_asset_id": 14,
  "tick_size": "0.5",
  "index_type": "spot_pair"
}
```

Details of an index used in trading, including its constituents and characteristics.

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer(int64)	false	none	Unique identifier for the index.
symbol	string	false	none	Symbol representing the index, typically prefixed by '.DE' followed by base asset and quoting asset.
constituent_exchanges	[object]	false	none	Details of constituent exchanges, including their names and weights in the index.
» name	string	false	none	Name of the constituent exchange.
» weight	number	false	none	Weight of the exchange in the index.
underlying_asset_id	integer	false	none	ID of the underlying asset for the index.
quoting_asset_id	integer	false	none	ID of the quoting asset for the index.
tick_size	string	false	none	Precision of the spot price in decimal format.
index_type	string	false	none	Type of the index.

ENUMERATED VALUES

Property	Value
index_type	spot_pair

Property	Value
index_type	fixed_interest_rate
index_type	floating_interest_rate

ArrayOfIndices

```
[
  {
    "id": 14,
    "symbol": ".DEXBTUSD",
    "constituent_exchanges": [
      {
        "name": "ExchangeA",
        "weight": 0.25
      }
    ],
    "underlying_asset_id": 13,
    "quoting_asset_id": 14,
    "tick_size": "0.5",
    "index_type": "spot_pair"
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[Index]	false	none	[Details of an index used in trading, including its constituents and characteristics.]

ProductSpecs

```
{
  "funding_clamp_value": 0.05,
  "only_reduce_only_orders_allowed": false,
  "tags": [
    "layer_1"
  ]
}
```

Specifications related to the specific product or contract.

PROPERTIES

Name	Type	Required	Restrictions	Description
funding_clamp_value	number	false	none	The maximum allowable funding rate clamp value.

Name	Type	Required	Restrictions	Description
only_reduce_only_orders_allowed	boolean	false	none	Indicates whether only reduce-only orders are allowed.
tags	[string]	false	none	Tags associated with the product specifications.

Asset

```
{
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.0000000000000000",
  "min_withdrawal_amount": "0.0000000000000000"
}
```

Details of the asset used in the product or contract.

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer(int64)	false	none	Unique identifier for the asset.
symbol	string	false	none	Symbol representing the asset.
precision	integer	false	none	Number of decimal places supported for the asset.
deposit_status	string	false	none	Indicates if deposits are enabled for the asset.
withdrawal_status	string	false	none	Indicates if withdrawals are enabled for the asset.
base_withdrawal_fee	string	false	none	Fixed withdrawal fee for the asset.
min_withdrawal_amount	string	false	none	Minimum allowable withdrawal amount for the asset.

ENUMERATED VALUES

Property	Value
deposit_status	enabled
deposit_status	disabled
withdrawal_status	enabled
withdrawal_status	disabled

ArrayOfAssets

```
[
  {
    "id": 14,
    "symbol": "USD",
    "precision": 8,
    "deposit_status": "enabled",
    "withdrawal_status": "enabled",
    "base_withdrawal_fee": "0.000000000000000000",
    "min_withdrawal_amount": "0.000000000000000000"
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[Asset]	false	none	[Details of the asset used in the product or contract.]

Product

```
{
  "id": 27,
  "symbol": "BTCUSD",
  "description": "Bitcoin Perpetual futures, quoted, settled & margined in USD",
  "created_at": "2023-12-18T13:10:39Z",
  "updated_at": "2024-11-15T02:47:50Z",
  "settlement_time": null,
  "notional_type": "vanilla",
  "impact_size": 10000,
  "initial_margin": "0.5",
  "maintenance_margin": "0.25",
  "contract_value": "0.001",
  "contract_unit_currency": "BTC",
  "tick_size": "0.5",
  "product_specs": {
    "funding_clamp_value": 0.05,
    "only_reduce_only_orders_allowed": false,
    "tags": [
      "layer_1"
    ]
  },
  "state": "live",
  "trading_status": "operational",
  "max_leverage_notional": "100000",
  "default_leverage": "200",
  "initial_margin_scaling_factor": "0.0000025",
  "maintenance_margin_scaling_factor": "0.00000125",
  "taker_commission_rate": "0.0005",
  "maker_commission_rate": "0.0002",
  "liquidation_penalty_factor": "0.5",
  "contract_type": "perpetual_futures",
  "position_size_limit": 229167,
  "basis_factor_max_limit": "10.95",
  "is_quanto": false,
```



```
"funding_method": "mark_price",
"annualized_funding": "10.95",
"price_band": "2.5",
"underlying_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.000000000000000000",
  "min_withdrawal_amount": "0.000000000000000000"
},
"quoting_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.000000000000000000",
  "min_withdrawal_amount": "0.000000000000000000"
},
"settling_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.000000000000000000",
  "min_withdrawal_amount": "0.000000000000000000"
},
"spot_index": {
  "id": 14,
  "symbol": ".DEXBTUSD",
  "constituent_exchanges": [
    {
      "name": "ExchangeA",
      "weight": 0.25
    }
  ],
  "underlying_asset_id": 13,
  "quoting_asset_id": 14,
  "tick_size": "0.5",
  "index_type": "spot_pair"
}
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer(int64)	false	none	Unique identifier of a product or contract.
symbol	string	false	none	Symbol of the product or contract like BTCUSD, ETHUSD.
description	string	false	none	Detailed description of the product or contract.
created_at	string	false	none	Creation timestamp of the product or contract.
updated_at	string	false	none	Last update timestamp of the product or contract.
settlement_time	string	false	none	Settlement timestamp for futures contracts.

Name	Type	Required	Restrictions	Description
notional_type	string	false	none	Type of notional calculation.
impact_size	integer	false	none	Size of a typical trade used for mark price computation.
initial_margin	string	false	none	Margin required to open a position.
maintenance_margin	string	false	none	Minimum margin required to maintain a position.
contract_value	string	false	none	Notional value of the contract (spot price x contract amount).
contract_unit_currency	string	false	none	Unit of the contract (underlying asset or settling asset).
tick_size	string	false	none	Minimum price interval between two successive prices.
product_specs	ProductSpecs	false	none	Specifications related to the specific product or contract.
state	string	false	none	Current state of the product.
trading_status	string	false	none	Trading status of the contract.
max_leverage_notional	string	false	none	Maximum notional position size at the highest leverage.
default_leverage	string	false	none	Default leverage assigned to the product.
initial_margin_scaling_factor	string	false	none	Scaling factor for initial margin.
maintenance_margin_scaling_factor	string	false	none	Scaling factor for maintenance margin.
taker_commission_rate	string	false	none	Commission rate for taker trades.
maker_commission_rate	string	false	none	Commission rate for maker trades.
liquidation_penalty_factor	string	false	none	Factor used to calculate liquidation penalty.
contract_type	string	false	none	Type of contract (e.g., futures, perpetual).
position_size_limit	integer	false	none	Maximum size for a single contract order.
basis_factor_max_limit	string	false	none	Maximum value for annualized basis.
is_quanto	boolean	false	none	Indicates if the contract is quanto.
funding_method	string	false	none	Method used for funding calculation.
annualized_funding	string	false	none	Maximum allowed annualized funding rate.
price_band	string	false	none	Price range allowed around the mark price (percentage).

Name	Type	Required	Restrictions	Description
underlying_asset	Asset	false	none	Details of the asset used in the product or contract.
quoting_asset	Asset	false	none	Details of the asset used in the product or contract.
settling_asset	Asset	false	none	Details of the asset used in the product or contract.
spot_index	Index	false	none	Details of an index used in trading, including its constituents and characteristics.

ENUMERATED VALUES

Property	Value
notional_type	vanilla
notional_type	inverse
state	live
state	expired
state	upcoming
trading_status	operational
trading_status	disrupted_cancel_only
trading_status	disrupted_post_only

ProductCategories

```
{
  "PutOptions": "string",
  "CallOptions": "string",
  "MoveOptions": "string",
  "Spot": "string",
  "Futures": "string",
  "Perpetual Futures": "string"
}
```

List of all the product category names on delta exchange. Please refer to this list while subscribing to various public and private channels on delta exchange websocket

PROPERTIES

Name	Type	Required	Restrictions	Description
PutOptions	string	false	none	put_options
CallOptions	string	false	none	call_options

Name	Type	Required	Restrictions	Description
MoveOptions	string	false	none	move_options
Spot	string	false	none	spot
Futures	string	false	none	futures
Perpetual Futures	string	false	none	perpetual_futures

ArrayOfProducts

```
[
  {
    "id": 27,
    "symbol": "BTCUSD",
    "description": "Bitcoin Perpetual futures, quoted, settled & margined in USD",
    "created_at": "2023-12-18T13:10:39Z",
    "updated_at": "2024-11-15T02:47:50Z",
    "settlement_time": null,
    "notional_type": "vanilla",
    "impact_size": 10000,
    "initial_margin": "0.5",
    "maintenance_margin": "0.25",
    "contract_value": "0.001",
    "contract_unit_currency": "BTC",
    "tick_size": "0.5",
    "product_specs": {
      "funding_clamp_value": 0.05,
      "only_reduce_only_orders_allowed": false,
      "tags": [
        "layer_1"
      ]
    },
    "state": "live",
    "trading_status": "operational",
    "max_leverage_notional": "100000",
    "default_leverage": "200",
    "initial_margin_scaling_factor": "0.0000025",
    "maintenance_margin_scaling_factor": "0.00000125",
    "taker_commission_rate": "0.0005",
    "maker_commission_rate": "0.0002",
    "liquidation_penalty_factor": "0.5",
    "contract_type": "perpetual_futures",
    "position_size_limit": 229167,
    "basis_factor_max_limit": "10.95",
    "is_quanto": false,
    "funding_method": "mark_price",
    "annualized_funding": "10.95",
    "price_band": "2.5",
    "underlying_asset": {
      "id": 14,
      "symbol": "USD",
      "precision": 8,
      "deposit_status": "enabled",
      "withdrawal_status": "enabled",
      "base_withdrawal_fee": "0.000000000000000000",
      "min_withdrawal_amount": "0.000000000000000000"
    }
  },
]
```

```
"quoting_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.0000000000000000",
  "min_withdrawal_amount": "0.0000000000000000"
},
"settling_asset": {
  "id": 14,
  "symbol": "USD",
  "precision": 8,
  "deposit_status": "enabled",
  "withdrawal_status": "enabled",
  "base_withdrawal_fee": "0.0000000000000000",
  "min_withdrawal_amount": "0.0000000000000000"
},
"spot_index": {
  "id": 14,
  "symbol": ".DEXBTUSD",
  "constituent_exchanges": [
    {
      "name": "ExchangeA",
      "weight": 0.25
    }
  ],
  "underlying_asset_id": 13,
  "quoting_asset_id": 14,
  "tick_size": "0.5",
  "index_type": "spot_pair"
}
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[Product]	false	none	none

Order

```
{
  "id": 123,
  "user_id": 453671,
  "size": 10,
  "unfilled_size": 2,
  "side": "buy",
  "order_type": "limit_order",
  "limit_price": "59000",
  "stop_order_type": "stop_loss_order",
  "stop_price": "55000",
  "paid_commission": "0.5432",
  "commission": "0.5432",
  "reduce_only": false,
  "client_order_id": "34521712",
  "state": "open",
}
```

```
"created_at": "1725865012000000",
"product_id": 27,
"product_symbol": "BTCUSD"
}
```

An Order object

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer	false	none	Generated order id
user_id	integer	false	none	Client id
size	integer	false	none	Order size
unfilled_size	integer	false	none	Order size which is not filled yet
side	string	false	none	Side for which to place order
order_type	string	false	none	Order type - limit_order/market_order
limit_price	string	false	none	Price level on which order must be triggered
stop_order_type	string	false	none	Stop order type - stop loss or take profit
stop_price	string	false	none	Stop price level for the stop order
paid_commission	string	false	none	Commission paid for filled order
commission	string	false	none	Commission blocked for order
reduce_only	string	false	none	if set, will only close positions. New orders will not be placed
client_order_id	string	false	none	client order id provided by the user while creating order
state	string	false	none	Order Status
created_at	string	false	none	Created at unix timestamp of the order in micro seconds
product_id	integer	false	none	Product id of the ordered product
product_symbol	string	false	none	Product symbol of the ordered product

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order

Property	Value
stop_order_type	stop_loss_order
reduce_only	false
reduce_only	true
state	open
state	pending
state	closed
state	cancelled

ArrayOfOrders

```
[
  {
    "id": 123,
    "user_id": 453671,
    "size": 10,
    "unfilled_size": 2,
    "side": "buy",
    "order_type": "limit_order",
    "limit_price": "59000",
    "stop_order_type": "stop_loss_order",
    "stop_price": "55000",
    "paid_commission": "0.5432",
    "commission": "0.5432",
    "reduce_only": false,
    "client_order_id": "34521712",
    "state": "open",
    "created_at": "1725865012000000",
    "product_id": 27,
    "product_symbol": "BTCUSD"
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	[Order]	false	none	[An Order object]

CreateOrderRequest

```
{
  "product_id": 27,
```

```

"product_symbol": "BTCUSD",
"limit_price": "59000",
"size": 10,
"side": "buy",
"order_type": "limit_order",
"stop_order_type": "stop_loss_order",
"stop_price": "56000",
"trail_amount": "50",
"stop_trigger_method": "last_traded_price",
"bracket_stop_trigger_method": "last_traded_price",
"bracket_stop_loss_limit_price": "57000",
"bracket_stop_loss_price": "56000",
"bracket_trail_amount": "50",
"bracket_take_profit_limit_price": "62000",
"bracket_take_profit_price": "61000",
"time_in_force": "gtc",
"mmp": "disabled",
"post_only": false,
"reduce_only": false,
"client_order_id": "34521712",
"cancel_orders_accepted": false
}

```

A create order object

PROPERTIES

Name	Type	Required	Restrictions	Description
product_id	integer	true	none	Only one of either product_id or product_symbol must be sent.
product_symbol	string	true	none	Only one of either product_id or product_symbol must be sent.
limit_price	string	false	none	Price level for limit orders
size	integer	true	none	Order size
side	string	true	none	Buy order or Sell order
order_type	string	true	none	Limit order(limit_price must be defined) or Market order
stop_order_type	string	false	none	Stop order type - stop loss or take profit
stop_price	string	false	none	Stop loss price level if the order is stop order
trail_amount	string	false	none	Use trail amount if you want a trailing stop order. Required if stop price is empty.
stop_trigger_method	string	false	none	Stop order trigger method - mark_price/last_traded_price/spot_price
bracket_stop_trigger_method	string	false	none	stop order trigger method for bracket orders - mark_price/last_traded_price/spot_price
bracket_stop_loss_limit_price	string	false	none	Bracket order stop loss limit price
bracket_stop_loss_price	string	false	none	Bracket order stop loss trigger price
bracket_trail_amount	string	false	none	use bracket trail amount if you want a trailing stop order. Required if bracket stop price is empty

Name	Type	Required	Restrictions	Description
bracket_take_profit_limit_price	string	false	none	Bracket order take profit limit price
bracket_take_profit_price	string	false	none	take profit trigger price for bracket order
time_in_force	string	false	none	GTC/IOC order type
mmp	string	false	none	MMP level for the order - disabled/mmp1/mmp2/mmp3/mmp4/mmp5
post_only	string	false	none	Post only order
reduce_only	string	false	none	if set, will only close positions. New orders will not be placed
client_order_id	string	false	none	client order id provided by the user while creating order
cancel_orders_accepted	string	false	none	if set, will cancel all existing orders for the product

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
stop_order_type	stop_loss_order
stop_order_type	take_profit_order
stop_trigger_method	mark_price
stop_trigger_method	last_traded_price
stop_trigger_method	spot_price
bracket_stop_trigger_method	mark_price
bracket_stop_trigger_method	last_traded_price
bracket_stop_trigger_method	spot_price
time_in_force	gtc
time_in_force	ioc
mmp	disabled
mmp	mmp1
mmp	mmp2

Property	Value
mmp	mmp3
mmp	mmp4
mmp	mmp5
post_only	true
post_only	false
reduce_only	true
reduce_only	false
cancel_orders_accepted	true
cancel_orders_accepted	false

BatchCreateOrder

```
{
  "limit_price": "59000",
  "size": 10,
  "side": "buy",
  "order_type": "limit_order",
  "time_in_force": "gtc",
  "mmp": "disabled",
  "post_only": false,
  "client_order_id": "34521712"
}
```

A create order object

PROPERTIES

Name	Type	Required	Restrictions	Description
limit_price	string	false	none	Price level for limit orders
size	integer	true	none	Order size
side	string	true	none	Buy order or Sell order
order_type	string	true	none	Limit order(limit_price must be defined) or Market order
time_in_force	string	false	none	GTC/IOC order type
mmp	string	false	none	MMP level for the order - disabled/mmp1/mmp2/mmp3/mmp4/mmp5
post_only	string	false	none	Post only order

Name	Type	Required	Restrictions	Description
client_order_id	string	false	none	client order id provided by the user while creating order

ENUMERATED VALUES

Property	Value
side	buy
side	sell
order_type	limit_order
order_type	market_order
time_in_force	gtc
time_in_force	ioc
mmp	disabled
mmp	mmp1
mmp	mmp2
mmp	mmp3
mmp	mmp4
mmp	mmp5
post_only	true
post_only	false

BatchCreateOrdersRequest

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "orders": [
    {
      "limit_price": "59000",
      "size": 10,
      "side": "buy",
      "order_type": "limit_order",
      "time_in_force": "gtc",
      "mmp": "disabled",
      "post_only": false,
      "client_order_id": "34521712"
    }
  ]
}
```

```
}

```

PROPERTIES

Name	Type	Required	Restrictions	Description
product_id	integer	false	none	Only one of either product_id or product_symbol must be sent.
product_symbol	string	false	none	Only one of either product_id or product_symbol must be sent.
orders	[BatchCreateOrder]	false	none	[A create order object]

oneOf

Name	Type	Required	Restrictions	Description
anonymous	object	false	none	none

xor

Name	Type	Required	Restrictions	Description
anonymous	object	false	none	none

ArrayOfCreateOrderRequest

```
[
  {
    "product_id": 27,
    "product_symbol": "BTCUSD",
    "limit_price": "59000",
    "size": 10,
    "side": "buy",
    "order_type": "limit_order",
    "stop_order_type": "stop_loss_order",
    "stop_price": "56000",
    "trail_amount": "50",
    "stop_trigger_method": "last_traded_price",
    "bracket_stop_trigger_method": "last_traded_price",
    "bracket_stop_loss_limit_price": "57000",
    "bracket_stop_loss_price": "56000",
    "bracket_trail_amount": "50",
    "bracket_take_profit_limit_price": "62000",
    "bracket_take_profit_price": "61000",
    "time_in_force": "gtc",
    "mmp": "disabled",
    "post_only": false,
    "reduce_only": false,
    "client_order_id": "34521712",
    "cancel_orders_accepted": false
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[CreateOrderRequest]	false	none	[A create order object]

EditOrderRequest

```
{
  "id": 34521712,
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "limit_price": "59000",
  "size": 15,
  "mmp": "disabled",
  "post_only": false,
  "cancel_orders_accepted": false,
  "stop_price": "56000",
  "trail_amount": "50"
}
```

edit order object

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer	true	none	existing order id to be edited
product_id	integer	true	none	Only one of either product_id or product_symbol must be sent.
product_symbol	string	true	none	Only one of either product_id or product_symbol must be sent.
limit_price	string	false	none	Price level for limit orders
size	integer	true	none	total size after editing order
mmp	string	false	none	MMP level for the order - disabled/mmp1/mmp2/mmp3/mmp4/mmp5
post_only	string	false	none	Post only order
cancel_orders_accepted	string	false	none	if set, will cancel all existing orders for the product
stop_price	string	false	none	price to trigger stop order
trail_amount	string	false	none	Use trail amount if you want a trailing stop order. Required if stop price is empty.

ENUMERATED VALUES

Property	Value
mmp	disabled
mmp	mmp1
mmp	mmp2
mmp	mmp3
mmp	mmp4
mmp	mmp5
post_only	true
post_only	false
cancel_orders_accepted	true
cancel_orders_accepted	false

BatchEditOrder

```
{
  "id": 34521712,
  "limit_price": "59000",
  "size": 15,
  "mmp": "disabled",
  "post_only": false
}
```

edit order object

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer	true	none	existing order id to be edited
limit_price	string	false	none	Price level for limit orders
size	integer	true	none	total size after editing order
mmp	string	false	none	MMP level for the order - disabled/mmp1/mmp2/mmp3/mmp4/mmp5
post_only	string	false	none	Post only order

ENUMERATED VALUES

Property	Value
mmp	disabled
mmp	mmp1
mmp	mmp2
mmp	mmp3
mmp	mmp4
mmp	mmp5
post_only	false
post_only	true

BatchEditOrdersRequest

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "orders": [
    {
      "id": 34521712,
      "limit_price": "59000",
      "size": 15,
      "mmp": "disabled",
      "post_only": false
    }
  ]
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
product_id	integer	false	none	Only one of either product_id or product_symbol must be sent.
product_symbol	string	false	none	Only one of either product_id or product_symbol must be sent.
orders	[BatchEditOrder]	false	none	[edit order object]

oneOf

Name	Type	Required	Restrictions	Description
anonymous	object	false	none	none

xor

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	object	false	none	none

CreateBracketOrderRequest

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "stop_loss_order": {
    "order_type": "limit_order",
    "stop_price": "56000",
    "trail_amount": "50",
    "limit_price": "55000"
  },
  "take_profit_order": {
    "order_type": "limit_order",
    "stop_price": "65000",
    "limit_price": "64000"
  },
  "bracket_stop_trigger_method": "last_traded_price"
}
```

bracket order object

PROPERTIES

Name	Type	Required	Restrictions	Description
product_id	integer	true	none	Only one of either product_id or product_symbol must be sent.
product_symbol	string	true	none	Only one of either product_id or product_symbol must be sent.
stop_loss_order	object	false	none	none
» order_type	string	false	none	Limit order(limit_price must be defined) or Market order
» stop_price	string	false	none	Stop loss price level
» trail_amount	string	false	none	Use trail amount if you want a trailing stop order. Required if stop price is empty.
» limit_price	string	false	none	required for limit orders
take_profit_order	object	false	none	none
» order_type	string	false	none	Limit order(limit_price must be defined) or Market order
» stop_price	string	false	none	Stop price level
» limit_price	string	false	none	required for limit orders

Name	Type	Required	Restrictions	Description
bracket_stop_trigger_method	string	false	none	stop order trigger method for bracket orders- mark_price/last_traded_price/spot_price

ENUMERATED VALUES

Property	Value
order_type	limit_order
order_type	market_order
order_type	limit_order
order_type	market_order
bracket_stop_trigger_method	mark_price
bracket_stop_trigger_method	last_traded_price
bracket_stop_trigger_method	spot_price

EditBracketOrderRequest

```
{
  "id": 34521712,
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "bracket_stop_loss_limit_price": "55000",
  "bracket_stop_loss_price": "56000",
  "bracket_take_profit_limit_price": "65000",
  "bracket_take_profit_price": "64000",
  "bracket_trail_amount": "50",
  "bracket_stop_trigger_method": "last_traded_price"
}
```

bracket order object

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer	true	none	Order ID for which bracket params are being updated
product_id	integer	true	none	Only one of either product_id or product_symbol must be sent.
product_symbol	string	true	none	Only one of either product_id or product_symbol must be sent.
bracket_stop_loss_limit_price	string	false	none	stop loss limit price for bracket order
bracket_stop_loss_price	string	false	none	stop loss trigger price for bracket order

Name	Type	Required	Restrictions	Description
bracket_take_profit_limit_price	string	false	none	take profit limit price for bracket order
bracket_take_profit_price	string	false	none	take profit trigger price for bracket order
bracket_trail_amount	string	false	none	trail amount of bracket order
bracket_stop_trigger_method	string	false	none	stop order trigger method for bracket orders- mark_price/last_traded_price/spot_price

ENUMERATED VALUES

Property	Value
bracket_stop_trigger_method	mark_price
bracket_stop_trigger_method	last_traded_price
bracket_stop_trigger_method	spot_price

BatchDeleteOrder

```
{
  "id": 13452112,
  "client_order_id": "34521712"
}
```

A delete order object

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer	false	none	use bracket trail amount if you want a trailing stop order. Required if bracket stop price is empty
client_order_id	string	false	none	client order id provided by the user while creating order

DeleteOrderRequest

```
{
  "id": 13452112,
  "client_order_id": "34521712",
  "product_id": 27
}
```

A delete order object

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer	false	none	use bracket trail amount if you want a trailing stop order. Required if bracket stop price is empty
client_order_id	string	false	none	client order id provided by the user while creating order
product_id	integer	false	none	product_id of the product in the order

CancelAllFilterObject

```
{
  "product_id": 27,
  "contract_types": "perpetual_futures,put_options,call_options",
  "cancel_limit_orders": false,
  "cancel_stop_orders": false,
  "cancel_reduce_only_orders": false
}
```

Cancel all request filter object

PROPERTIES

Name	Type	Required	Restrictions	Description
product_id	integer	false	none	Only one of either product_id or product_symbol must be sent.
contract_types	string	false	none	comma separated list of desired contract types
cancel_limit_orders	string	false	none	set true to cancel open limit orders
cancel_stop_orders	string	false	none	set as true to cancel stop orders
cancel_reduce_only_orders	string	false	none	set as true to cancel reduce only orders

ENUMERATED VALUES

Property	Value
cancel_limit_orders	true
cancel_limit_orders	false
cancel_stop_orders	true
cancel_stop_orders	false
cancel_reduce_only_orders	true

Property	Value
cancel_reduce_only_orders	false

BatchDeleteOrdersRequest

```
{
  "product_id": 27,
  "product_symbol": "BTCUSD",
  "orders": [
    {
      "id": 13452112,
      "client_order_id": "34521712"
    }
  ]
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
product_id	integer	false	none	Only one of either product_id or product_symbol must be sent.
product_symbol	string	false	none	Only one of either product_id or product_symbol must be sent.
orders	[BatchDeleteOrder]	false	none	[A delete order object]

oneOf

Name	Type	Required	Restrictions	Description
anonymous	object	false	none	none

xor

Name	Type	Required	Restrictions	Description
anonymous	object	false	none	none

Position

```
{
  "user_id": 0,
  "size": 0,
  "entry_price": "string",
  "margin": "string",
  "liquidation_price": "string",
  "bankruptcy_price": "string",
}
```

```
"adl_level": 0,
"product_id": 0,
"product_symbol": "string",
"commission": "string",
"realized_pnl": "string",
"realized_funding": "string"
}
```

A position object

PROPERTIES

Name	Type	Required	Restrictions	Description
user_id	integer	false	none	none
size	integer	false	none	Position size, negative for short and positive for long
entry_price	string	false	none	none
margin	string	false	none	none
liquidation_price	string	false	none	none
bankruptcy_price	string	false	none	none
adl_level	integer	false	none	none
product_id	integer	false	none	none
product_symbol	string	false	none	none
commission	string	false	none	commissions blocked in the position
realized_pnl	string	false	none	Net realized pnl since the position was opened
realized_funding	string	false	none	Net realized funding since the position was opened

ArrayOfPositions

```
[
{
  "user_id": 0,
  "size": 0,
  "entry_price": "string",
  "margin": "string",
  "liquidation_price": "string",
  "bankruptcy_price": "string",
  "adl_level": 0,
  "product_id": 0,
  "product_symbol": "string",
  "commission": "string",
  "realized_pnl": "string",
  "realized_funding": "string"
}
```

```
]

```

PROPERTIES

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	[Position]	false	none	[A position object]

Fill

```
{
  "id": 0,
  "size": 0,
  "fill_type": "normal",
  "side": "buy",
  "price": "string",
  "role": "taker",
  "commission": "string",
  "created_at": "string",
  "product_id": 0,
  "product_symbol": "string",
  "order_id": "string",
  "settling_asset_id": 0,
  "settling_asset_symbol": "string",
  "meta_data": {
    "commission_deto": "string",
    "commission_deto_in_settling_asset": "string",
    "effective_commission_rate": "string",
    "liquidation_fee_deto": "string",
    "liquidation_fee_deto_in_settling_asset": "string",
    "order_price": "string",
    "order_size": "string",
    "order_type": "string",
    "order_unfilled_size": "string",
    "tfc_used_for_commission": "string",
    "tfc_used_for_liquidation_fee": "string",
    "total_commission_in_settling_asset": "string",
    "total_liquidation_fee_in_settling_asset": "string"
  }
}
```

A fill object

PROPERTIES

Name	Type	Required	Restrictions	Description
id	integer	false	none	none
size	integer	false	none	none
fill_type	string	false	none	none
side	string	false	none	none

Name	Type	Required	Restrictions	Description
price	string	false	none	Price at which the fill happened, BigDecimal sent as string
role	string	false	none	none
commission	string	false	none	Commission paid on this fill, negative value means commission was earned because of maker role
created_at	string	false	none	none
product_id	integer	false	none	none
product_symbol	string	false	none	none
order_id	string	false	none	Will be order_id(Integer) in most cases. Will be UUID string of order when fill_type is settlement
settling_asset_id	integer	false	none	none
settling_asset_symbol	string	false	none	none
meta_data	FillMetaData	false	none	Meta data inside fill

ENUMERATED VALUES

Property	Value
fill_type	normal
fill_type	adl
fill_type	liquidation
fill_type	settlement
fill_type	otc
side	buy
side	sell
role	taker
role	maker

ArrayOfFills

```
[
  {
    "id": 0,
    "size": 0,
    "fill_type": "normal",
```

```
"side": "buy",
"price": "string",
"role": "taker",
"commission": "string",
"created_at": "string",
"product_id": 0,
"product_symbol": "string",
"order_id": "string",
"settling_asset_id": 0,
"settling_asset_symbol": "string",
"meta_data": {
  "commission_deto": "string",
  "commission_deto_in_settling_asset": "string",
  "effective_commission_rate": "string",
  "liquidation_fee_deto": "string",
  "liquidation_fee_deto_in_settling_asset": "string",
  "order_price": "string",
  "order_size": "string",
  "order_type": "string",
  "order_unfilled_size": "string",
  "tfc_used_for_commission": "string",
  "tfc_used_for_liquidation_fee": "string",
  "total_commission_in_settling_asset": "string",
  "total_liquidation_fee_in_settling_asset": "string"
}
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	[Fill]	false	none	[A fill object]

FillMetaData

```
{
  "commission_deto": "string",
  "commission_deto_in_settling_asset": "string",
  "effective_commission_rate": "string",
  "liquidation_fee_deto": "string",
  "liquidation_fee_deto_in_settling_asset": "string",
  "order_price": "string",
  "order_size": "string",
  "order_type": "string",
  "order_unfilled_size": "string",
  "tfc_used_for_commission": "string",
  "tfc_used_for_liquidation_fee": "string",
  "total_commission_in_settling_asset": "string",
  "total_liquidation_fee_in_settling_asset": "string"
}
```

Meta data inside fill

PROPERTIES

Name	Type	Required	Restrictions	Description
commission_deto	string	false	none	none
commission_deto_in_settling_asset	string	false	none	none
effective_commission_rate	string	false	none	none
liquidation_fee_deto	string	false	none	none
liquidation_fee_deto_in_settling_asset	string	false	none	none
order_price	string	false	none	none
order_size	string	false	none	none
order_type	string	false	none	none
order_unfilled_size	string	false	none	none
tfc_used_for_commission	string	false	none	none
tfc_used_for_liquidation_fee	string	false	none	none
total_commission_in_settling_asset	string	false	none	none
total_liquidation_fee_in_settling_asset	string	false	none	none

OrderLeverage

```
{
  "leverage": 10,
  "order_margin": "563.2",
  "product_id": 27
}
```

Order Leverage for a product

PROPERTIES

Name	Type	Required	Restrictions	Description
leverage	string	false	none	Leverage of all open orders for this product
order_margin	string	false	none	Margin blocked in open orders for this product
product_id	integer	false	none	Product id of the ordered product

L2Orderbook

```
{
  "buy": [
    {
      "depth": "983",
      "price": "9187.5",
      "size": 205640
    }
  ],
  "last_updated_at": 1654589595784000,
  "sell": [
    {
      "depth": "1185",
      "price": "9188.0",
      "size": 113752
    }
  ],
  "symbol": "BTCUSD"
}
```

L2 orderbook

PROPERTIES

Name	Type	Required	Restrictions	Description
buy	[object]	false	none	none
» depth	string	false	none	sum of size till that price level
» price	string	false	none	none
» size	integer	false	none	for derivatives -> number of contracts, for spot -> amount in underlying
last_updated_at	integer	false	none	none
sell	[object]	false	none	none
» depth	string	false	none	sum of size till that price level
» price	string	false	none	none
» size	integer	false	none	for derivatives -> number of contracts, for spot -> amount in underlying
symbol	string	false	none	none

Trades

```
{
  "trades": [
    {
```

```
    "side": "buy",
    "size": 0,
    "price": "string",
    "timestamp": 0
  }
]
```

trades of a symbol

PROPERTIES

Name	Type	Required	Restrictions	Description
trades	[object]	false	none	none
» side	string	false	none	none
» size	integer	false	none	none
» price	string	false	none	none
» timestamp	integer	false	none	none

ENUMERATED VALUES

Property	Value
side	buy
side	sell

Wallet

```
{
  "asset_id": 0,
  "asset_symbol": "string",
  "available_balance": "string",
  "available_balance_for_robo": "string",
  "balance": "string",
  "blocked_margin": "string",
  "commission": "string",
  "cross_asset_liability": "string",
  "cross_commission": "string",
  "cross_locked_collateral": "string",
  "cross_order_margin": "string",
  "cross_position_margin": "string",
  "id": 0,
  "interest_credit": "string",
  "order_margin": "string",
  "pending_referral_bonus": "string",
  "pending_trading_fee_credit": "string",
  "portfolio_margin": "string",
  "position_margin": "string",
  "trading_fee_credit": "string",
  "unvested_amount": "string",

```

```
"user_id": 0
}
```

Wallet Data for each asset.

PROPERTIES

Name	Type	Required	Restrictions	Description
asset_id	integer	false	none	Id for assets like BTC
asset_symbol	string	false	none	Symbol for assets like BTC
available_balance	string	false	none	Balance available for trading
available_balance_for_robo	string	false	none	Balance available for robo trading
balance	string	false	none	Total wallet balance
blocked_margin	string	false	none	Total blocked margin including commissions for all modes
commission	string	false	none	Commissions blocked in Isolated Mode
cross_asset_liability	string	false	none	Asset liability in Cross margin mode
cross_commission	string	false	none	Commision blocked in Cross margin mode
cross_locked_collateral	string	false	none	collateral blocked in Cross margin mode
cross_order_margin	string	false	none	margin blocked for open orders in Cross margin mode
cross_position_margin	string	false	none	margin blocked for open positions in Cross margin mode
id	integer	false	none	Wallet Id
interest_credit	string	false	none	Total interest credited
order_margin	string	false	none	margin blocked for open positions in isolated mode
pending_referral_bonus	string	false	none	Pending referral bonus
pending_trading_fee_credit	string	false	none	Credit of trading fee pending
portfolio_margin	string	false	none	Total margin blocked including commissions in portfolio margin mode
position_margin	string	false	none	Margin blocked in open positions in isolated mode
trading_fee_credit	string	false	none	Credit of trading fee
unvested_amount	string	false	none	Amount currently unvested
user_id	integer	false	none	User Id linked to this wallet

WalletPayload

```
{
  "meta": {
    "net_equity": "string",
    "robo_trading_equity": "string"
  },
  "result": [
    {
      "asset_id": 0,
      "asset_symbol": "string",
      "available_balance": "string",
      "available_balance_for_robo": "string",
      "balance": "string",
      "blocked_margin": "string",
      "commission": "string",
      "cross_asset_liability": "string",
      "cross_commission": "string",
      "cross_locked_collateral": "string",
      "cross_order_margin": "string",
      "cross_position_margin": "string",
      "id": 0,
      "interest_credit": "string",
      "order_margin": "string",
      "pending_referral_bonus": "string",
      "pending_trading_fee_credit": "string",
      "portfolio_margin": "string",
      "position_margin": "string",
      "trading_fee_credit": "string",
      "unvested_amount": "string",
      "user_id": 0
    }
  ],
  "success": true
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
meta	WalletMetaData	false	none	Meta data for robo trading
result	ArrayOfWallets	false	none	Array of wallet for every asset
success	boolean	false	none	none

WalletMetaData

```
{
  "net_equity": "string",
  "robo_trading_equity": "string"
}
```

Meta data for robo trading

PROPERTIES

Name	Type	Required	Restrictions	Description
net_equity	string	false	none	Net equity for robo trading
robo_trading_equity	string	false	none	trading equity for robo trading

ArrayOfWallets

```
[
  {
    "asset_id": 0,
    "asset_symbol": "string",
    "available_balance": "string",
    "available_balance_for_robo": "string",
    "balance": "string",
    "blocked_margin": "string",
    "commission": "string",
    "cross_asset_liability": "string",
    "cross_commission": "string",
    "cross_locked_collateral": "string",
    "cross_order_margin": "string",
    "cross_position_margin": "string",
    "id": 0,
    "interest_credit": "string",
    "order_margin": "string",
    "pending_referral_bonus": "string",
    "pending_trading_fee_credit": "string",
    "portfolio_margin": "string",
    "position_margin": "string",
    "trading_fee_credit": "string",
    "unvested_amount": "string",
    "user_id": 0
  }
]
```

Array of wallet for every asset

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[Wallet]	false	none	Array of wallet for every asset

AssetTransferSubaccountReq

```
{
  "transferrer_user_id": "string",
  "transferee_user_id": "string",
```

```
"asset_symbol": "string",
"amount": null
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
transferrer_user_id	string	false	none	Debit account
transferee_user_id	string	false	none	Credit account
asset_symbol	string	false	none	Asset to transfer
amount	big_decimal	false	none	Amount to transfer. Only postive values allowed.

SubaccountTransferHistory

```
{
  "subaccount_user_id": "string",
  "before": "string",
  "after": "string",
  "page_size": 10
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
subaccount_user_id	string	false	none	subaccount user id
before	string	false	none	before cursor for pagination
after	string	false	none	after cursor for pagination
page_size	big_decimal	false	none	records per page

TransactionTypes

```
"string"
```

PROPERTIES

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	string	false	none	none
transaction_type	string	false	none	none

ENUMERATED VALUES

Property	Value
transaction_type	cashflow
transaction_type	deposit
transaction_type	withdrawal
transaction_type	commission
transaction_type	conversion
transaction_type	funding
transaction_type	settlement
transaction_type	liquidation_fee
transaction_type	spot_trade
transaction_type	withdrawal_cancellation
transaction_type	referral_bonus
transaction_type	sub_account_transfer
transaction_type	commission_rebate
transaction_type	promo_credit
transaction_type	trading_credits
transaction_type	trading_credits_forfeited
transaction_type	trading_credits_paid
transaction_type	trading_fee_credits_paid_liquidation_fee
transaction_type	trading_credits_reverted
transaction_type	interest_credit
transaction_type	external_deposit
transaction_type	credit_line
transaction_type	trading_competition

Property	Value
transaction_type	fund_deposit
transaction_type	fund_withdrawal
transaction_type	fund_wallet_deposit
transaction_type	fund_wallet_withdrawal
transaction_type	fund_reward
transaction_type	trade_farming_reward
transaction_type	interest_credit
transaction_type	revert
transaction_type	raf_bonus
transaction_type	fill_appropriation
transaction_type	incident_compensation

Transaction

```
{
  "id": 0,
  "amount": "string",
  "balance": "string",
  "transaction_type": "string",
  "meta_data": {},
  "product_id": 0,
  "asset_id": 0,
  "asset_symbol": 0,
  "created_at": "string"
}
```

PROPERTIES

Name	Type	Required	Restrictions		Description
id	integer	false	none	none	
amount	string	false	none		amount credited/debited in this transaction (+ for credited, - for debited)
balance	string	false	none		net wallet balance after this transaction
transaction_type	TransactionTypes	false	none	none	
meta_data	object	false	none	none	

Name	Type	Required	Restrictions		Description
product_id	integer	false	none	none	
asset_id	integer	false	none	none	
asset_symbol	integer	false	none	none	
created_at	string	false	none	none	

ArrayOfTransactions

```
[
  {
    "id": 0,
    "amount": "string",
    "balance": "string",
    "transaction_type": "string",
    "meta_data": {},
    "product_id": 0,
    "asset_id": 0,
    "asset_symbol": 0,
    "created_at": "string"
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	[Transaction]	false	none	none

SubaccountTransferLog

```
{
  "transferrer_user_id": "string",
  "transferee_user_id": "string",
  "asset_symbol": "string",
  "amount": null,
  "created_at": "string",
  "transferee_user": {},
  "transferrer_user": {}
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
transferrer_user_id	string	false	none	User id of the account debited with the asset.
transferee_user_id	string	false	none	User id of the account credited with the asset.
asset_symbol	string	false	none	Asset symbol transferred.
amount	big_decimal	false	none	Amount transferred.
created_at	string	false	none	transfer creation date and time
transferee_user	object	false	none	User details
transferrer_user	object	false	none	User details

ArrayOfSubaccountTransferLog

```
[
  {
    "transferrer_user_id": "string",
    "transferee_user_id": "string",
    "asset_symbol": "string",
    "amount": null,
    "created_at": "string",
    "transferee_user": {},
    "transferrer_user": {}
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[SubaccountTransferLog]	false	none	none

greeks

```
{
  "delta": "0.25",
  "gamma": "0.10",
  "rho": "0.05",
  "theta": "-0.02",
  "vega": "0.15"
}
```

The Greeks represent different factors that influence the pricing of options. These are key measures for assessing risk and managing option positions.

PROPERTIES

Name	Type	Required	Restrictions	Description
delta	string	false	none	The rate of change of the option price with respect to changes in the underlying asset price. A measure of sensitivity to the asset price movement.
gamma	string	false	none	The rate of change of delta with respect to changes in the underlying asset price. A measure of the curvature of the option's price sensitivity to the asset price.
rho	string	false	none	The rate of change of the option price with respect to changes in the risk-free interest rate. A measure of interest rate sensitivity.
theta	string	false	none	The rate of change of the option price with respect to time, often referred to as time decay. A measure of how the option's price declines as expiration approaches.
vega	string	false	none	The rate of change of the option price with respect to changes in the volatility of the underlying asset. A measure of volatility sensitivity.

price_band

```
{
  "lower_limit": "61120.45",
  "upper_limit": "72300.00"
}
```

The price band defines the permissible price range for a product. The lower and upper limits represent the boundaries within which the product's price can fluctuate.

PROPERTIES

Name	Type	Required	Restrictions	Description
lower_limit	string	false	none	The minimum price limit for the product. It defines the lowest allowable price before triggering a price band constraint.
upper_limit	string	false	none	The maximum price limit for the product. It defines the highest allowable price before triggering a price band constraint.

quotes

```
{
  "ask_iv": "0.25",
  "ask_size": "100",
  "best_ask": "150.00",
  "best_bid": "148.00",
  "bid_iv": "0.22",
  "bid_size": "50"
```

```
}

```

The 'quotes' object contains the latest bid and ask prices, their respective implied volatilities (IV), and order sizes for an asset. It provides key market data for understanding liquidity and pricing.

PROPERTIES

Name	Type	Required	Restrictions	Description
ask_iv	string	false	none	The implied volatility (IV) for the ask price. Represents the market's expectation of the future volatility of the underlying asset.
ask_size	string	false	none	The size of the ask order, representing the quantity of the asset available for sale at the ask price.
best_ask	string	false	none	The best (lowest) ask price available in the market for the asset.
best_bid	string	false	none	The best (highest) bid price available in the market for the asset.
bid_iv	string	false	none	The implied volatility (IV) for the bid price. Represents the market's expectation of future volatility for the bid side of the order book.
bid_size	string	false	none	The size of the bid order, representing the quantity of the asset that buyers are willing to purchase at the bid price.

Ticker

```
{
  "close": 67321,
  "contract_type": "futures",
  "greeks": {
    "delta": "0.25",
    "gamma": "0.10",
    "rho": "0.05",
    "theta": "-0.02",
    "vega": "0.15"
  },
  "high": 68500.5,
  "low": 66300.25,
  "mark_price": "67000.00",
  "mark_vol": "500",
  "oi": "15000",
  "oi_value": "1000000",
  "oi_value_symbol": "USD",
  "oi_value_usd": "1050000",
  "open": 67000,
  "price_band": {
    "lower_limit": "61120.45",
    "upper_limit": "72300.00"
  },
  "product_id": 123456,
  "quotes": {
    "ask_iv": "0.25",
    "ask_size": "100",
    "best_ask": "150.00",
    "best_bid": "148.00",
    "bid_iv": "0.22",

```

```

    "bid_size": "50"
  },
  "size": 100,
  "spot_price": "67000.00",
  "strike_price": "68000.00",
  "symbol": "BTCUSD",
  "timestamp": 1609459200,
  "turnover": 5000000,
  "turnover_symbol": "USD",
  "turnover_usd": 5200000,
  "volume": 25000
}

```

The 'Ticker' object provides real-time trading data for a specific product, including prices, volumes, open interest, and Greek values (for options). This data is essential for analyzing market trends and asset performance.

PROPERTIES

Name	Type	Required	Restrictions	Description
close	integer	false	none	The closing price of the last trade for the product.
contract_type	string	false	none	Comma-separated list of contract types, such as futures, perpetual_futures, call_options, put_options.
greeks	greeks	false	none	The Greeks represent different factors that influence the pricing of options. These are key measures for assessing risk and managing option positions.
high	number	false	none	The highest price reached during the trading session.
low	number	false	none	The lowest price reached during the trading session.
mark_price	string	false	none	The market price of the product, reflecting the most recent transaction.
mark_vol	string	false	none	The market volume at the most recent trade price.
oi	string	false	none	The open interest, or the number of outstanding contracts, for the product.
oi_value	string	false	none	The value of the open interest in the base currency.
oi_value_symbol	string	false	none	The symbol representing the currency of the open interest value.
oi_value_usd	string	false	none	The open interest value converted to USD.
open	number	false	none	The opening price at the start of the trading session.
price_band	price_band	false	none	The price band defines the permissible price range for a product. The lower and upper limits represent the boundaries within which the product's price can fluctuate.
product_id	number	false	none	A unique identifier for the product.
quotes	quotes	false	none	The 'quotes' object contains the latest bid and ask prices, their respective implied volatilities (IV), and order sizes for an asset. It provides key market data for understanding liquidity and pricing.

Name	Type	Required	Restrictions	Description
size	number	false	none	The size of the most recent order executed in the market.
spot_price	string	false	none	The current spot price of the underlying asset.
strike_price	string	false	none	The strike price for options contracts associated with the product.
symbol	string	false	none	The ticker symbol for the product.
timestamp	number	false	none	The timestamp of the last trade or update to the ticker.
turnover	number	false	none	The total turnover (value traded) for the product during the trading session.
turnover_symbol	string	false	none	The symbol representing the currency in which the turnover is measured.
turnover_usd	number	false	none	The turnover value converted to USD.
volume	integer	false	none	The total trading volume for the product during the trading session.

ArrayOfTickers

```
[
  {
    "close": 67321,
    "contract_type": "futures",
    "greeks": {
      "delta": "0.25",
      "gamma": "0.10",
      "rho": "0.05",
      "theta": "-0.02",
      "vega": "0.15"
    },
    "high": 68500.5,
    "low": 66300.25,
    "mark_price": "67000.00",
    "mark_vol": "500",
    "oi": "15000",
    "oi_value": "1000000",
    "oi_value_symbol": "USD",
    "oi_value_usd": "1050000",
    "open": 67000,
    "price_band": {
      "lower_limit": "61120.45",
      "upper_limit": "72300.00"
    },
    "product_id": 123456,
    "quotes": {
      "ask_iv": "0.25",
      "ask_size": "100",
      "best_ask": "150.00",
      "best_bid": "148.00",
      "bid_iv": "0.22",
      "bid_size": "50"
    },
    "size": 100,
    "spot_price": "67000.00",
```

```
    "strike_price": "68000.00",
    "symbol": "BTCUSD",
    "timestamp": 1609459200,
    "turnover": 5000000,
    "turnover_symbol": "USD",
    "turnover_usd": 5200000,
    "volume": 25000
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[Ticker]	false	none	[The 'Ticker' object provides real-time trading data for a specific product, including prices, volumes, open interest, and Greek values (for options). This data is essential for analyzing market trends and asset performance.]

PaginationMeta

```
{
  "after": "g3QAAACZAAKY3J1YXR1ZF9hdHQAAAAA",
  "before": "a2PQRSACZAAKY3J1YXR1ZF3fnqHBBBNZL"
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
after	string	false	none	after cursor for pagination; becomes null if page after the current one does not exist
before	string	false	none	before cursor for pagination; becomes null if page before the current one does not exist

OHLCData

```
{
  "time": 0,
  "open": 0,
  "high": 0,
  "low": 0,
  "close": 0,
  "volume": 0
}
```

A ohlc object

PROPERTIES

Name	Type	Required	Restrictions	Description
time	integer	false	none	none
open	number	false	none	none
high	number	false	none	none
low	number	false	none	none
close	number	false	none	none
volume	number	false	none	none

ArrayOfOHLCData

```
[
  {
    "time": 0,
    "open": 0,
    "high": 0,
    "low": 0,
    "close": 0,
    "volume": 0
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	[OHLCData]	false	none	[A ohlc object]

SparklineData

```
{
  "ETHUSD": [
    [
      1594214051,
      0.00003826
    ],
    [
      1594214051,
      0.00003826
    ]
  ],
  "MARK:BTCUSD": [
    [
      1594215270,
      0.00003826
    ]
  ]
}
```

```
]
]
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
additionalProperties	[integer]	false	none	array of timestamp and closing value

Stats

```
{
  "last_30_days_volume": 0,
  "last_7_days_volume": 0,
  "total_volume": 0
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
last_30_days_volume	integer	false	none	sum of turnover usd in the last 30 days
last_7_days_volume	integer	false	none	sum of turnover usd in the last 7 days
total_volume	integer	false	none	sum of turnover usd in the last 24 hours

MMPConfigUpdateRequest

```
{
  "asset": "string",
  "window_interval": 0,
  "freeze_interval": 0,
  "trade_limit": "string",
  "delta_limit": "string",
  "vega_limit": "string",
  "mmp": "mmp1"
}
```

MMP config for an underlying

PROPERTIES

Name	Type	Required	Restrictions	Description
asset	string	false	none	none

Name	Type	Required	Restrictions	Description
window_interval	integer	false	none	Window interval in seconds
freeze_interval	integer	false	none	MMP freeze interval in seconds. Setting this to zero will require a manual reset once mmp is triggered.
trade_limit	string	false	none	Notional trade limit for mmp to trigger (in USD)
delta_limit	string	false	none	Delta Adjusted notional trade limit for mmp to trigger (in USD)
vega_limit	string	false	none	vega traded limit for mmp to trigger (in USD)
mmp	string	false	none	Specify mmp flag for the config update

ENUMERATED VALUES

Property	Value
mmp	mmp1
mmp	mmp2
mmp	mmp3
mmp	mmp4
mmp	mmp5

MMPResetRequest

```
{
  "asset": "string",
  "mmp": "mmp1"
}
```

MMP config for an underlying

PROPERTIES

Name	Type	Required	Restrictions	Description
asset	string	false	none	none
mmp	string	false	none	specify mmp flag to reset

ENUMERATED VALUES

Property	Value
mmp	mmp1

Property	Value
mmp	mmp2
mmp	mmp3
mmp	mmp4
mmp	mmp5

ChangeMarginModeRequest

```
{
  "margin_mode": "isolated",
  "subaccount_user_id": "5112346"
}
```

Request to change the margin mode for a main or subaccount.

PROPERTIES

Name	Type	Required	Restrictions	Description
margin_mode	string	false	none	The target margin mode: 'isolated' or 'portfolio'.
subaccount_user_id	string	false	none	The user ID of the account. Provide the main user ID for the main account or the respective subaccount user ID.

ENUMERATED VALUES

Property	Value
margin_mode	isolated
margin_mode	portfolio

UserPreference

```
{
  "user_id": 57354187,
  "default_auto_topup": true,
  "mmp_config": null,
  "deto_for_commission": false,
  "vip_level": 0,
  "vip_discount_factor": "0.00",
  "volume_30d": "1060.675333",
  "email_preferences": {
    "adl": true,
    "liquidation": true,
  }
}
```

```

    "margin_topup": false,
    "marketing": true,
    "order_cancel": true,
    "order_fill": true,
    "stop_order_trigger": true
  },
  "notification_preferences": {
    "adl": true,
    "liquidation": true,
    "margin_topup": false,
    "marketing": true,
    "order_cancel": false,
    "order_fill": true,
    "price_alert": true,
    "stop_order_trigger": true
  },
  "price_alert_assets": [
    "BTC",
    "ETH"
  ],
  "enabled_portfolios": {},
  "interest_credit": false
}

```

User trading preferences

PROPERTIES

Name	Type	Required	Restrictions	Description
user_id	integer	false	none	Unique identifier for the user
default_auto_topup	boolean	false	none	Default auto top-up setting for newly acquired positions (only for isolated mode)
mmp_config	object null	false	none	Config object for market maker protection (only for MMP-enabled accounts)
deto_for_commission	boolean	false	none	Flag to determine whether to pay commissions in DETO
vip_level	integer	false	none	VIP level for this account. Customers get better fee discounting for higher VIP levels
vip_discount_factor	string	false	none	Discount factor based on the VIP level
volume_30d	string	false	none	30-day trading volume for the user
email_preferences	object	false	none	Email preferences for different events
» adl	boolean	false	none	none
» liquidation	boolean	false	none	none
» margin_topup	boolean	false	none	none
» marketing	boolean	false	none	none
» order_cancel	boolean	false	none	none
» order_fill	boolean	false	none	none

Name	Type	Required	Restrictions	Description
» stop_order_trigger	boolean	false	none	none
notification_preferences	object	false	none	Notification preferences for different events
» adl	boolean	false	none	none
» liquidation	boolean	false	none	none
» margin_topup	boolean	false	none	none
» marketing	boolean	false	none	none
» order_cancel	boolean	false	none	none
» order_fill	boolean	false	none	none
» price_alert	boolean	false	none	none
» stop_order_trigger	boolean	false	none	none
price_alert_assets	[string]	false	none	Assets for which price alerts are set
enabled_portfolios	object	false	none	Enabled portfolios for the user
interest_credit	boolean	false	none	Whether the user is receiving interest credits

EditUserPreference

```
{
  "default_auto_topup": true,
  "showMarketOrdersForOptionsInBracket": true,
  "interest_credit": false,
  "email_preferences": {
    "adl": true,
    "liquidation": true,
    "order_fill": true,
    "stop_order_trigger": true,
    "order_cancel": true,
    "marketing": true
  },
  "notification_preferences": {
    "adl": false,
    "liquidation": true,
    "order_fill": true,
    "stop_order_trigger": true,
    "price_alert": true,
    "marketing": true
  }
}
```

Edit User Preference Object

PROPERTIES

Name	Type	Required	Restrictions	Description
default_auto_topup	boolean	false	none	Default auto top-up setting for newly acquired positions
showMarketOrdersForOptionsInBracket	boolean	false	none	Flag to display market orders for options in bracket orders
interest_credit	boolean	false	none	Whether the user is receiving interest credits
email_preferences	object	false	none	Email preferences for different events
» adl	boolean	false	none	none
» liquidation	boolean	false	none	none
» order_fill	boolean	false	none	none
» stop_order_trigger	boolean	false	none	none
» order_cancel	boolean	false	none	none
» marketing	boolean	false	none	none
notification_preferences	object	false	none	Notification preferences for different events
» adl	boolean	false	none	none
» liquidation	boolean	false	none	none
» order_fill	boolean	false	none	none
» stop_order_trigger	boolean	false	none	none
» price_alert	boolean	false	none	none
» marketing	boolean	false	none	none

User

```
{
  "id": "98765432",
  "email": "rajtrader2342@gmail.com",
  "account_name": "Main",
  "first_name": "Rajesh",
  "last_name": "Sharma",
  "dob": "1985-08-25",
  "country": "India",
  "phone_number": "9876543210",
  "margin_mode": "isolated",
  "pf_index_symbol": ".DEXTUSD",
  "is_sub_account": false,
  "is_kyc_done": true
}
```

```
}
```

Represents a user account with personal and account-related details.

PROPERTIES

Name	Type	Required	Restrictions	Description
id	string	false	none	Unique user identifier, which can be an integer or string.
email	string	false	none	User's email address.
account_name	string	false	none	The main account or subaccount name.
first_name	string	false	none	User's first name.
last_name	string	false	none	User's last name.
dob	string(date)	false	none	Date of birth in YYYY-MM-DD format.
country	string	false	none	User's country of residence.
phone_number	string	false	none	User's phone number with country code.
margin_mode	string	false	none	The user's margin mode, which can be 'isolated' or 'portfolio'.
pf_index_symbol	string	false	none	Portfolio index symbol if the account is in portfolio margin mode.
is_sub_account	boolean	false	none	Indicates if the user account is a sub-account.
is_kyc_done	boolean	false	none	Indicates if the user's KYC verification is completed.

ChangeMarginModeResponse

```
{
  "id": "5112346",
  "margin_mode": "isolated"
}
```

Response returned after changing the user's margin mode

PROPERTIES

Name	Type	Required	Restrictions	Description
id	string	false	none	Unique identifier of the user(user_id) whose margin mode was updated
margin_mode	string	false	none	The updated margin mode. Possible values: isolated, portfolio, or cross

CreateHeartbeat

```
{
  "heartbeat_id": "string",
  "impact": "low",
  "contract_types": [
    "string"
  ],
  "underlying_assets": [
    "string"
  ],
  "product_symbols": [
    "string"
  ],
  "config": [
    {
      "action": "cancel_orders",
      "unhealthy_count": 0,
      "tag": "string"
    }
  ]
}
```

Create Heartbeat Request Object

PROPERTIES

Name	Type	Required	Restrictions	Description
heartbeat_id	string	true	none	Unique identifier for the heartbeat
impact	string	true	none	Impact level
contract_types	[string]	false	none	Array of contract types to monitor
underlying_assets	[string]	false	none	Array of underlying assets to monitor
product_symbols	[string]	false	none	Array of specific product symbols to monitor
config	[HeartbeatConfig]	true	none	Array of action configurations

ENUMERATED VALUES

Property	Value
impact	low
impact	medium
impact	high

HeartbeatConfig

```
{
  "action": "cancel_orders",
  "unhealthy_count": 0,
  "tag": "string"
}
```

Heartbeat Configuration Object

PROPERTIES

Name	Type	Required	Restrictions	Description
action	string	true	none	Action to take when heartbeat expires
unhealthy_count	integer	true	none	Number of unhealthy heartbeats before action
tag	string	false	none	Tag for the action (e.g., 'mmp')

ENUMERATED VALUES

Property	Value
action	cancel_orders
action	spreads

HeartbeatResponse

```
{
  "heartbeat_id": "string",
  "status": "string"
}
```

Heartbeat Response Object

PROPERTIES

Name	Type	Required	Restrictions	Description
heartbeat_id	string	false	none	none
status	string	false	none	none

HeartbeatAck

```
{
  "heartbeat_id": "string",
```

```
"ttl": 0
}
```

Heartbeat Acknowledgment Request Object

PROPERTIES

Name	Type	Required	Restrictions	Description
heartbeat_id	string	true	none	Heartbeat identifier
ttl	integer	true	none	Time to live in milliseconds

HeartbeatAckResponse

```
{
  "process_enabled": "string",
  "heartbeat_timestamp": "string"
}
```

Heartbeat Acknowledgment Response Object

PROPERTIES

Name	Type	Required	Restrictions	Description
process_enabled	string	false	none	Acknowledgement status (true/false)
heartbeat_timestamp	string	false	none	Expiry timestamp after which actions will be triggered

ArrayOfHeartbeats

```
[
  {
    "heartbeat_id": "string",
    "impact": "string",
    "contract_types": [
      "string"
    ],
    "underlying_assets": [
      "string"
    ],
    "product_symbols": [
      "string"
    ],
    "config": [
      {
        "action": "cancel_orders",
        "unhealthy_count": 0,
        "tag": "string"
      }
    ]
  }
]
```

```
    }
  ],
  "status": "string",
  "last_ack": "string",
  "next_ack_required_by": "string"
}
```

PROPERTIES

Name	Type	Required	Restrictions	Description
<i>anonymous</i>	[Heartbeat]	false	none	[Heartbeat Object]

Heartbeat

```
{
  "heartbeat_id": "string",
  "impact": "string",
  "contract_types": [
    "string"
  ],
  "underlying_assets": [
    "string"
  ],
  "product_symbols": [
    "string"
  ],
  "config": [
    {
      "action": "cancel_orders",
      "unhealthy_count": 0,
      "tag": "string"
    }
  ],
  "status": "string",
  "last_ack": "string",
  "next_ack_required_by": "string"
}
```

Heartbeat Object

PROPERTIES

Name	Type	Required	Restrictions	Description
heartbeat_id	string	false	none	none
impact	string	false	none	none
contract_types	[string]	false	none	none
underlying_assets	[string]	false	none	none
product_symbols	[string]	false	none	none

Name	Type	Required	Restrictions	Description
config	[HeartbeatConfig]	false	none	[Heartbeat Configuration Object]
status	string	false	none	none
last_ack	string	false	none	none
next_ack_required_by	string	false	none	none

ArrayOfSubaccounts

```
[
  {
    "id": "98765432",
    "email": "rajtrader2342@gmail.com",
    "account_name": "Main",
    "first_name": "Rajesh",
    "last_name": "Sharma",
    "dob": "1985-08-25",
    "country": "India",
    "phone_number": "9876543210",
    "margin_mode": "isolated",
    "pf_index_symbol": ".DEXBTUSD",
    "is_sub_account": false,
    "is_kyc_done": true
  }
]
```

PROPERTIES

Name	Type	Required	Restrictions	Description
anonymous	[User]	false	none	[Represents a user account with personal and account-related details.]

Deadman Switch

The Deadman Switch is a safety mechanism that automatically cancels orders or takes other protective actions when a client fails to send heartbeat signals within specified time intervals. This feature is essential for risk management and preventing unwanted positions from accumulating due to client disconnections or failures.

Overview

The Deadman Switch system consists of several components:

- **Heartbeat Creation:** Clients register a heartbeat with specific configuration
- **Heartbeat Acknowledgment:** Clients periodically send acknowledgments to keep the heartbeat alive
- **Automatic Actions:** When heartbeats expire, the system automatically executes configured actions

Authentication

All Deadman Switch endpoints require authentication. Include your API key and signature in the request headers as described in the Authentication section.

Heartbeat Management

CREATE HEARTBEAT

Creates a new heartbeat with specific configuration for automatic actions.

Endpoint: POST /heartbeat/create

Request Body:

```
{
  "heartbeat_id": "my_trading_bot_001",
  "impact": "contracts",
  "contract_types": ["perpetual_futures", "call_options"],
  "underlying_assets": ["BTC", "ETH"],
  "product_symbols": ["BTCUSD", "ETHUSD"],
  "config": [
    {
      "action": "cancel_orders",
      "unhealthy_count": 1
    },
    {
      "action": "spreads",
      "unhealthy_count": 3,
      "value": 100
    }
  ]
}
```

Parameters:

Parameter	Type	Required	Description
heartbeat_id	string	Yes	Unique identifier for the heartbeat
impact	string	Yes	Impact : contracts, products
contract_types	array	Yes	Array of contract types to monitor, required if impact is contracts
underlying_assets	array	No	Array of underlying assets to monitor
product_symbols	array	Yes	Array of specific product symbols to monitor, required if impact is products

Parameter	Type	Required	Description
config	array	Yes	Array of action configurations

Config Actions:

- cancel_orders: Cancels all open orders
- spreads: Adds spreads to orders

Response:

```
{
  "success": true,
  "result": {
    "heartbeat_id": "my_trading_bot_001"
  }
}
```

HEARTBEAT ACKNOWLEDGMENT

Sends an acknowledgment to keep the heartbeat active. Set ttl to 0 to disable heartbeat.

Endpoint: POST /heartbeat

Request Body:

```
{
  "heartbeat_id": "my_trading_bot_001",
  "ttl": 30000
}
```

Parameters:

Parameter	Type	Required	Description
heartbeat_id	string	Yes	Heartbeat identifier
ttl	integer/string	Yes	Time to live in milliseconds

Response:

```
{
  "success": true,
  "result": {
    "heartbeat_timestamp": "1243453435", //next expiry timestamp
    "process_enabled": "true" // true/false
  }
}
```

GET HEARTBEATS

Retrieves all active heartbeats for a user.

Endpoint: GET /heartbeat

Query Parameters:

Parameter	Type	Required	Description
heartbeat_id	string	No	Specific heartbeat ID to retrieve

Response:

```
{
  "success": true,
  "result": [
    {
      "user_id": "user_id",
      "heartbeat_id": "my_trading_bot_001",
      "impact": "contracts",
      "contract_types": ["perpetual_futures", "call_options"],
      "underlying_assets": ["BTC", "ETH"],
      "product_symbols": ["BTCUSD", "ETHUSD"],
      "config": [
        {
          "action": "cancel_orders",
          "unhealthy_count": 1
        }
      ]
    }
  ]
}
```

Implementation Guidelines

BEST PRACTICES

- 1. **Regular Heartbeats:** Send heartbeat acknowledgments at regular intervals (recommended: every 30 seconds)
- 2. **Error Handling:** Implement proper error handling for heartbeat failures
- 3. **Monitoring:** Monitor heartbeat status and implement alerts for failures
- 4. **Graceful Shutdown:** Properly disable heartbeats when shutting down trading systems

PYTHON EXAMPLE

```
import requests
import time
import json

class DeadmanSwitch:
    def __init__(self, api_key, api_secret, base_url):
        self.api_key = api_key
        self.api_secret = api_secret
        self.base_url = base_url
        self.heartbeat_id = "trading_bot_" + str(int(time.time()))

    def create_heartbeat(self):
        """Create a new heartbeat"""
        url = f"{self.base_url}/heartbeat/create"
        headers = self._get_auth_headers()

        payload = {
            "heartbeat_id": self.heartbeat_id,
            "impact": "contracts",
            "contract_types": ["perpetual_futures"],
            "config": [
                {
```



```

        "action": "cancel_orders",
        "unhealthy_count": 1 # if heartbeat missed 1 time than cancel all orders
    }
]
}

response = requests.post(url, json=payload, headers=headers)
return response.json()

def send_heartbeat(self):
    """Send heartbeat acknowledgment"""
    url = f"{self.base_url}/heartbeat"
    headers = self._get_auth_headers()

    payload = {
        "heartbeat_id": self.heartbeat_id,
        "ttl": 30000 # 30 seconds
    }

    response = requests.post(url, json=payload, headers=headers)
    return response.json()

def start_heartbeat_loop(self):
    """Start continuous heartbeat loop"""
    while True:
        try:
            result = self.send_heartbeat()
            print(f"Heartbeat sent: {result}")
            time.sleep(25) # Send every 25 seconds (TTL is 30)
        except Exception as e:
            print(f"Heartbeat failed: {e}")
            time.sleep(5)

def _get_auth_headers(self):
    # Implement authentication headers
    return {
        "Content-Type": "application/json",
        "api-key": self.api_key,
        # Add signature generation logic
    }

# Usage example
deadman = DeadmanSwitch("your_api_key", "your_api_secret", "https://api.delta.exchange")
deadman.create_heartbeat()
deadman.start_heartbeat_loop()

```

NODE.JS EXAMPLE

```

const axios = require('axios');
const crypto = require('crypto');

class DeadmanSwitch {
    constructor(apiKey, apiSecret, baseUrl) {
        this.apiKey = apiKey;
        this.apiSecret = apiSecret;
        this.baseUrl = baseUrl;
        this.heartbeatId = `trading_bot_${Date.now()}`;
    }

    async createHeartbeat() {
        const url = `${this.baseUrl}/heartbeat/create`;
        const headers = this.getAuthHeaders();

        const payload = {
            heartbeat_id: this.heartbeatId,

```

```
        impact: "contracts",
        contract_types: ["perpetual_futures"],
        config: [
            {
                action: "cancel_orders",
                unhealthy_count: 1
            }
        ]
    };

    const response = await axios.post(url, payload, { headers });
    return response.data;
}

async sendHeartbeat() {
    const url = `${this.baseUrl}/heartbeat`;
    const headers = this.getAuthHeaders();

    const payload = {
        heartbeat_id: this.heartbeatId,
        ttl: 30000
    };

    const response = await axios.post(url, payload, { headers });
    return response.data;
}

startHeartbeatLoop() {
    setInterval(async () => {
        try {
            const result = await this.sendHeartbeat();
            console.log('Heartbeat sent:', result);
        } catch (error) {
            console.error('Heartbeat failed:', error);
        }
    }, 25000); // Send every 25 seconds
}

getAuthHeaders() {
    // Implement authentication headers
    return {
        'Content-Type': 'application/json',
        'api-key': this.apiKey,
        // Add signature generation logic
    };
}

// Usage example
const deadman = new DeadmanSwitch('your_api_key', 'your_api_secret', 'https://api.delta.exchange');
deadman.createHeartbeat();
deadman.startHeartbeatLoop();
```

Error Codes

Error Code	Description
unauthorized	Authentication failed

Error Code	Description
rate_limit_exceeded	Too many requests

Security Considerations

1. **API Key Security:** Keep your API keys secure and never expose them in client-side code
2. **Network Security:** Use HTTPS for all API communications
3. **Monitoring:** Implement proper monitoring and alerting for heartbeat failures
4. **Backup Systems:** Consider implementing backup heartbeat mechanisms for critical trading systems

Place order errors

This section lists various errors returned by the system while placing order. The error format looks like this

```
{
  "success": false,
  "error": {
    "code": "...",      // error code
    "context": {
      "..."
    }
  }
}
```

Here is a list of error codes and their explanation

error code	description
insufficient_margin	Margin required to place order with selected leverage and quantity is insufficient.
order_size_exceed_available	The order book doesn't have sufficient liquidity, hence the order couldnt be filled (for ex - ioc orders).
risk_limits_breached	orders couldn't be placed as it will breach allowed risk limits.
invalid_contract	The contract/product is either doesn't exist or has already expired.
immediate_liquidation	Order will cause immediate liquidation.
out_of_bankruptcy	Order prices are out of position bankruptcy limits.
self_matching_disrupted_post_only	Self matching is not allowed during auction.
immediate_execution_post_only	orders couldn't be placed as it includes post only orders which will be immediately executed.

Errors

Delta API uses the following error codes:

Error Code	Meaning
400	Bad Request -- Your request is invalid.
401	Unauthorized -- Your API key/Signature is wrong.
403	Forbidden Error -- Request blocked by CDN (e.g., missing User-Agent header or hidden/blocked IP from certain hosted environments).
404	Not Found -- The specified resource could not be found.
405	Method Not Allowed -- You tried to access a resource with an invalid method.
406	Not Acceptable -- You requested a format that isn't json.
429	Too Many Requests -- You have exhausted your rate limits! Slow down!
500	Internal Server Error -- We had a problem with our server. Try again later.
503	Service Unavailable -- We're temporarily offline for maintenance. Please try again later.

Rest Clients

Delta API conforms to the Swagger spec for REST endpoints. Any Swagger-compatible client can connect to the Delta API and execute commands.

You can find the swagger spec json for Delta Api [here](#)

We also have Rest Api Clients available for the following languages

- Nodejs
- Python

CCXT

CCXT is our authorized SDK provider and you may access our API through CCXT.

For more information, please visit [ccxt website](#).

Websocket Feed

Websocket api can be used for the following use cases

- Get real time feed of market data, this includes L2 orderbook and recent trades.
- Get price feeds - Mark prices of different contracts, price feed of underlying indexes etc.
- Get account specific notifications like fills, liquidations, ADL and PnL updates.
- Get account specific updates on orders, positions and wallets.

Websocket url for Delta Exchange

- **Production** - wss://socket.india.delta.exchange
- **Testnet(Demo Account)** - wss://socket-ind.testnet.deltaex.org

There is a limit of 150 connections every 5 minutes per IP address. A connection attempt that goes beyond the limit will be disconnected with 429 HTTP status error. On receiving this error, wait for 5 to 10 minutes before making new connection requests.

You will be disconnected, if there is no activity within **60 seconds** after making connection.

Subscribing to Channels

Subscribe

To begin receiving feed messages, you must first send a subscribe message to the server indicating which channels and contracts to subscribe for.

To specify contracts within each channel, just pass a list of symbols inside the channel payload. Mention ***["all"]*** in symbols if you want to receive updates across all the contracts. Please note that snapshots are sent only for specified symbols, meaning no snapshots are sent for symbol: ***"all"***.

Once a subscribe message is received the server will respond with a subscriptions message that lists all channels you are subscribed to. Subsequent subscribe messages will add to the list of subscriptions.

Subscription Sample

```
// Request: Subscribe to BTCUSD and ETHUSD with the ticker and orderbook(L2) channels.
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "v2/ticker",
        "symbols": [
          "BTCUSD",
          "ETHUSD"
        ]
      },
      {
        "name": "l2_orderbook",
        "symbols": [
          "BTCUSD"
        ]
      }
    ]
  }
}
```

```

        {
            "name": "funding_rate",
            "symbols": [
                "all"
            ]
        }
    ]
}

// Response: Success
{
    "type": "subscriptions",
    "channels": [
        {
            "name": "l2_orderbook",
            "symbols": [
                "BTCUSD"
            ],
        },
        {
            "name": "v2/ticker",
            "symbols": [
                "BTCUSD",
                "ETHUSD"
            ],
        },
        {
            "name": "funding_rate",
            "symbols": [
                "all"
            ]
        }
    ]
}

// Response: Error
{
    "type": "subscriptions",
    "channels": [
        {
            "name": "l2_orderbook",
            "symbols": [
                "BTCUSD"
            ],
        },
        {
            "name": "trading_notifications",
            "error": "subscription forbidden on trading_notifications. Unauthorized user"
        }
    ]
}

```

Unsubscribe

If you want to unsubscribe from channel/contracts pairs, send an "unsubscribe" message. The structure is equivalent to subscribe messages. If you want to unsubscribe for specific symbols in a channel, you can pass it in the symbol list. As a shorthand you can also provide no symbols for a channel, which will unsubscribe you from the channel entirely.

Unsubscribe Sample

```
// Request: Unsubscribe from BTCUSD and ETHUSD with the ticker and orderbook(L2) channels.
{
  "type": "unsubscribe",
  "payload": {
    "channels": [
      {
        "name": "v2/ticker",           // unsubscribe from ticker channel only for BTCUSD
        "symbols": [
          "BTCUSD"
        ]
      },
      {
        "name": "l2_orderbook"       // unsubscribe from all symbols for l2_orderbook channel
      }
    ]
  }
}
```

Authenticating a connection

Authentication allows clients to receive private messages, like trading notifications. Examples of the trading notifications are: fills, liquidations, adl and pnl updates.

To authenticate, you need to send a signed request of type **'auth'** on your socket connection. Check the authentication section above for more details on how to sign a request using api key and secret.

The payload for the signed request will be **'GET' + *timestamp* + '/live'**

To subscribe to private channels, the client needs to first send an auth event, providing api-key, and signature.

Authentication sample

```
# auth message with signed request
import websocket
import hashlib
import hmac
import time

api_key = 'a207900b7693435a8fa9230a38195d'
api_secret = '7b6f39dcf660ec1c7c664f612c60410a2bd0c258416b498bf0311f94228f'

def generate_signature(secret, message):
    message = bytes(message, 'utf-8')
    secret = bytes(secret, 'utf-8')
    hash = hmac.new(secret, message, hashlib.sha256)
    return hash.hexdigest()

# Get open orders
method = 'GET'
timestamp = str(int(time.time()))
path = '/live'
signature_data = method + timestamp + path
signature = generate_signature(api_secret, signature_data)

ws = websocket.WebSocketApp('wss://socket.india.delta.exchange')
ws.send(json.dumps({
    "type": "auth",
    "payload": {
        "api-key": api_key,
```

```
        "signature": signature,
        "timestamp": timestamp
    }
    )))
```

To unsubscribe from all private channels, just send a '**unauth**' message on the socket. This will automatically unsubscribe the connection from all authenticated channels.

```
ws.send(json.dumps({
    "type": 'unauth',
    "payload": {}
})))
```

Sample Python Code

Public Channels

Summary: The python script(right panel) connects to the Delta Exchange WebSocket to receive real-time market data.

- It opens a connection.
- Subscribes to `v2/ticker` (tickers data) and `candlestick_1m` (1 minute ohlc candlesticks) channels. (**MARK:BTCUSD** - mark price ohlc in candlesticks channel)
- When data arrives, it processes and prints it.
- If an error occurs, it prints an error message.
- If the connection closes, it notifies the user.
- The connection remains open indefinitely to keep receiving updates.

```
import websocket
import json

# production websocket base url
WEBSOCKET_URL = "wss://socket.india.delta.exchange"

def on_error(ws, error):
    print(f"Socket Error: {error}")

def on_close(ws, close_status_code, close_msg):
    print(f"Socket closed with status: {close_status_code} and message: {close_msg}")

def on_open(ws):
    print(f"Socket opened")
    # subscribe tickers of perpetual futures - BTCUSD & ETHUSD, call option C-BTC-95200-200225 and put option - P-BTC-95200-200225
    subscribe(ws, "v2/ticker", ["BTCUSD", "ETHUSD", "C-BTC-95200-200225", "P-BTC-95200-200225"])
    # subscribe 1 minute ohlc candlestick of perpetual futures - MARK:BTCUSD(mark price) & ETHUSD(ltp), call option C-BTC-95200-200225(ltp) and put option - P-BTC-95200-200225
    subscribe(ws, "candlestick_1m", ["MARK:BTCUSD", "ETHUSD", "C-BTC-95200-200225", "P-BTC-95200-200225"])

def subscribe(ws, channel, symbols):
    payload = {
        "type": "subscribe",
        "payload": {
            "channels": [
                {
```



```

        "name": channel,
        "symbols": symbols
    }
]
}
}

ws.send(json.dumps(payload))

def on_message(ws, message):
    # print json response
    message_json = json.loads(message)
    print(message_json)

if __name__ == "__main__":
    ws = websocket.WebSocketApp(WEB_SOCKET_URL, on_message=on_message, on_error=on_error, on_close=on_close)
    ws.on_open = on_open
    ws.run_forever() # runs indefinitely

```

Private Channels

Summary: The python script(right panel) connects to the Delta Exchange WebSocket to receive real-time market data.

- It opens a connection.
- Sends authentication payload over socket with api_key, signature & timestamp.
- When authentication update arrives, it checks for success and then sends subscription for `orders` and `positions` channels for all contracts.
- Prints all other updates in json format.
- If an error occurs, it prints an error message.
- If the connection closes, it notifies the user.
- The connection remains open indefinitely to keep receiving updates.

```

import websocket
import hashlib
import hmac
import json
import time

# production websocket base url and api keys/secrets
WEB_SOCKET_URL = "wss://socket.india.delta.exchange"
API_KEY = 'a207900b7693435a8fa9230a38195d'
API_SECRET = '7b6f39dcf660ec1c7c664f612c60410a2bd0c258416b498bf0311f94228f'

def on_error(ws, error):
    print(f"Socket Error: {error}")

def on_close(ws, close_status_code, close_msg):
    print(f"Socket closed with status: {close_status_code} and message: {close_msg}")

def on_open(ws):
    print(f"Socket opened")
    # api key authentication
    send_authentication(ws)

def send_authentication(ws):
    method = 'GET'
    timestamp = str(int(time.time()))
    path = '/live'
    signature_data = method + timestamp + path
    signature = generate_signature(API_SECRET, signature_data)

```

```

ws.send(json.dumps({
    "type": "auth",
    "payload": {
        "api-key": API_KEY,
        "signature": signature,
        "timestamp": timestamp
    }
}))

def generate_signature(secret, message):
    message = bytes(message, 'utf-8')
    secret = bytes(secret, 'utf-8')
    hash = hmac.new(secret, message, hashlib.sha256)
    return hash.hexdigest()

def on_message(ws, message):
    message_json = json.loads(message)
    # subscribe private channels after successful authentication
    if message_json['type'] == 'success' and message_json['message'] == 'Authenticated':
        # subscribe orders channel for order updates for all contracts
        subscribe(ws, "orders", ["all"])
        # subscribe positions channel for position updates for all contracts
        subscribe(ws, "positions", ["all"])
    else:
        print(message_json)

def subscribe(ws, channel, symbols):
    payload = {
        "type": "subscribe",
        "payload": {
            "channels": [
                {
                    "name": channel,
                    "symbols": symbols
                }
            ]
        }
    }
    ws.send(json.dumps(payload))

if __name__ == "__main__":
    ws = websocket.WebSocketApp(WEB_SOCKET_URL, on_message=on_message, on_error=on_error, on_close=on_close)
    ws.on_open = on_open
    ws.run_forever() # runs indefinitely

```

Detecting Connection Drops

Some client libraries might not detect connection drops properly. We provide two methods for the clients to ensure they are connected and getting subscribed data.

Heartbeat (Recommended)

The client can enable heartbeat on the socket. If heartbeat is enabled, the server is expected to periodically send a heartbeat message to the client. Right now, the heartbeat time is set to 30 seconds.

HOW TO IMPLEMENT ON CLIENT SIDE

- Enable heartbeat (check sample code) after each successful socket connection
- Set a timer with duration of 35 seconds (We take 5 seconds buffer for heartbeat to arrive).
- When you receive a new heartbeat message, you reset the timer
- If the timer is called, that means the client didn't receive any heartbeat in last 35 seconds. In this case, the client should exit the existing connection and try to reconnect.

```
// Enable Heartbeat on successful connection
ws.send({
  "type": "enable_heartbeat"
})

// Disable Heartbeat
ws.send({
  "type": "disable_heartbeat"
})

// Sample Heartbeat message received periodically by client
{
  "type": "heartbeat"
}
```

Ping/Pong

The client can periodically (~ every 30 seconds) send a ping frame or a raw ping message and the server will respond back with a pong frame or a raw pong response. If the client doesn't receive a pong response in next 5 seconds, the client should exit the existing connection and try to reconnect.

```
// Ping Request
ws.send({
  "type": "ping"
})

// Pong Response
ws.send({
  "type": "pong"
})
```

Public Channels

v2 ticker

The ticker channel provides **price change data** for the last **24 hrs** (rolling window). It is published every **5 seconds**.

To subscribe to the ticker channel, you need to send the list of **symbols** for which you would like to receive updates.

You can also subscribe to ticker updates for a **category of products** by sending a list of category names. For example, to receive updates for **put options** and **futures**, use the following format:

```
{"symbols": ["put_options", "futures"]}
```

If you would like to subscribe to all listed contracts, pass:

```
{ "symbols": [ "all" ] }
```

Important:

If you subscribe to the ticker channel without specifying a symbols list, you will **not** receive any data.

Ticker Sample

```
// Subscribe to specific symbol
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "v2/ticker",
        "symbols": [
          "BTCUSD"
        ]
      }
    ]
  }
}
```

```
// Subscribe to all symbols
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "v2/ticker",
        "symbols": [
          "all"
        ]
      }
    ]
  }
}
```

```
// Response
{
  "open": 0.0001347, // The price at the beginning of the 24-hour period
  "close": 0.0001327, // The price at the end of the 24-hour period
  "high": 0.0001359, // The highest price during the 24-hour period
  "low": 0.0001323, // The lowest price during the 24-hour period
  "mark_price": "0.0001325", // The current market price
  "mark_change_24h": "-0.1202", // Percentage change in market price over the last 24 hours
  "oi": "812.6100", // Open interest, indicating the total number of outstanding contracts
  "product_id": 27, // The unique identifier for the product
  "quotes": {
    "ask_iv": "0.25", // Implied volatility for the ask price (if available)
    "ask_size": "922", // The size of the ask (the amount available for sale)
    "best_ask": "3171.5", // The best ask price (the lowest price at which the asset is being offered)
    "best_bid": "3171.4", // The best bid price (the highest price a buyer is willing to pay)
    "bid_iv": "0.25", // Implied volatility for the bid price (if available)
    "bid_size": "191", // The size of the bid (the amount a buyer is willing to purchase)
    "impact_mid_price": "61200", // Mid price impact, if available (the price midpoint between the best bid and ask)
    "mark_iv": "0.29418049" // Mark volatility (volatility of the asset used for mark price calculation)
  },
  "greeks": { // Options-related metrics, will be null for Futures and Spot products
    "delta": "0.01939861", // Rate of change of the option price with respect to the underlying asset's price
    "gamma": "0.00006382", // Rate of change of delta with respect to the underlying asset's price
    "rho": "0.00718630", // Rate of change of option price with respect to interest rate
    "spot": "63449.5", // The current spot price of the underlying asset
    "theta": "-81.48397021", // Rate of change of option price with respect to time (time decay)
  }
}
```

```

    "vega": "0.72486575" // Sensitivity of the option price to volatility changes
  },
  "size": 1254631, // Number of contracts traded
  "spot_price": "0.00001326", // Spot price at the time of the ticker
  "symbol": "BTCUSD", // The symbol of the contract
  "timestamp": 1595242187705121, // The timestamp of the data (in microseconds)
  "turnover": 16.805033569999996, // The total turnover in the settling symbol
  "turnover_symbol": "BTC", // The symbol used for settling
  "turnover_usd": 154097.09108233, // The turnover value in USD
  "volume": 1254631 // Total volume, defined as contract value * size
}

```

L1_orderbook

L1_orderbook channel provides level1 orderbook updates. You need to send the list of symbols for which you would like to subscribe to L1 orderbook. You can also subscribe to orderbook updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`. If you would like to subscribe for all the listed contracts, pass: `{"symbols": ["all"]}`. Please note that if you subscribe to L1 channel without specifying the symbols list, you will not receive any data.

Publish interval: 100 millisecs

Max interval (in case of same data): 5 secs

L1 Orderbook Sample

```

//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "l1_orderbook",
        "symbols": [
          "ETHUSD"
        ]
      }
    ]
  }
}

```

```

// l1_orderbook Response
{
  "ask_qty": "839",
  "best_ask": "1211.3",
  "best_bid": "1211.25",
  "bid_qty": "772",
  "last_sequence_no": 1671603257645135,
  "last_updated_at": 1671603257623000,
  "product_id": 176, "symbol": "ETHUSD",
  "timestamp": 1671603257645134,
  "type": "l1_orderbook"
}

```

l2_orderbook

l2_orderbook channel provides the complete level2 orderbook for the sepecified list of symbols at a pre-determined frequency. The frequency of updates may vary for different symbols. You can only subscribe to upto 20 symbols on a single connection. Unlike L1 orderbook channel, L2 orderbook channel does not accept product category names or "all" as valid symbols. Please note that if you subscribe to L2 channel without specifying the symbols list, you will not receive any data.

Publish interval: 1 sec

Max interval (in case of same data): 10 secs

L2 Orderbook Sample

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "l2_orderbook",
        "symbols": [
          "ETHUSD"
        ]
      }
    ]
  }
}
```

```
// l2_orderbook Response
{
  "type": "l2_orderbook",
  "symbol": "ETHUSD",
  "product_id": 176,
  "buy": [
    {
      "limit_price": "101.5",
      "size": 10,           // For Futures & Options: number of contracts integer. Spot product: Asset token quantity in string.
      "depth": "10"        // total size from best bid
    },
  ],
  "sell": [
    {
      "limit_price": "102.0",
      "size": 20,
      "depth": "20"        // total size from best ask
    },
  ],
  "last_sequence_no": 6435634,
  "last_updated_at": 1671600133884000,
  "timestamp": 1671600134033215,
}
```

l2_updates

l2_updates channel provides initial snapshot and then incremental orderbook data. The frequency of updates may vary for different symbols. You can only subscribe to upto 100 symbols on a single connection. l2_updates channel does not accept product category names or "all" as valid symbols. Please note that if you subscribe to l2_updates channel without specifying the symbols list, you will not receive any

data.

Publish interval: 100 millisecs

"action"="update" messages wont be published till there is an orderbook change.

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "l2_updates",
        "symbols": [
          "BTCUSD"
        ]
      }
    ]
  }
}

// Initial snapshot response
{
  "action": "snapshot",
  "asks": [[["16919.0", "1087"], ["16919.5", "1193"], ["16920.0", "510"]],
  "bids": [[["16918.0", "602"], ["16917.5", "1792"], ["16917.0", "2039"]],
  "timestamp": 1671140718980723,
  "sequence_no": 6199,
  "symbol": "BTCUSD",
  "type": "l2_updates",
  "cs": 2178756498
}

// Incremental update response
{
  "action": "update",
  "asks": [[["16919.0", "0"], ["16919.5", "710"]],
  "bids": [[["16918.5", "304"]],
  "sequence_no": 6200,
  "symbol": "BTCUSD",
  "type": "l2_updates",
  "timestamp": 1671140769059031,
  "cs": 3409694612
}

// Error response
{
  "action": "error",
  "symbol": "BTCUSD",
  "type": "l2_updates",
  "msg": "Snapshot load failed. Verify if product is live and resubscribe after a few secs."
}
```

HOW TO MAINTAIN ORDERBOOK LOCALLY USING THIS CHANNEL:

- 1) When you subscribe to this channel, the first message with "action"= "snapshot" resembles the complete l2_orderbook at this time. "asks" and "bids" are arrays of ["price", "size"]. (size is number of contracts at this price)
- 2) After the initial snapshot, messages will be with "action" = "update", resembling the difference between current and previous orderbook state. "asks" and "bids" are arrays of ["price", "new size"]. "asks" are sorted in increasing order of price. "bids" are sorted in decreasing order of price. This is true for both "snapshot" and "update" messages.
- 3) "sequence_no" field must be used to check if any messages were dropped. "sequence_no" must be +1 of the last message. e.g. In the snapshot message it is 6199, and the update message has 6200. The next update message must have 6201. In case of sequence_no mismatch, resubscribe to the channel, and start from the beginning.

4) If sequence_no is correct, edit the in-memory orderbook using the "update" message.

Case 1: price already exists, new size is 0 -> Delete this price level.

Case 2: price already exists, new size isn't 0 -> Replace the old size with new size.

Case 3: price doesn't exist -> insert the price level.

e.g. for the shown snapshot and update messages to create the new orderbook: in the ask side, price level of "16919.0" will be deleted. Size at price level "16919.5" will be changed from "1193" to "710". In the bids side there was no price level of "16918.5", so add a new level of "16918.5" of size "304". Other price levels from the snapshot will remain the same.

5) If "action":"error" message is received, resubscribe this symbol after a few seconds. Can occur in rare cases, e.g. Failed to send "action":"snapshot" message after subscribing due to a race condition, instead an "error" message will be sent.

Checksum: Using this, users can verify the accuracy of orderbook data created using l2_updates. checksum is the "cs" key in the message payload.

Steps to calculate checksum:

1) Edit the old in-memory orderbook with the "update" message received.

2) Create asks_string and bids_string as shown below. where priceN = price at Nth level, sizeN = size at Nth level. Asks are sorted in increasing order and bids in decreasing order by price.

asks_string = price0:size0,price1:size1,...,price9:size9

bids_string = price0:size0,price1:size1,...,price9:size9

checksum_string = asks_string + "|" + bids_string

Only consider the first 10 price levels on both sides. If orderbook has less than 10 levels, use only them.

e.g. If after applying the update, the new orderbook becomes ->

asks = [{"100.00", "23"}, {"100.05", "34"}]

bids = [{"99.04", "87"}, {"98.65", "102"}, {"98.30", "16"}]

checksum_string = "100.00:23,100.05:34|99.04:87,98.65:102,98.30:16"

3) Calculate the CRC32 value (32-bit unsigned integer) of checksum_string. This should be equal to the checksum provided in the "update" message.

all_trades

all_trades channel provides a real time feed of all trades (fills). You need to send the list of symbols for which you would like to subscribe to all trades channel. After subscribing to this channel, you get a snapshot of last 50 trades and then trade data in real time. You can also subscribe to all trades updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`. If you would like to subscribe for all the listed contracts, pass: `{"symbols": ["a11"]}`. Please note that if you subscribe to all_trades channel without specifying the symbols list, you will not receive any data.

All Trades Sample

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "all_trades",
        "symbols": [
          "BTCUSD"
        ]
      }
    ]
  }
}
```



```
// All Trades Response Snapshot
{
  "symbol": "BTCUSD",
  "type": "all_trades_snapshot",      // "type" is not "all_trades"
  "trades": [                       // Recent trades list
    {
      "buyer_role": "maker",
      "seller_role": "taker",
      "size": 53,                   // size in contracts
      "price": "25816.5",
      "timestamp": 1686577411879974 // time of the trade.
    },
    // More recent trades.
  ]
}
```

```
// All Trades Response
{
  "symbol": "BTCUSD",
  "price": "25816.5",
  "size": 100,
  "type": "all_trades",
  "buyer_role": "maker",
  "seller_role": "taker",
  "timestamp": 1686577411879974
}
```

mark_price

mark_price channel provides mark price updates at a fixed interval. This is the price on which all open positions are marked for liquidation. Please note that the product symbol is prepended with a "MARK:" to subscribe for mark price.

You need to send the list of symbols for which you would like to subscribe to mark price channel. You can also subscribe to mark price updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`.

If you would like to subscribe for all the listed contracts, pass: `{"symbols": ["all"] }`.

You can also subscribe to a Options chain, by passing 'Asset-Expiry', e.g. `{"symbols": ["BTC-310524"] }` will subscribe to all BTC Options expiring on 31st May 2024.

Please note that if you subscribe to mark price channel without specifying the symbols list, you will not receive any data.

Publish interval: 2 secs.

Mark Price Sample

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "mark_price",
        "symbols": [
          "MARK:C-BTC-13000-301222"
        ]
      }
    ]
  }
}
```

```
}  
}
```

```
// Mark Price Response  
{  
  "ask_iv":null,  
  "ask_qty":null,  
  "best_ask":null,  
  "best_bid":"9532",  
  "bid_iv":"5.000",  
  "bid_qty":"896",  
  "delta":"0",  
  "gamma":"0",  
  "implied_volatility":"0",  
  "price":"3910.088012",  
  "price_band":{"lower_limit":"3463.375340559572217228510815","upper_limit":"4354.489445440427782771489185"},  
  "product_id":39687,  
  "rho":"0",  
  "symbol":"MARK:C-BTC-13000-301222",  
  "timestamp":1671867039712836,  
  "type":"mark_price",  
  "vega":"0"  
}
```

candlesticks

This channel provides last ohlc candle for given time resolution. Traded price candles and Mark Price candles data can be received by sending appropriate symbol string. "product_symbol" gives traded_price candles, and "MARK:product_symbol" gives mark_price candles. e.g. symbols: ["BTCUSD"] gives you Traded Price candlestick data for BTCUSD
symbols: ["MARK:C-BTC-75000-310325"] gives you Mark Price candlestick data for C-BTC-75000-310325

Subscribe to **candlestick_\${resolution}** channel for updates.

List of supported resolutions ["1m","3m","5m","15m","30m","1h","2h","4h","6h","12h","1d","1w","2w","30d"]

You need to send the list of symbols for which you would like to subscribe to candlesticks channel. You can also subscribe to candlesticks updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`. Please note that if you subscribe to candlesticks channel without specifying the symbols list, you will not receive any data.

OHLC candles update sample

```
Sample Subscribe Request  
{  
  "name": "candlestick_1m",      // "candlestick_" + resolution  
  "symbols": [ "BTCUSD", "MARK:ETHUSD" ] // gives BTCUSD traded price, ETHUSD mark price candle data.  
}
```

```
Sample feed response  
  
{  
  "candle_start_time": 1596015240000000,  
  "close": 9223,  
  "high": 9228,  
  "low": 9220,  
  "open": 9221,
```

```
"resolution": "1m",
"symbol": "BTCUSD",
"timestamp": 1596015289339699,
"type": "candlestick_1m",
"volume": 1.2           // volume not present in Mark Price candlestick
}
```

spot_price

spot_price channel provides a real time feed of the underlying index prices. Specifying symbols when subscribing to spot_price is necessary to receive updates. No updates are sent for symbol: **"all"**

Spot Price Sample

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "spot_price",
        "symbols": [
          ".DEBNBBTC"
        ]
      }
    ]
  }
}
```

```
// Spot Price Response
{
  "symbol": ".DEBNBBTC",
  "price": "0.0014579",
  "type": "spot_price"
}
```

v2/spot_price

v2/spot_price channel publishes data of underlying index prices at a fixed interval. Specifying symbols when subscribing to v2/spot_price is necessary to receive updates. No updates are sent for symbol: **"all"**

Publish interval: 1 sec

v2/spot_price Subscribe

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "v2/spot_price",

```

```
        "symbols": [
            ".DEETHUSD"
        ]
    }
]
}
```

```
// Response
{
    "s": ".DEETHUSD",    # spot index symbol
    "p": 1349.3412141,   # spot price
    "type": "v2/spot_price"
}
```

spot_30mtwap_price

spot_30mtwap_price channel provides a real time feed of the 30 min twap of underlying index prices. This is the price used for settlement of options. Specifying symbols when subscribing to spot_30mtwap_price is necessary to receive updates. No updates are sent for symbol: **"all"**

Spot Price 30mtwap Sample

```
//Subscribe
{
    "type": "subscribe",
    "payload": {
        "channels": [
            {
                "name": "spot_30mtwap_price",
                "symbols": [
                    ".DEXBTUSD"
                ]
            }
        ]
    }
}
```

```
// Spot 30 minutes twap Price Response
{
    "symbol": ".DEXBTUSD",
    "price": "0.0014579",
    "type": "spot_30mtwap_price",
    "timestamp": 1561634049751430
}
```

funding_rate

funding_rate channel provides a real time feed of funding rates for perpetual contracts.

You need to send the list of symbols for which you would like to subscribe to funding rate channel. You can also subscribe to funding rate updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`. If you would like to subscribe for all the listed contracts, pass: `{"symbols": ["all"] }`. Please note that if you subscribe to funding rate channel without specifying the symbols list, you will not receive any data.

Funding Rate Sample

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "funding_rate",
        "symbols": [
          "BTCUSD"
        ]
      }
    ]
  }
}
```

```
// Funding Rate Response
{
  "symbol": "BTCUSD",
  "product_id": 139,
  "type": "funding_rate",
  "funding_rate": 0.005701298078111892, // %
  "funding_rate_8h": 0.005701298078111892, // %
  "next_funding_realization": 1683734400000000, // %
  "predicted_funding_rate": 0.007221329334075148, // in us
  "timestamp": 1683711930547419 // in us
}
```

product_updates

This channel provides updates when markets are disrupted and resumed. On opening, we conduct a single price auction and auction starting and finish events are also published on this channel. To subscribe, you dont need to pass the symbol list. This channel automatically subscribes to all markets by default.

Product Updates Sample

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "product_updates"
      }
    ]
  }
}
```

```
// Market Disruption Response
{
```

```

    "type": "product_updates",
    "event": "market_disruption",
    "product": {
      "id": 17,
      "symbol": "NEOUSDQ",
      "trading_status": "disrupted_cancel_only",
    },
    "timestamp": 1561634049751430,
  }

// Auction Started Response
{
  "type": "product_updates",
  "event": "start_auction",
  "product": {
    "id": 17,
    "symbol": "NEOUSDQ",
    "trading_status": "disrupted_post_only",
  },
  "timestamp": 1561634049751430,
}

// Auction Finished Response
{
  "type": "product_updates",
  "event": "finish_auction",
  "product": {
    "id": 17,
    "symbol": "NEOUSDQ",
    "trading_status": "operational",
  },
  "timestamp": 1561634049751430,
}

```

MARKET DISRUPTION

When markets are disrupted, orderbook enters into cancel only mode. You can refer to "trading_status" field in product info to determine this. In cancel only mode, you can only cancel your orders. No matching happens in this mode.

AUCTION STARTED

When markets need to come up, we conduct a single price auction. In this case, orderbook enters into post only mode. In post only mode, you can post new orders, cancel existing orders, add more margin to open positions. No matching happens in this mode. It is possible to see an overlap between asks and bids during this time.

AUCTION FINISHED

When auction finishes, markets enter into operational mode and trading continues as usual.

You can read more about the single price auction [here](#)

announcements

This channel provides updates on system wide announcements like scheduled maintenance, maintenance started etc. No need to pass any symbols while subscribing to this channel. Below are types and examples of messages sent for more details:

1. "event": "maintenance_scheduled" is sent when maintenance is scheduled. This is around 6 to 24 hours before the actual maintenance. Contains estimated start and finish time for maintenance.

2. "event": "maintenance_started" is sent when maintenance actually starts and markets are disrupted. Contains estimated finish time for maintenance.

3. "event": "maintenance_finished" is sent when maintenance finishes and market is set in auction node for short duration; then trading is resumed.

Announcements Sample

```
//Subscribe
{
  "type": "subscribe",
  "payload": {
    "channels": [
      {
        "name": "announcements"
      }
    ]
  }
}
```

```
// Maintenance Scheduled Response
{
  "type": "announcements",
  "event": "maintenance_scheduled",
  "maintenance_start_time": 1742301546000000, // estimated maintenance start time in microseconds
  "maintenance_finish_time": 1742301647000000 // estimated finish time
}

```json
// Maintenance Started Response
{
 "type": "announcements",
 "event": "maintenance_started",
 "maintenance_finish_time": 1561638049751430, // estimated finish time in microseconds.
 "timestamp": 1561634049751430,
}

// Maintenance Finished Response
{
 "type": "announcements",
 "event": "maintenance_finished",
 "timestamp": 1561634049751430,
}
```

---

# Private Channels

---

Private channels require clients to authenticate.

---

## Margins

This channel provides updates on wallet balances. Updates are sent for a specific asset whenever there is a change in wallet balances and margins for that asset.

### Margins Sample

```
//Subscribe
{
 "type": "subscribe",
 "payload": {
 "channels": [
 {
 "name": "margins"
 }
]
 }
}
```

```
// margin update
{
 "action": "update",
 "asset_id": 2, // BTC
 "asset_symbol": "BTC", // BTC
 "available_balance": "9.385", // Available balance for trading = balance - blocked_margin
 "available_balance_for_robo": "9.385", // Available balance for robo trading = balance - blocked_margin
 "balance": "10", // Wallet balance = deposits - withdrawals + realised_cashflows
 "blocked_margin": "0.615", // Total Margin blocked
 "commission": "0.001", // Commissions blocked in isolated margined positions and orders
 "cross_asset_liability": "0", // Liability between asset in cross margin mode
 "cross_commission": "0.002", // Commissions blocked in cross margined positions and orders
 "cross_locked_collateral": "0.003", // Balance blocked for collateral
 "cross_order_margin": "0.004", // Margin blocked in cross margined open orders
 "cross_position_margin": "0.005", // Margin blocked in cross margined positions
 "id": 1, // Wallet Id
 "interest_credit": "0", // Interest credited
 "order_margin": "0.1", // Margin blocked in isolated margined open orders
 "pending_referral_bonus": "0", // Bonus pending
 "pending_trading_fee_credit": "0", // Pending trading fee to credit
 "portfolio_margin": "0.2", // Margin blocked for portfolio margined positions and orders. Same as blocked margin in portfolio margins channel.
 "position_margin": "0.3", // Margin blocked in isolated margined positions
 "robo_trading_equity": "0", // Equity for robo trading
 "timestamp": 1719397302569921, // Unix timestamp in microseconds
 "trading_fee_credit": "0", // Trading fee credited
 "type": "margins", // Margins channel
 "unvested_amount": "0", // Amount locked. Relevant only for DETO
 "user_id": 1 // User id
}
```

## Positions

This channel provides updates whenever there is any change in your open positions.

A snapshot of current open position will be sent after subscribing a symbol, incremental updates will be sent on trade executions. You need to send the list of symbols for which you would like to subscribe to positions channel. You can also subscribe to positions updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{ "symbols": [ "put_options", "futures" ] }`. If you would like to subscribe for all the listed contracts, pass: `{ "symbols": [ "all" ] }`. Please note that if you subscribe to positions channel without specifying the symbols list, you will not receive any data.

### Positions Sample

```
//Subscribe
{
 "type": "subscribe",
```



```

 "payload": {
 "channels": [
 {
 "name": "positions",
 "symbols": ["BTCUSD"]
 }
]
 }
 }
}

```

//Subscribe for all the symbols

```

{
 "type": "subscribe",
 "payload": {
 "channels": [
 {
 "name": "positions",
 "symbols": ["all"]
 }
]
 }
}

```

// Position update

```

{
 "type": "positions",
 "action": "", // "create"/"update"/"delete"
 "reason": "", // null, "auto_topup"
 "symbol": "BTCUSD", // Product Symbol
 "product_id": 1, // Product ID
 "size": -100, // Position size, if > 0 -> long else short
 "margin": "0.0121", // Margin blocked in the position
 "entry_price": "3500.0", // Avg Entry price of the position
 "liquidation_price": "3356.0", // Liquidation trigger price
 "bankruptcy_price": "3300.0", // Bankruptcy Price
 "commission": "0.00001212" // Commissions blocked for closing the position
}

```

//Snapshot

```

{
 "result":[
 {
 "adl_level": "4.3335",
 "auto_topup": false,
 "bankruptcy_price": "261.82",
 "commission": "17.6571408",
 "created_at": "2021-04-29T07:25:59Z",
 "entry_price": "238.023457888493475682",
 "liquidation_price": "260.63",
 "margin": "4012.99",
 "product_id": 357,
 "product_symbol": "ZECUSD",
 "realized_funding": "-3.08",
 "realized_pnl": "6364.57",
 "size": -1686,
 "updated_at": "2021-04-29T10:00:05Z",
 "user_id": 1,
 "symbol": "ZECUSD"
 }
],
 "success": true,
 "type": "positions",
 "action": "snapshot"
}

```

# Orders

Channel provides updates when any order is updated for any action such as fill, quantity change. Need to pass list of product symbols while subscribing.

A snapshot of all open/pending orders will be sent after subscribing a symbol. And all incremental updates will be sent on create/update/delete of orders

All updates including snapshot will have incremental seq\_id. seq\_id is separate for each symbol.

Any of the following events can be tracked by the reason field in this channel

- fill
- stop\_update
- stop\_trigger
- stop\_cancel
- liquidation
- self\_trade

You need to send the list of symbols for which you would like to subscribe to orders channel. You can also subscribe to orders updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`. If you would like to subscribe for all the listed contracts, pass: `{"symbols": ["all"]}`. Please note that if you subscribe to orders channel without specifying the symbols list, you will not receive any data.

## Orders Sample

```
//Subscribe
{
 "type": "subscribe",
 "payload": {
 "channels": [
 {
 "name": "orders",
 "symbols": ["BTCUSD"]
 }
]
 }
}
```

```
// Order update

{
 "type": "orders",
 "action": "create", // "create"/"update"/"delete"
 "reason": "", // "fill"/"stop_update"/"stop_trigger"/"stop_cancel"/"liquidation"/"self_trade"/null
 "symbol": "BTCUSD", // Product Symbol
 "product_id": 27, // Product ID
 "order_id": 1234, // Order id
 "client_order_id": "", // Client order id
 "size": 100, // Order size
 "unfilled_size": 55, // Unfilled size
 "average_fill_price": "8999.00", // nil for unfilled orders
 "limit_price": "9000.00", // Price of the order
 "side": "buy", // Order side (buy or sell)
 "cancellation_reason": "cancelled_by_user", // Cancellation reason in case of cancelled order, null otherwise
 "stop_order_type": "stop_loss_order", // If a Stop Order -> "stop_loss_order"/"take_profit_order", null otherwise
 "bracket_order": false, // true for a bracket_order, false otherwise
 "state": "open", // "open"/"pending"/"closed"/"cancelled"
 "seq_no": 1, // Incremental sequence number
 "timestamp": 1594105083998848, // Unix timestamp in microseconds
}
```

```

"stop_price": "9010.00", // stop_price of stop order
"trigger_price_max_or_min": "9020.00", // for trailing stop orders
"bracket_stop_loss_price": "8090.00",
"bracket_stop_loss_limit_price": "8090.00",
"bracket_take_profit_price": "9020",
"bracket_take_profit_limit_price": "9020",
"bracket_trail_amount": "10.00"
}

```

```

// Snapshot
{
 "meta": {
 "seq_no": 7,
 "timestamp": 1594149235554045
 },
 "result": [
 {
 "id": 1592130,
 "limit_price": "9000",
 "order_type": "limit_order",
 "product_id": 13,
 "reduce_only": false,
 "side": "buy",
 "size": 1,
 "state": "open",
 "stop_order_type": null,
 "stop_price": null,
 "time_in_force": "gtc",
 "trail_amount": null,
 "unfilled_size": 1,
 "average_fill_price": "8999.00",
 "user_id": 1132
 }
],
 "success": true,
 "symbol": "BTCUSD",
 "type": "orders",
 "action": "snapshot"
}

```

## UserTrades

Please use "v2/user\_trades" channel for better latency.

Channel provides updates for fills. Need to pass list of product symbols while subscribing.

All updates will have incremental seq\_id. seq\_id is separate for each symbol.

Auto Deleverage Liquidations of a position can be tracked by reason: "adl" in the user\_trades channel. You need to send the list of symbols for which you would like to subscribe to user trades channel. You can also subscribe to user trades updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`. If you would like to subscribe for all the listed contracts, pass: `{ "symbols": ["all"] }`. Please note that if you subscribe to user trades channel without specifying the symbols list, you will not receive any data.

User Trades Sample

```
//Subscribe
{
 "type": "subscribe",
 "payload": {
 "channels": [
 {
 "name": "user_trades",
 "symbols": ["BNBBTC_30Nov"]
 }
]
 }
}
```

```
// user_trades
{
 "symbol": "BNBBTC_30Nov",
 "fill_id": "1234-abcd-qwer-3456",
 "reason": "normal", // "normal" or "adl"
 "product_id": 7,
 "type": "user_trades",
 "user_id": 1998,
 "order_id": 3283999,
 "side": "buy",
 "size": 190,
 "price": "0.00145791",
 "role": "taker",
 "client_order_id": "GA123",
 "timestamp": 1544091555086559,
 "seq_no": 1
}
```

## v2/user\_trades

Channel provides updates for fills. Need to pass list of product symbols while subscribing. This channel is similar to user\_trades channel, only difference is that, it is faster than user\_trades and doesn't contain commission data.

All updates will have incremental sequence\_id. sequence\_id is separate for each symbol, useful for identifying if any v2/user\_trades messages were missed/dropped. The sequence\_id will reset to 1 after our systems restart. (usually after maintainaince/market disruption).

Auto Deleverage Liquidations of a position can be tracked by reason: "adl". Liquidation of a position can be tracked by reason: "liquidation". You need to send the list of symbols for which you would like to subscribe to v2/user\_trades channel. You can also subscribe to v2/user\_trades updates for category of products by sending category-names. For example: to receive updates for put options and futures, refer this: `{"symbols": ["put_options", "futures"]}`.

If you would like to subscribe for all the listed contracts, pass: `{ "symbols": ["all"] }`.

Please note that if you subscribe to v2/user\_trades channel without specifying the symbols list, you will not receive any data.

### v2/user\_trades Sample

```
//Subscribe
{
 "type": "subscribe",
 "payload": {
 "channels": [
 {
 "name": "v2/user_trades",
 "symbols": ["BTCUSD"]
 }
]
 }
}
```

```
}
]
}
}
```

```
// v2/user_trades
{
 "type": "v2/user_trades",
 "sy": "BTCUSD", // symbol
 "f": "1234-abcd-qwer-3456", // fill_id
 "R": "normal", // reason: "normal", "adl", "liquidation"
 "u": 1998, // user_id
 "o": 3283999, // order_id
 "S": "buy", // side: "buy" or "sell"
 "s": 190, // size in contracts
 "p": "17289.2", // price
 "po": 5, // position (in contracts) after this fill.
 "r": "taker", // role: "taker" or "maker"
 "c": "GA123", // client_order_id
 "t": 1685794274866438, // timestamp of fill creation
 "se": 4 // incremental sequence_no
}
```

## PortfolioMargins

Channel provides updates for portfolio margin values of the selected sub-account. These updates are sent every 2 seconds. In case portfolio margin is not enabled on the selected sub-account, no updates will be sent on this channel.

For detailed description of portfolio margin please see user guide

UCF: is unrealised cashflows of your portfolio. These are the cashflows (negative for outgoing and positive for incoming) that will take place if all the positions in your portfolio are closed at prevailing mark prices.

### Portfolio Margin Sample

```
//Subscribe
{
 "type": "subscribe",
 "payload": {
 "channels": [
 {
 "name": "portfolio_margins",
 "symbols": [".DEXBTUSD"]
 }
]
 }
}
```

```
// portfolio margin update
{
 "type": "portfolio_margins",
 "user_id": 1,
 "asset_id": 2, // BTC
 "index_symbol": ".DEXBTUSD",
 "liquidation_risk": false,
```

```

"blocked_margin": "100", // Margin blocked for current portfolio. Same as portfolio_margin in margins channel.
"mm_wo_ucf": "80",
"mm_w_ucf": "80",
"im_wo_ucf": "100",
"im_w_ucf": "100",
"positions_up1": "0",
"risk_margin": "100",
"risk_matrix": {"down": [{"is_worst": false, "pn1": "230.03686162", "price_shock": "10"}], "unchanged": [{"is_worst": false, "pn1": "230.03686162", "price_shock": "10"}], "up": []},
"futures_margin_floor": "20",
"short_options_margin_floor": "20",
"long_options_margin_floor": "20",
"under_liquidation": false,
"commission": "3.444",
"margin_floor": "60",
"timestamp": "1544091555086559", //timestamp in microseconds
"margin_shortfall": "4.5" // key sent when liquidation_risk is true
}

```

Keys -

index\_symbol

This is the coin on which portfolio margin is enabled.

positions\_up1

This is unrealised cashflows (UCF) of your portfolio. These are the cashflows (negative for outgoing and positive for incoming) that will take place if all the positions in your portfolio are closed at prevailing mark prices. Unrealised cashflow is positive for long options and negative for short options.

im\_w\_ucf

This is the initial margin (IM) requirement for the portfolio. IM is computed as  $\max(\text{risk\_margin}, \text{margin\_floor}) - \text{UCF}$ .

If  $\text{UCF} > \max(\text{risk\_margin}, \text{margin\_floor})$  then IM is negative. Negative margin requirement results in increase in your balance available for trading.

If the Wallet Balance (ex spot orders) is less than IM then you would only be able to place orders that reduce the risk of the portfolio.

im\_wo\_ucf

This is IM without UCF.

mm\_w\_ucf

This is the maintenance margin (MM) requirement for the portfolio. MM is computed as  $80\% * \max(\text{risk\_margin}, \text{margin\_floor}) - \text{UCF}$ .

If the Wallet Balance (ex spot orders) is less than MM then the portfolio will go into liquidation.

mm\_wo\_ucf

This is MM without UCF.

commission

This is the trading fees blocked for the open orders/positions (for closing the positions) in the portfolio.

This is in addition to the IM requirement.

blocked\_margin

The margin actually blocked for your portfolio. If your Wallet Balance (ex spot orders) is greater than IM + commission then  $\text{blocked\_margin} = \text{IM} + \text{commissions}$ . Otherwise blocked\_margin is equal to the maximum amount we are able to block to meet the portfolio margin requirement.

If  $\text{blocked\_margin} < \text{MM}$  then the portfolio goes into liquidation.

liquidation\_risk

This flag indicates if the portfolio is at liquidation risk.

This flag is set to TRUE when  $\text{blocked\_margin} < \text{im\_w\_ucf} + \text{commissions}$ .

under\_liquidation

This flag is set to TRUE when the portfolio is under liquidation.

margin\_shortfall

This is the minimum topup amount needed to bring the portfolio out of liquidation risk state.

risk\_margin

The maximum likely loss of the portfolio under the various simulated stress scenarios.

risk\_matrix

Matrix showing the profit/loss of the portfolio under various simulated stress scenarios.

Profit/loss for each position and open order is computed with reference to the prevailing mark prices. Positive numbers indicate profit and negative numbers indicate loss.

margin\_floor

Margin Floor is the minimum risk\_margin required for a portfolio.

It is comprised of sum of futures\_margin\_floor, long\_options\_margin\_floor, short\_options\_margin\_floor

---

## MMP Trigger

Channel provides updates when MMP is triggered. Market maker protection is available to registered market makers by default. Others can reach out to support for getting access to MMP. More info [here](#).

### MMP Trigger Sample

```
//Subscribe
{
 "type": "subscribe",
 "payload": {
 "channels": [
 {
 "name": "mmp_trigger"
 }
]
 }
}
```

```
// mmp_trigger response
{
 "user_id": 1,
 "asset": "BTC",
 "frozen_till": 1561634049751430 # timestamp is microseconds, will be -1 if manual reset is enabled
}
```

---

## Changelog

---

### 01.06.25

1. Added reason:"liquidation" for v2/user\_trades websocket channel. Updated documentation for the v2/user\_trades channel.
2. Added Connection rate limit for websocket URL endpoint of 150 connections per IP address per 5 minutes. Updated this info under 'Websocket Info' tab.

---

### 21.03.25

1. Better documentation for Websocket public channel 'candlesticks', now clearly states even Mark Price candles can be fetched.
2. /positions REST API fixed documentation for query parameters.
3. Better documentation for Websocket public channel 'announcements'.

---

## 20.02.25

1. Removed Websocket RPC section. Use REST API endpoints with normal HTTP requests (RPC requests still work, it is recommended not to use them).
2. Fixed documentation: "User-Agent" header is necessary for authenticated HTTP requests.
3. Fixed documentation: Better examples and documentation under 'General Information', 'Authentication', etc tabs.
4. Created separate documentation site for Indian and Global site for easier understanding and better examples for both.

---

## 18.10.2024

1. Added clickable links to the corresponding API response json schemas under some API "Responses" table. Updated some schemas to match the response. (This is a documentation fix, no changes in API)
2. Added "Testnet-India" REST and Websocket host endpoints.
3. Changed "Rate Limits" description and added an example.

---

## 01.05.2024

1. Added "po" positions key in /v2/user\_trades websocket private channel.
2. Removed "fok" type for orders, as they are no longer supported.

---

## V2 Rest Api

Our v2 Api is significantly faster than the v1 api. Our focus while rebuilding v2 Apis was on the following

1. Remove Api gateway overheads as much as possible.
2. Remove overheads due to deep nesting in response payload.
3. Better Api structure to query only required data.

### New Response Format

```
// The new format supports sending meta data alongside response body.
// Success format
{
 success: true,
 result: {} // response body
 meta: {
 after: "...", // cursor for pagination, is returned in meta
 before: null,
 },
}

// Error Format
{
 success: false,
 error: {
 code: :insufficient_margin, // error code
 context: { // error context
 additional_margin_required: "0.121"
 }
 }
}
```



```
}
}
}
```

#### KEY API CHANGES

- We have completely removed nested product/asset payloads from live orders and live positions. This ensures the payload is light.
- Rate limiting now works on a fixed window instead of a rolling window.
- Ticker Api - now includes turnover in USD, mark price, spot price.
- Orderbook and trades are now returned in separate Apis.
- For supporting trading strategies which require latest positions, Now we have two different Apis to query position.

/v2/positions - returns only size and entry price. This should be used when you want to get the latest position, but dont need the margin dependent fields like liquidation price, bankruptcy price etc

/v2/positions/margined - returns all fields including margin dependent fields. When the position is updated due to a fill, changes might take some time to reflect in this Api.

- All Apis that support pagination now use cursor based pagination, instead of fixed page size pagination. Check more details in our python rest client docs

---

## New Socket Channels

- Socket Api interface hasn't changed much in terms of connection management and authentication.
- We have deprecated old channels and created new channels which make integration easier.
- To support easy management of live data, all private data channels now support initial snapshots and sequence numbers.

#### LIST OF NEW PUBLIC CHANNELS

- v2/ticker - now includes turnover in USD, mark price, spot price
- candlesticks - subscribe to ohlc candle updates for different resolutions
- all\_trades - subscribe to all public trades for a symbol

#### LIST OF NEW PRIVATE CHANNELS

- orders - subscribe to lifecycle of live orders
- user\_trades - subscribe to live user trades/fills feed
- positions - subscribe to position updates
- margins - get margin/wallet updates

---

## Security

We take the security, integrity, availability of our services, and the privacy of our users seriously. We appreciate all security concerns brought forth and are constantly striving to keep on top of the latest threats. Being proactive rather than reactive to emerging security issues is a fundamental belief at Delta Exchange. Every day new security issues and attack vectors are created. Delta Exchange strives to keep abreast of the latest state-of-the-art security developments by working with independent security researchers. We appreciate the community's efforts in creating a more secure world.

---

## Targets In scope

- [https://\\*.delta.exchange](https://*.delta.exchange)

Any domain/property of Delta Exchange Network not listed in the targets section is out of scope. This includes any/all subdomains not listed above.

---

## How to access

- You will access the Delta Exchange service using test accounts and unauthenticated guests.
- Please create a Delta Exchange test account on your own using your test email address. Your email must contain keyword 'test' for example *test@gmail.com*.
- All emails will go to the email address associated with your account. You will need to activate your account by confirming receipt of the activation email.
- NOTE: Once a vulnerability is found please file a submission immediately. Our security team will investigate and assess the impact.

---

## Reward range

### Focus Areas

- User Data / User information Leaks
- Injection attacks (Server/Client side)
- RCE(Remote Code Execution)
- Authentication bypass/validation (Client/Server side)
- Privilege escalation (Vertical/Horizontal)

Technical	Severity	Reward
P1	Critical	Decided by internal team, can be from (\$10-\$1000)
P2	Severe	Decided by internal team, can be from (\$10-\$1000)
P3	Moderate	\$10 - \$100

---

## Rules of engagement

We are interested in hearing about security issues in Production/Dev Delta Exchange environments. There are some things we explicitly ask you not to do

- Do not run automated scans without checking with us first. They are often very noisy.(If running any automated testing tools, be sure to keep well under 75-100 requests per second - otherwise you're likely to get locked out.)
- Do not test the physical security of Delta Exchange offices, employees, equipment, etc.
- Do not run Full fledged exploits which can cause application crashes and affect integrity of our active services. (If you believe you have a exploit that need serious fixes please email us, and we will provide you with said instance for said service.)
- Do not test using social engineering techniques (phishing, vishing, etc.)
- Do not test against any type of customer account without explicit permission from our side.
- Do not access, Destroy or otherwise negatively impact any residential or business customers, or customer data in any way.
- Do not perform DoS or DDoS attacks (Application level, Network Level DOS / DDOS / port flooding attacks are strictly not appreciated as this can cause delay in delivery of our services to our users we suggest you to not to use such methodologies).
- Do not engage In any way attack our end users, or in the trade of stolen user credentials.

- Interacting with real customers or real customer accounts is forbidden.

---

## Rules of reporting

We take our Internal process and workflow seriously, We have a dedicated security team working and testing round the clock, so we would like you to send your reports to only security@delta.exchange and follow below said rules or not following will void you from our bug bounty benefits, Also we would like you to encrypt emails sent to us with a PGP key provided below if the Vulnerability is Severe or Critical.

- We recommend you to use emails which contain test as a keyword in the email address for example test@gmail.com.
- For testing and reporting so that we can identify your activity on our environments, also whitelist for IDS/FDS blocks)
- Do not CC or tag other staff while reporting.
- Do not callout on social media or make blog posts to report or without reporting (this can lead to legal actions to be taken against you.)
- Do not discuss this with any, but only Delta Exchange technical staff.
- Do not send us external-links/executable/scripts in report if possible attach a text file or pdf. Without zipp'ing or rar'ing it.
- Screenshots are accepted if only in PNG and JIF formats for internal security reasons.
- POC Videos are accepted if only in MP4,AVI, WEBM, MOV formats for internal security reasons.
- Any or All reports must only reach security@delta.exchange.

---

## Recommended Reporting format

### Summary

Help us get an idea of what this vulnerability is about.

(eg. "hey i have found a xss on your server")

### Target

Select the vulnerable target Domain name / Subdomain name

(eg. "so and so on https://example.delta.exchange")

### Vulnerability details

What is the Bug/Vulnerability. And URL / Location of vulnerability .

(eg. because of unfiltered characters the url [/search.php?q=] path /search.php on q parameter)

### Description of Technical severity

Help us understand the bug/vulnerability technical details Describe the vulnerability and its impact.

(eg. client side executes the javascript which is rendering through /search.php?q=somescript)

### Recreation

Provide a proof of concept or replication steps.

(eg. steps to be used by our team in order to recreate the attack scenario)

### Additional information

Provide us with Request and Response dump / trace dump / HTTP request

## Attachments (recommended)

Attach proof-of-concept scripts, screenshots, screen recordings, etc.

---

## Ineligible issues

These issues Will be closed as out of scope hence not rewardable.

- Theoretical vulnerabilities without actual proof of concept
- Open redirects (through headers and parameters) / Lack of security speed bump when leaving the site.
- Internal IP address / version disclosure.
- Email verification deficiencies, expiration of password reset links, and password complexity policies
- Invalid or missing SPF (Sender Policy Framework) records (incomplete or missing SPF/DKIM/DMARC)
- Click jacking/UI redressing with minimal security impact
- Text/code injection without any impact.
- Email or mobile enumeration (E.g. the ability to identify emails via password reset)
- Information disclosure with minimal security impact (E.g. stack traces, path disclosure, directory listings, logs)
- Internally known issues, duplicate issues, or issues which have already been made public
- Rate limiting issues / Tab-nabbing
- Non url Selfless / HTMLi
- Known CVE without proper testing.
- Vulnerabilities only exploitable on out-of-date browsers or platforms
- CSRF issues that don't impact the integrity of an account (e.g. log in or out, contact forms and other publicly accessible forms)
- Vulnerabilities related to auto-fill web forms
- Use of known vulnerable libraries without actual proof of concept
- Lack of security flags in cookies
- Issues related to unsafe SSL/TLS cipher suites or protocol version
- Session expiry / Cookie issues / Content spoofing
- Cache-control related issues
- Missing security headers that do not lead to direct exploitation
- CSRF with negligible security impact (E.g. adding to favorites, adding to cart, subscribing to a non-critical feature)
- Vulnerabilities that require root/jailbreak
- Vulnerabilities that require physical access to a user's device
- Issues that have no security impact (E.g. Failure to load a web page)
- Phishing (E.g. HTTP Basic Authentication Phishing)
- Any activity (like DoS/DDoS) that disrupts our services
- Installation Path Permissions
- Reports from automated tools or scans

Not following any one of the above rule will disqualify you from our bug bounty program.

If any doubts related to your submissions or creative dialogue please feel free to email [saurabh.goyal@delta.exchange](mailto:saurabh.goyal@delta.exchange) or [venkatesh.sharma@delta.exchange](mailto:venkatesh.sharma@delta.exchange).